

POLITECNICO
MILANO 1863

**SCUOLA DI INGEGNERIA
INDUSTRIALE E DELL'INFORMAZIONE**
Laurea Magistrale in Mechanical Engineering

**Development of MES integration
functionality for a Digital Twin in
manufacturing environment**

Supervisor: Ing. Luca Fumagalli
Co-Supervisor: Dott.ssa Elisa Negri

Candidate:
Stefano Berardi ID 876659

ACADEMIC YEAR 2017/2018

CONTENTS

List of Figures	i
List of Tables	iv
List of Acronyms	vi
ABSTRACT — ENGLISH	viii
ABSTRACT — ITALIAN	ix
EXECUTIVE SUMMARY	x
1 INTRODUCTION	1
1.1 Industry 4.0	1
1.2 CPS-based Automation Pyramid	2
1.3 Digital Twin	4
1.3.1 Digital Twin in Simulation	4
1.4 Manufacturing Execution System	5
1.4.1 MES in Information System Architecture	6
1.4.2 MES Functionalities	7
1.4.3 MES Benefits	8
1.4.4 MES Communication	9
2 LITERATURE REVIEW	11
2.1 Topics, motivations and aims	11
2.2 Literature Review Methodology	12
2.3 Literature Review Results	15
2.3.1 Analysis of papers	15
2.3.2 Definitions, Synonyms, Relationships	16
2.3.3 Enabling technologies and applications	19
3 DIGITAL TWIN FOR OPTIMIZATION AND CONTROL	24
3.1 Optimization Scenarios	25
3.2 Digital Shadow vs Digital Twin	27
3.3 Control Models and Autonomy	29
3.3.1 Autonomy	30
3.3.2 Control models	30
3.4 Integration of Digital Twin with MES	31
4 RESEARCH DESIGN	33
4.1 Research Gaps	33
4.2 Research Objectives	34
4.3 Practical Objectives and Application Case Study	35
5 PROPOSED MODEL DEVELOPMENT	38
5.1 Error States Management Framework	39
5.1.1 Nature of error states	39
5.1.2 The error states management framework	39
5.1.3 Simulation model proposal	41
5.2 Reactive Disassembly Framework	41
5.2.1 The reactive disassembly framework	42

5.2.2	Simulation model proposal	43
6	INDUSTRY 4.0 LABORATORY — ENVIRONMENT & TECHNOLOGIES	44
6.1	Laboratory Research Objectives	44
6.2	Production line	45
6.3	Product	48
6.4	Work Process	49
6.5	OPC Unified Architecture (UA)	53
6.5.1	OPC UA in I4.0 Laboratory	55
6.5.2	OPC UA in Matlab environment	57
6.5.3	OPC UA in Simulink environment	58
6.6	MES in I4.0 Lab: MES4	60
6.6.1	Order creation & tracking	60
6.6.2	PCB Box handling	62
6.6.3	PLC Simulator	63
6.7	TCP/IP communication protocol	64
6.7.1	Fields of application	65
6.7.2	Communication with MES	66
6.8	String based protocol	68
6.8.1	Definition of string	69
6.8.2	Communication with MES4	70
6.8.3	Syntax of the string for MES communication	72
7	MES INTEGRATION	76
7.1	Integration on Matlab	77
7.1.1	Sending information	78
7.1.2	Getting information	79
7.1.3	The multiple string issue	84
7.2	Integration on Simulink	86
7.2.1	Sending information	88
7.2.2	Getting information	89
7.2.3	Syntax adjustments	91
8	APPLICATIONS	93
8.1	Error states management	93
8.1.1	Error states list and framework application	94
8.1.2	Simulation Models	98
8.2	Disassembly Proposal	110
8.2.1	Camera Inspection	110
8.2.2	Simulation Model	111
8.2.3	Disassembly framework application	112
9	VALIDATION AND RESULTS	114
9.1	Results: Error states management	114
9.2	Results: Disassembly	116
9.3	Validation of model	117
10	CONCLUSIONS	118
10.1	Practical Objectives	118
10.2	Research Objectives	120

10.3 Future Developments	121
Bibliography	122
Appendix A STRINGS LIBRARY	129
Appendix B DATA TYPE ID IN LEVEL-2 MATLAB S-FUNCTION	133
Appendix C MATLAB SCRIPT	134
Acknowledgements	136

LIST OF FIGURES

Figure 1.1	Industrial Revolutions throughout the last centuries	1
Figure 1.2	Automation Pyramid representation based on IEC 62264 normative	2
Figure 1.3	CPS-based automation pyramid	3
Figure 1.4	MES Context Model	7
Figure 1.5	Communication paradigm between MES and external tools	9
Figure 2.1	Sample data flow	11
Figure 3.1	Literature study progress	24
Figure 3.2	Data Flow in a Digital Model [1]	28
Figure 3.3	Data Flow in a Digital Shadow [1]	28
Figure 3.4	Data Flow in a Digital Twin [1]	29
Figure 3.5	Notion of the digital twin in the lifecycle of a product [54]	30
Figure 4.1	Enhancement of the Digital Shadow to a Digital Twin [1]	34
Figure 4.2	Graphical representation of the objectives	35
Figure 4.3	Development phases of the work of thesis	36
Figure 5.1	Proposed framework of integrated management of error states with a digital simulation tool	40
Figure 5.2	Simulation model basic functions for application of error states framework	41
Figure 5.3	Proposed framework of integrated management of disassembly with a digital simulation tool	42
Figure 5.4	Simulation model basic functions for application of reactive disassembly framework .	43
Figure 6.1	Location of the I4.0 Lab at PoliMi	44
Figure 6.2	Layout of the line, with a number for each workstation.	46
Figure 6.3	Automated process structure	46
Figure 6.4	Product features	48
Figure 6.5	Detail of the working position for the Front Magazine station	49
Figure 6.6	Working assignment at Drilling station: first step	50
Figure 6.7	Working assignment at Drilling station: second step	50
Figure 6.8	Working assignment at Drilling station: third step	50

Figure 6.9	Working assignment at Drilling station: fourth step	51
Figure 6.10	Working assignment at Drilling station: fifth step	51
Figure 6.11	Working assignment at Drilling station: sixth step	51
Figure 6.12	Working assignment at Drilling station: seventh step	52
Figure 6.13	Working steps	52
Figure 6.14	OPC UA client/server connection	54
Figure 6.15	UAExpert software interface with detail on Data Access View (top side) and Address Space (left-hand side)	57
Figure 6.16	Level-2 MATLAB S-Function code to read the CarrierID value from the server	59
Figure 6.17	MES4 interfaces to create a production order and launch it in the production line	61
Figure 6.18	The window of MES4 that shows the processing of orders along the line	62
Figure 6.19	The <i>Utilities</i> interface of MES4	63
Figure 6.20	The MES4 interface that simulates information exchange with the PLC database	64
Figure 6.21	Alternative communication types between MES (left) and the PLCs (right)	67
Figure 7.1	Steps of the development of integration work	76
Figure 7.2	Manual sending of commands on Matlab from the user, when certain conditions are met	77
Figure 7.3	Operations performed by the "send" information model	78
Figure 7.4	Sample string use in Matlab to send an order to the MES	79
Figure 7.5	Operations performed by the "get" information model	80
Figure 7.6	Data contained into a TCP/IP object in Matlab	81
Figure 7.7	Example of the calling of the <i>BytesAvailable</i> field inside the TCP/IP object "t"	82
Figure 7.8	The <i>DataReceived</i> object in Matlab Workspace	83
Figure 7.9	Code to get information from a previously initialized TCP/IP query	84
Figure 7.10	Sample use of multiple consecutive strings on Matlab	86
Figure 7.11	Automatic sending of commands on Simulink, using the Digital Twin model	86
Figure 7.12	Operation of the Simulink model based on the definition of trigger states	88

Figure 7.13	Example of a MATLAB function block inside Simulink environment	88
Figure 7.14	Code inside a Simulink Matlab function to send a communication string whether a particular condition is met	89
Figure 7.15	Code inside a Simulink Matlab function to communicate with the PLC in order to get information	91
Figure 7.16	Combined use of a persistent variable and strings in Simulink Matlab function block	92
Figure 8.1	Working steps for the application of integrated Digital twin for error states management	94
Figure 8.2	Example of "Solver" Simulink Matlab function block	98
Figure 8.3	Structure of the Digital Twin on Simulink with the integration of the "Solver" function to connect to the MES	99
Figure 8.4	Magazine Front Workstation	100
Figure 8.5	Drilling Workstation	102
Figure 8.6	Robot Assembly Workstation	104
Figure 8.7	Magazine Back Workstation	106
Figure 8.8	Press Workstation	108
Figure 8.9	Disassembly tasks proposal to be performed by a tailored Digital Twin	110
Figure 8.10	Camera Inspection Workstation	111
Figure 8.11	Structure of the Digital Twin on Simulink with the integration of a "Disassembler" function to connect to the MES	112

LIST OF TABLES

Table 1.1	Common points of MES with the other software system categories of Information System Architecture	7
Table 1.2	Communication protocols used in literature to exchange data with MES	10
Table 2.1	Analyzed articles from Scopus inquiries .	17
Table 2.2	Citations by authors of researched papers	18
Table 2.3	Summary of researches performed on Scopus	23
Table 4.1	Practical development steps of the work of the- sis	37
Table 6.1	Function of each workstation in the production line	47
Table 6.2	Possible working programs of the drilling sta- tion.	49
Table 6.3	IP addresses of the <i>I4.0 Lab</i> workstations	55
Table 6.4	Steps to find a variable via OPC UA, on <i>MAT- LAB</i>	58
Table 6.5	Meaning of the colour of a row in the <i>Actual Orders</i> window of MES4	62
Table 6.6	Character and string extensions in MATLAB	69
Table 6.7	String basic syntax rules	72
Table 6.8	Structure of the header and data sections of the string	75
Table 7.1	Values of the IP address of the MES and the relative communication port	78
Table 7.2	Basic steps to send commands to the PLC via TCP/IP communication protocol in Matlab en- vironment	79
Table 7.3	Data fields contained in the TCP/IP on Matlab Workspace	80
Table 7.4	Information extraction from a TCP/IP object previously stored in Matlab Workspace .	82
Table 7.5	Code to express specific extensions in Matlab function "textscan"	84
Table 7.6	Information extraction procedure from TCP/IP object, tailored for Simulink simulation run	90
Table 8.1	Error states of the assembly line	95
Table 8.2	Error states mean downtime without optimiza- tion	97
Table 8.3	Sensors combination for error states at the Mag- azine Front	101

Table 8.4	Sensors combination for error states at the Drilling	103
Table 8.5	Sensors combination for error states at the Robot	105
Table 8.6	Sensors combination for error states at the Magazine Back	107
Table 8.7	Sensors combination for error states at the Press	109
Table 9.1	Mean downtime reduction of error states thanks to the use of integrated Digital Twin . . .	115
Table A.1	MClass = 10 queries	129
Table A.2	MClass = 100 queries	130
Table A.3	MClass = 101 queries	131
Table A.4	MClass = 110 queries	131
Table A.5	MClass = 150 queries	131
Table A.6	MClass = 151 queries	132
Table B.1	DataTypeID for an input/output port in a Level-2 Matlab S-Function	133

LIST OF ACRONYMS

AI Artificial Intelligence. 11, 13, 18, 19, 115
ANN Artificial Neural Networks. 19, 22
API Application Programming Interface. 65

BD Big Data. xii, 11, 13, 17, 18, 31
BDA Big Data Analytics. x, 1, 17, 18
BN Bayesian Networks. 19, 22

CBM Condition Based Maintenance. 20
CPS Cyber Physical Systems. x, xiii, 1, 2, 4, 19, 20, 33

DA Data Analytics. 11, 13, 18
DARPA Defence Advanced Research Projects Agency. 64
DCS Distributed Control System. 3
DM Data Mining. 11, 13, 18, 19

ERP Enterprise Resource Planning. 3, 5, 7, 9, 53

HMI Human-Machine Interface. 3, 20, 46, 96, 97, 100, 102, 104, 106, 108, 116

ICT Information and Communications Technology. x, 1, 20
IP Internet Protocol. 64

KDD Knowledge Discovery in Databases. 18
KPI Key Performance Indicator. 114

LC Linear Classification. 19
LR Linear Regression. 19

M2M Machine-to-Machine. 53
MES Manufacturing Execution System. ix, xi–xiv, xvi, xix–xxi, 3, 5, 6, 24–27, 31–35, 37, 41, 43, 44, 47, 48, 53, 60–62, 64, 66–68, 70–74, 76, 79, 85, 93, 96, 98, 101, 103, 105–107, 109, 111, 113, 114, 118–120
MESA Manufacturing Enterprise Solutions Association. xiii, xxi, 5, 6, 35
ML Machine Learning. 11, 13, 19
MMS Manufacturing Message Specification. 65

OEE Overall Equipment Effectiveness. 60
OPC Open Platform Communication. 53
OPC UA Open Platform Communication Unified Architecture. xiv, xvi, 37, 53–58, 96, 98, 119

P/PE Product and Process Engineering. 7
PCB Printed Circuit Board. xvi, xix, 47, 48, 62, 63, 95, 104, 112, 113
PHM Prognostic and Health Management. 20
PLC Programmable Logic Controller. xvii, 3, 9, 46, 55, 63, 66, 67, 71, 72, 79, 120
PMML Predictive Model Markup Language. 20

RFID Radio-Frequency IDentification. xvi, 47, 60, 68

SCADA Supervisory Control And Data Acquisition. 3, 5, 9
SCM Supply Chain Management. 7
SQL Structured Query Language. 10, 60, 63, 68
SSM Sales & Service Management. 7
SVM Support Vector Machine. 19, 22

TCP Transmission Control Protocol. 64
TCP/IP Transmission Control Protocol/Internet Protocol. xiv, xvii, 64–66, 68, 70–72, 85, 119

UA Unified Architecture. 53

VFD Variable-Frequency Drive. 67

WIP Work In Progress. 9

XML eXtensible Markup Language. 10

ABSTRACT — ENGLISH

Within the constantly growing Industry 4.0 environment, the spread of smart tools to aid manufacturing processes increases everyday. Specifically, the possibility to exploit simulation tools in order to monitor physical resources and extract useful data from them is tangible. One of the main technological enablers of the Industry 4.0 paradigm is, in fact, the so-called Digital Twin. In the Cyber Physical System (CPS) paradigm, the Digital Twin makes it possible to create a virtual copy of a physical asset, enabling real-time monitoring functions for disparate applications within the industrial production field. Given the fact that this tool is at an early development stage worldwide, there is still no specific or cohesive definition of it to be found in literature. However, most of the technological endeavors refer to the Digital Twin as an asset whose mere function is limited to the monitoring of its physical counterpart. As a consequence, almost the totality of the case studies found in literature can be referred to as *Digital Shadows*, since they only change their state when the physical side does it. This work of thesis aims at transforming the *Digital Shadow* into a well-rounded Digital Twin, which is able to communicate in real-time in both directions between the digital and physical side of a resource. This is possible by integrating with the digital tool certain communication protocols, which enable the sending of commands to the Manufacturing Execution System (MES) software related to the specific production facility. The implementation starts with the proposal of optimization frameworks that can be applied to any manufacturing environment. These models involve the interaction of the physical and digital sides of an asset, and work in both directions, enabling real-time optimization measures with the use of a Digital Twin. An application study of the proposed frameworks is given in the context of the Industry 4.0 Laboratory at Politecnico di Milano. Here, the development of a tailored Digital Twin for each workstation belonging to assembly line has been carried out. The Digital Twins have been created in *Simulink* environment; these are modelled to tackle basic shop-floor events, allowing the reduction of downtime, enhancing human-machine interaction and re-scheduling the work orders under determined conditions. Validation is also granted by the calculation of the magnitude of the optimization provided by the use of the integrated Digital Twin in the working processes of the assembly line. Possible future developments might include the integration of maintenance models with the Digital Twin, in order to allow real-time automatic planning and scheduling based on forecasting models.

Nel contesto dell'Industria 4.0, la diffusione dei cosiddetti *smart tools* progettati per supportare i processi produttivi cresce ogni giorno. Nello specifico, la possibilità di utilizzare strumenti di simulazione al fine di monitorare le risorse fisiche per poi ricavarne dati utili è tangibile. Una delle principali tecnologie abilitanti del paradigma Industria 4.0 è il Digital Twin. Nel panorama dei Cyber Physical System (CPS), il Digital Twin rende possibile la creazione di una copia virtuale di una risorsa fisica, abilitando funzioni di monitoraggio in real-time per le svariate applicazioni all'interno del panorama produttivo industriale. Considerando che questo strumento si trova nella sua fase iniziale di sviluppo, non è stata ancora elaborata una sua definizione precisa e totalizzante in letteratura. In ogni caso, buona parte del materiale di ricerca si riferisce al Digital Twin esclusivamente in relazione alle funzionalità di mero monitoraggio della sua controparte fisica. Di conseguenza la quasi totalità dei casi studio trovati in letteratura può essere definita *Digital Shadow*, in quanto il modello digitale cambia stato soltanto quando lo fa anche la parte fisica. L'obiettivo di questo lavoro di tesi è la trasformazione del *Digital Shadow* in un Digital Twin capace di comunicare in real-time in entrambe le direzioni fra il lato fisico e digitale di una risorsa. Ciò è reso possibile grazie all'integrazione nel tool digitale di protocolli di comunicazione che permettono di inviare comandi al Manufacturing Execution System (MES) dello specifico impianto produttivo. Il lavoro è cominciato formalizzando due framework di ottimizzazione, pensati per qualunque ambiente di produzione industriale. È in questi modelli che si concretizza l'interazione *bi-laterale* fra il lato fisico e quello digitale di un asset, che rende possibile l'ottimizzazione in tempo reale tramite l'uso del Digital Twin. Un caso applicativo dei framework proposti è stato implementato nel contesto dell'Industry 4.0 Laboratory, presso il Politecnico di Milano. Qui, è stato sviluppato un set di Digital Twin, modellati su misura di ciascuna stazione di lavoro della linea di assemblaggio presente. I Digital Twin sono stati creati in ambiente *Simulink* al fine di gestire semplici eventi a livello produttivo, permettendo la riduzione dei tempi morti, il miglioramento dell'interazione uomo-macchina e la pianificazione reattiva degli ordini di lavoro in determinate condizioni. La validazione del modello è garantita anche dal calcolo dell'entità dell'ottimizzazione apportata facendo uso del Digital Twin integrato con il MES nell'ambito dei processi di assemblaggio della linea. Possibili sviluppi futuri includono l'integrazione di modelli di manutenzione con il Digital Twin, al fine di abilitare un planning automatico basato su modelli predittivi.

EXECUTIVE SUMMARY

INDUSTRIAL CONTEXT Industry 4.0 is an industrial paradigm that developed considerably during the latest years (cf. Figure 0.1). This trend fosters the so-called *smart factories* that create the conditions for a highly modular and digitalized production environment.

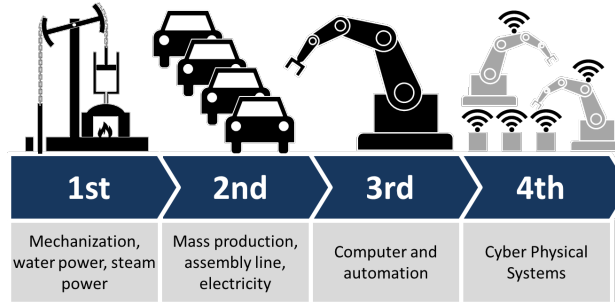


Figure 0.1. Industrial revolutions

Many enabling technologies make it possible to use this kind of paradigm like: Information and Communications Technology (ICT), Cloud Computing, Cyber Physical Systems (CPS), Big Data Analytics (BDA). A structured systematic literature review work has been performed in the proposed work of thesis, to provide a full definition of the main enabling technologies used in the field.

In general the mission of Industry 4.0 is to make use of its enablers to give birth to automated and continuously improved systems.

Specifically, the case of Cyber Physical Systems is going to be analyzed. According to the IEC 62264 normative, this kind of smart tools make it possible to reach all of the five levels of the Automation Pyramid in a transversal way, leaving behind the hierarchy at the base of its classical definition (cf. Figure 0.2).

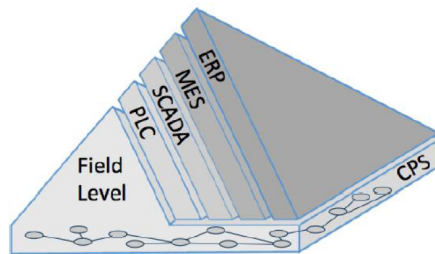


Figure 0.2. CPS-based Automation Pyramid

In manufacturing environment, within the vast CPS scene, the so-called **Digital Twin** can be found. Even if at an early development

stage worldwide, the Digital Twin allows the mirroring of a physical asset, enabling data elaboration for the optimization of a production system. In general, Digital Twins in literature are modelled using simulation techniques regarding both algorithms and real-time data gathering.

Furthermore, since it is fundamental for the thesis purposes, the analysis of the second level of the Automation Pyramid is provided: the **Manufacturing Execution System (MES)**.

This software tool is used in production environment to help and optimize production, enabling a big set of peculiar functionalities. These usually concern operations and scheduling, product tracking along the production path, process management, maintenance, data collection *etc.*

Furthermore, the MES can be reached also by external software tools and communicate with them, usually exploiting communication protocols that, in turn, use specific languages.

DIGITAL TWIN FOR OPTIMIZATION AND CONTROL

Various endeavors have been found in literature that try to give to Digital Twins the ability to control their physical counterpart in an overall optimization perspective.

Starting from the searching of optimization techniques exploiting smart tools and technologies like the Digital Twin, a set of theoretical scenarios can be defined, based on a literature study that proposes a synergy with the Industry 4.0 paradigm.

One of the main endeavors lies in the possibility of *time optimization*, trying to get rid of downtime due to error states, according to a *lean* policy. Furthermore, the possibility to align in a more efficient way the *human factor* with the machinery is proposed. Also, as a reaction to certain predicaments that may occur during production, a smart tool-based re-scheduling is considered in the research field. Besides, to apply a *circular economy-based* paradigm to production, the reactive scheduling of *disassembly* programs is also deliberated, aiming at raw materials recovery.

The crucial point of all these proposed optimization scenarios is the *lack of an event-driven autonomy* perspective, that a digital simulation-based tool might provide.

A further elaboration on this aspect can be given by introducing the concept of the *level of integration* of a Digital Twin within the physical world [1].

The main difference of the application cases found in literature lies between the definition of the two following types of digital models (cf. Figure 0.3) [1]:

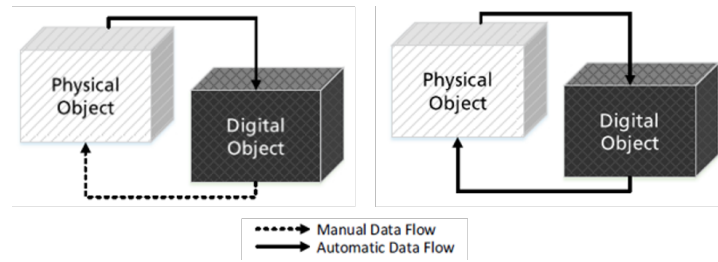


Figure 0.3. Digital Shadow (left) vs. Digital Twin (right)

- *Digital Shadow* (left hand side of Figure 0.3), where the automatic data exchange happens only from the physical to the digital object;
- *Digital Twin* (right hand side of Figure 0.3), where the automatic data exchange happens in both directions between the physical and digital object. This is the highest possible integration level of a Digital Twin in the physical world.

It has been proven that the majority of the case studies in literature fall in the *Digital Shadow* category. This implies a considerable lack of applications concerning the well-rounded Digital Twin featured with high-level integration with its physical side, which is able to communicate automatically in both directions with its physical counterpart.

Given this classification, a set of research efforts about the possibility to give control and autonomy features to a Digital Twin are provided. The cited control models usually take into consideration the product lifecycle in its wholeness; from corrective actions based on optimization strategy to product planning and design, arriving to management of shop-floor data and events. Models can be also predictive or corrective, including the use of Big Data techniques.

Summing up, the integration of this kind of models with a smart tool like the Digital Twin that introduces a *bi-lateral* communication, enables the possibility to have control capability from the digital side to the physical side of an asset.

Other scientific investigations regard the possibility to integrate the MES functionality to a Digital Twin [2]. This would enrich the latter with decision-making ability related to shop-floor level events in the production facility. Besides, this integration work can be also reckoned as an initial step to the integration of predictive and corrective models with the Digital Twin too.

RESEARCH DESIGN Based on the performed research work, a set of gaps in literature has been found.

- **Gap 1:** The majority of the applications can be referred to as *Digital Shadows*, with autonomous communication taking place only from the physical to the digital side, and *not* vice versa [1].
- **Gap 2:** Autonomy lack for CPS-based paradigms, in dealing with shop-floor events in manufacturing environment.
- **Gap 3:** Optimization models for manufacturing environment are proposed in literature, but not applied considering the CPSs field (*i.e.* Digital Twins).
- **Gap 4:** The integration of a MES software with a Digital Twin in order to have a stronger control ability in production is proposed, but not performed in literature.

The definitions of the gaps make room to the proposal of *research objectives* to be fulfilled with the work of thesis (cf. Figure 0.4).

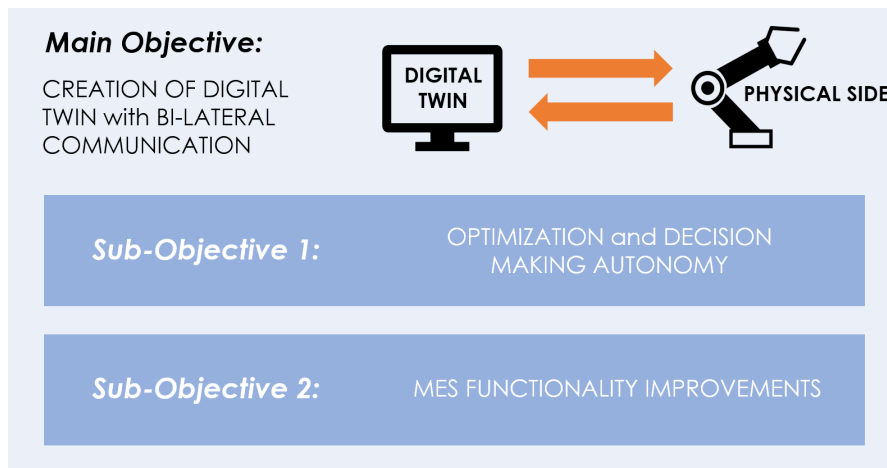


Figure 0.4. Research objectives

- **Main Objective:** Create a full-bodied *Digital Twin*, which allows the communication between the digital and physical side of a resource in *both directions*.
- **Sub-Objective 1:** Optimize and give better performance to an assembly line together with an autonomous decision-making control ability.
Optimization measures include the enhancement of: error-states management; human-machine interaction; re-scheduling; reactive disassembly.
- **Sub-Objective 2:** MES functionality improvement in terms of *Process Management*, based on the MESA International normative.

Based on the research objectives, some practical objectives can be formalized. These include the proposal of general optimization frameworks that have been applied and validated in Industry 4.0 Laboratory environment, at Politecnico di Milano.

It must be noticed that a full functioning *Digital Shadow* has already been implemented and validated in I4.0 Lab environment. Its functions consisted in the mere monitoring of machine state and energy consumption of the assembly line of the lab. This digital tool is essentially the starting point of this work of thesis. Hence, the practical objectives follow.

- Proposal of **optimization frameworks** models, together with Digital Twin tools to enable their application in the field.
- Study of the I4.0 Lab environment, together with the technologies that allow real-time data acquisition on *Matlab & Simulink* (*i.e.* Open Platform Communication Unified Architecture (OPC UA)).
- Study of the MES software of I4.0 Lab, combined with the analysis of communication protocols that enable command sending from external engineering tools (*i.e.* strings via TCP/IP).
- Integration of the MES communication protocol on *Matlab & Simulink*.
- Application of the optimization frameworks in I4.0 Lab by developing tailored Digital Twins for the workstations of the assembly line. The applications propose methodologies to: deal with error states that block the production line; reactively schedule disassembly plans for work orders that do not comply with assembly standards. The models are also validated thanks to the showcasing of numerical optimization results.

PROPOSED MODEL DEVELOPMENT The proposed models consist of two **general optimization frameworks** that put forward the possibility to optimize productive environments in terms of error states, human-machine interaction and re-scheduling.

Both of the models are separated in a *physical* and *digital* level, and show the possible *bi-lateral* interactions between these two worlds. Accordingly, the proposal of a prototypical Digital Twin, which enables this kind of interaction, is provided, together with the basic functions of each component of the digital model.

The first framework deals with **error states management** (cf. Figure 0.5). Only error states that block the production flow are considered. Therefore, they are classified according to the necessity of human intervention to solve them (*type A*: human necessary; *type B*: human

not necessary). Another criterion to categorize the error states can be their impact in terms of mean downtime, when they are solved manually, without the aid of the Digital Twin. Accordingly, when an error state occurs, the simulation tool described by the framework tackles the issue. The Digital Twin either helps the operator solving the error or solves it right away, if the operator is not strictly necessary. Once the error is solved, the flow can resume.

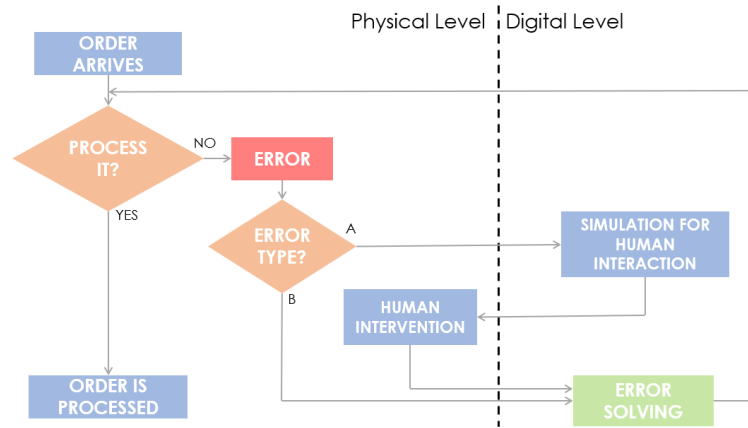


Figure 0.5. Error states management framework

The second framework deals with the **reactive disassembly** of products that do not respect certain standards (cf. Figure 0.6). If this condition is met, the workplan related to the non compliant workpiece is aborted, and disassembly plan is automatically scheduled for the same piece. In order for this framework to be applied, the followings are needed: an assembly tool; a resource able to check pieces (e.g. visually); a disassembly tool.

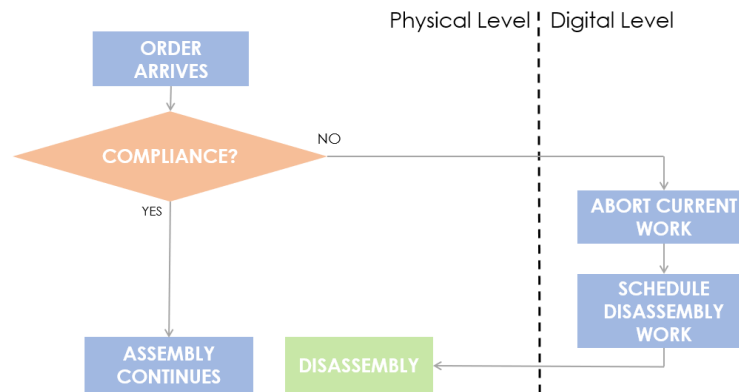


Figure 0.6. Disassembly framework

For both frameworks, basic functions to be encoded in a Digital Twin are proposed. A "Detector" function that reads/overwrites sensors values embedded in the production facility. A function that combines the sensors values, identifying a set of states that automatically

trigger the solving techniques of the Digital Twin; it is called "*Solver*" for the error states management framework and "*Disassembler*" for the disassembly one. For the just described functions, a possible way to solve the errors and schedule disassembly orders might consist in an integration of MES functionalities with the Digital Twin [3]. This is the method adopted in the proposed work of thesis.

INDUSTRY 4.0 LABORATORY The application field to validate the proposed optimization frameworks is the Industry 4.0 Laboratory at Politecnico di Milano.

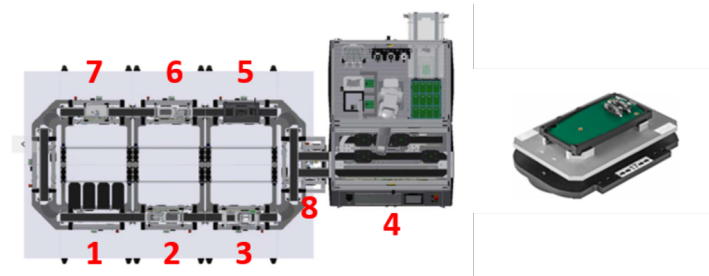


Figure 0.7. The assembly line (left) and the product (right)

The laboratory is endowed with an assembly line provided by *FESTO*, that produces prototypical smartphones (cf. Figure 0.7). The number of workstations is *seven*, and each one of them performs an activity necessary for the assembly of the final product. The nature of the workstations is modular, and they are connected by a conveyor that passes through all of them.

The workstations are: *Manual* where the pallet is put on the conveyor; *Magazine Front* where the front cover of the smartphone is released; *Drilling* where the drilling for screws in the cover is simulated; *Robot Assembly* that assembles PCBs and fuses on the cover; *Camera Inspection* that checks compliance of assembly with respect to the workplan expectations; *Magazine Back* that releases the back cover on the piece; *Press* that presses the back and front cover together.

The line is embedded with sensors (*e.g.* Radio-Frequency Identification (RFID)) that detect all of the operations necessary for the successful assembly of the product. The communication technology used to access and modify the sensors values is Open Platform Communication Unified Architecture (OPC UA). On *Matlab & Simulink*, the access procedure via OPC UA translates to the use of *Level-2 Matlab S-Functions*.

The assembly line in the lab is equipped with a MES software, also provided by *FESTO*: the *MES4*. According to its nature, this MES makes it possible to track the production orders, schedule new orders, abort them and so on. For this specific MES software, the production

house gives the possibility to send commands to it using a *string-based* communication protocol that exploits TCP/IP technology.

MES INTEGRATION This activity consists in the MES functionality integration inside engineering tools like *Matlab & Simulink*. This integration allows to send commands to the MES of the assembly line when certain conditions are met, both on *Matlab* and *Simulink*. The integration on *Matlab* is meant as a preparatory step for the MES integration on *Simulink*, where all the Digital Twins of the proposed work of thesis are implemented.

The development started with the study of the functions of the command strings that can be sent to the MES.

The strings have been categorized into two types: the *"send"* type; the *"get"* type. While the *send* strings are able to perform commands relative to the production line, the *get* strings are also able to store back information related to certain assets, which can be later used for other purposes.

Besides, for the *send* strings, a TCP/IP object is created and connected to the MES server. Then the string is sent, and the task is performed by the PLC. On the other hand, if the string belongs to the *"get"* type, after these tasks, the PLC stores back information in string form inside a field of a Matlab structure. A tailored parsing function has been implemented to decode the string and use the requested information.

Based on this distinction, the integration on *Matlab* took place. Here, the running is simple, since it consists in monitoring the assembly line and, when the situation demands it, sending manually the commands to the MES in order to perform the desired task (cf. Figure 0.8).



Figure 0.8. Integration of MES function on Matlab

The implementation on *Simulink* needs some additional measures. Basically, in this phase of the work, the application of the Digital Twin proposed in the optimization framework is taking place. As a matter of fact, the identification of peculiar *trigger states* has been performed, based on sensors combinations. Accordingly, when certain states are met, a Simulink Matlab function block is tailored to perform the desired activities, by communicating with the MES.

Therefore, it is enough to let the simulation tool run; when the *trigger states* occur, the Digital Twin autonomously sends commands to

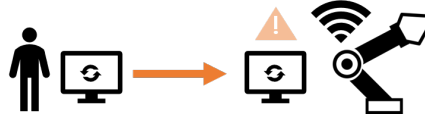


Figure 0.9. Integration of MES function on Simulink

the MES, according to how the model has been developed (cf. Figure 0.9).

APPLICATIONS AND RESULTS Once a method to integrate the MES functionalities with a Digital Twin has been developed, an application case was performed in Industry 4.0 Lab. The applications aim at validating the optimization frameworks previously mentioned. Accordingly, a set of applications was carried out for both the *error states management* and *disassembly* frameworks.

Starting with the application of the **error state management** framework, the work was structured as follows. The first activity to be performed was the identification and timing of the possible error states that involve the workstations of the assembly line. The final number of error states identified is **11**, and they have been categorized, as in the framework, in A and B type. The type A errors of the line, for example concern the situations in which magazines are empty and need to be filled up to resume the assembly production. On the other hand, the type B errors, for instance consist in the missing of an assembly piece that should be on the pallet *e.g.* the front cover of the smartphone.

Once these have been identified, an activity of timing of the mean downtime caused by their occurrence has been performed. This enabled the ranking of error states based on their impact on downtime. The impact of the error states can be either high, medium or low.

At this point, for each workstation where an error state might occur, a tailored Digital Twin has been modelled, following the typology described in the framework (cf. Figure 0.10).

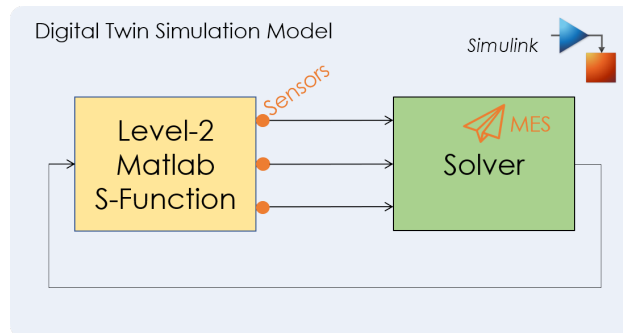


Figure 0.10. Digital Twin for error states management in Simulink

The Digital Twin has a *Level-2 Matlab S-Function* that reads the sensors values of the specific workstation. Then, the "*Solver*" Simulink Matlab function, thanks to the combination of the read sensors, identifies the error states relative to the specific workstation (cf. Table 0.1 for the Robot station example).

Error ID	xG1_BG50	xG1_BG51	xDisableMenu
5	0	0	1
6	1	1	1
7	0	1	1

Table 0.1. Sensors combination for error states at the Robot

When an error state is met during the working of the assembly line, the Digital Twin automatically transmits the associated command to the MES, in order to solve it promptly, whether it needs human intervention or not. The Solver block loops back to the *Level-2 Matlab S-Function* to enable the sensors overwriting to release the work orders from the workstations when the error has been solved.

At this point the application of the **reactive disassembly** framework is described. The requirements for the application of the framework are met, since all the necessary equipment are in the assembly line of I4.0 Lab.

Specifically, since the proposed framework aims at scheduling a disassembly process for a workpiece that is not compliant with certain standards, the used assets are: the *Robot Assembly* workstation that assembles the PCB on the smartphone; the *Camera Inspection* workstation that visually checks the compliance of assembly with respect to the workplan; the *Robot Assembly* workstation that can be reconfigured to perform disassembly operations as well.

According to the framework definition, a Digital Twin has been developed for the Camera Inspection workstation (cf. Figure 0.11).

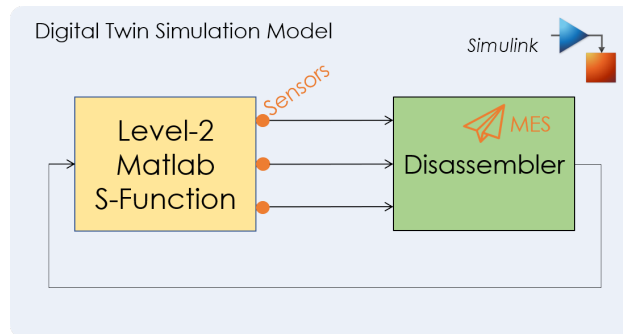


Figure 0.11. Digital Twin for disassembly framework application

Even in this case, a *Level-2 Matlab S-Function* reads the sensors values of the workstation. These are combined by a Simulink Matlab function to identify the condition in which the conformity of assem-

bly with respect to the workplan is not verified. When this occurs, thanks to the MES integration functionality, commands are sent to the MES in order to abort the current order, and schedule a disassembly order to be performed for that workpiece. Consequently, the function is called "*Disassembler*".

The **results** for the application of the **error states management framework** have been measured using the *percentage time reduction* KPI, calculated on the downtime relative to the error states before and after the application of the Digital Twin. The reduction of the mean downtime for the type A error states (with human intervention) range from 25% to almost 60%; on the other hand the reduction for type B error states (without human intervention) is around 85-90%.

It is to be noticed that the reduction of downtime of errors of type A range a lot, since the operator intervention is still a source of variability that cannot be controlled. On the other hand the error states of type B, when optimized, have an almost deterministic downtime duration. This is due to the fact that the computational time to solve the error state is almost constant.

Therefore, optimization is met and downtime related to the error states occurrence is reduced by the intervention of a Digital Twin. Also, human-machine interaction is improved.

The **results** of the application of the **disassembly framework** lie in the ability to schedule a disassembly plan when assembly standards are not met. Also, scraps are reduced thanks to the use of the Digital Twin.

CONCLUSIONS Based on the performed activities, an overview of the way **practical objectives** have been achieved is going to be given.

Starting by the formalization of two *optimization frameworks*, respectively concerning the **error states management** and the **reactive disassembly**, a methodology to use both physical and digital sides of a certain asset is given; the proposed frameworks find a way to create a *bi-lateral* communication between those two sides. Accordingly, the frameworks proposed to use a Digital Twin that might optimize the production by exploiting this *bi-lateral* interface.

To validate this model, the application of the frameworks has been performed in the Industry 4.0 Lab at Politecnico di Milano. In the laboratory, the integration of the MES to an already implemented "*Digital Shadow*" has been carried out with the aid of a communication protocol. Consequently the Digital Shadow, which had only monitoring functionalities, became a well-rounded *Digital Twin*.

At this point, having conceived a way to integrate the MES to the Digital Twin, the two frameworks have been applied in the field.

The first model dealt with the optimization of a set of error states found for the assembly line of the lab. These errors usually cause a certain downtime, and block the production flow. Thanks to the use of the integrated Digital Twin, the mean downtime has been critically reduced.

Secondly, an application related to the reactive scheduling of an disassembly order for a workpiece that is not compliant with the assembly standards, has been performed by means of the Digital Twin. This is the second proposed framework, whose benefit is the possibility to recover material and avoid scraps.

At this point, the fulfillment of the **research objectives** thanks to the implementation of the practical ones is showcased.

The **Main Objective** proposed to create a Digital Twin able to communicate in both directions between the physical and digital part. In this work, the development of a Digital Twin that sends automatically commands to the MES when certain conditions are met is performed. So the digital side change its state automatically when the physical side does it *and* vice versa. This validates the transformation of the Digital Shadow into a Digital Twin, and the fulfillment of the objective.

The **Sub-Objective 1** puts forward the application of optimization measures using a Digital Twin. This objective has been fulfilled thanks to the proposal and application, with subsequent validation, of the optimization frameworks. These models, concerning optimization in terms of error states management, human-machine interface, re-scheduling and disassembly, prove that the possibility to apply Digital Twins to perform this kind of activities in manufacturing environment is possible.

The **Sub-Objective 2** advances the possibility to improve the Process Management MES capability, with respect to the MESA standards. With the use of a Digital Twin with MES integration functionality, the MES itself is subject to an enhancing of its functionalities. As a matter of fact, the MES, being controlled by a real-time simulation tool, is endowed with a higher sensibility and autonomy of control of the flow of work, which is based on the actual production activities concerning the facility. This determines the fulfillment of the objective.

Possible **future developments** of the work of thesis concern the possibility to integrate with the proposed Digital Twin *maintenance policies* as well. The use of predictive and condition based maintenance policies allows, in fact, to reactively schedule orders according to forecasting models in real-time.

Furthermore, the possibility to use the integrated Digital Twin in order to monitor energy consumption and possibly re-schedule and abort orders when anomalies are detected could be interesting.

INTRODUCTION

1.1 INDUSTRY 4.0

The Industry 4.0 paradigm has been conceived and firstly promoted during the Hannover Fair, in Germany, in 2011. This term, according to common knowledge, is referred to the current trend of automation and data exchange in manufacturing technologies.

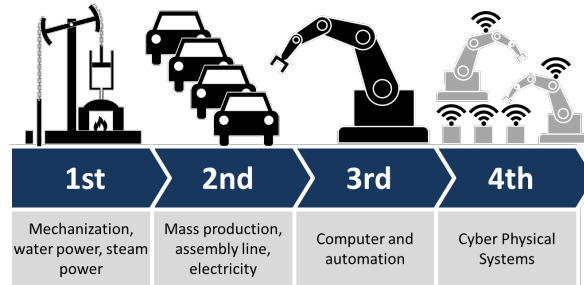


Figure 1.1. Industrial Revolutions throughout the last centuries

Like in Figure 1.1, manufacturing environment went through several phases and revolutions; the current one tackles the constantly growing need of automation in production with the use of **cyber** and **interconnected** systems.

Industry 4.0 fosters the so-called "*smart factory*", a highly modular and digitalized production environment. The cornerstones of this new industrial paradigm are: *visibility; connectivity; autonomy*.

Based on these principles, a growing endeavor is given to the research and development of the key enablers of Industry 4.0. In general, many technologies are used to enable this paradigm, but the most relevant are hereby briefly listed [4]:

- Information and Communications Technology (ICT)
- Cloud Computing
- Cyber Physical Systems (CPS)
- Big Data Analytics (BDA)

A deeper analysis of this environment as a whole, with particular attention to the definition of all its key enablers, has been performed with the literature review work in Chapter 2.

The field of application of Industry 4.0 paradigm regards many aspects of the manufacturing topics, such as *design*, *monitoring*, *control* and *scheduling*. Hence, innovation can be applied vastly in the industrial environment.

The use of smart technologies gives birth to a set of challenges for the future of industry. For several decades many time and cost-based inefficiency factors afflicted the industrial environment in fields like operations, paperwork, shop-floor executions, human interaction with machines. The mission of Industry 4.0 paradigm is to get rid of these inefficiencies, giving birth to automated and continuously improved smart systems [4].

1.2 CPS-BASED AUTOMATION PYRAMID

In this paragraph, an overview about the concepts at the base of Cyber Physical Systems is given. The history and development of CPSs is going to be explained, in order to provide the reader with the necessary premises for the description of a peculiar digital tool that is shaped in this environment: the *Digital Twin*. Although, the latter is going to be fully explained in the next paragraphs.

Automation in manufacturing is not a recent development. Within the industrial field, one of the first regulations that introduced a standardized way to define industrial automation is the ANSI/ISA-95¹. Within this standard, several reference models are included, that refer to the information exchange necessary to control activities connected to industrial production plants.

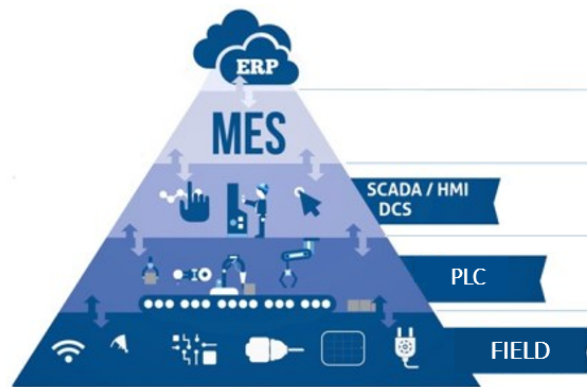


Figure 1.2. Automation Pyramid representation based on IEC 62264 normative

One of the numerous reference systems is the IEC 62264², that introduces the so-called **automation pyramid**. This system, pictured in

¹ <https://en.wikipedia.org/wiki/ANSI/ISA-95>

² https://en.wikipedia.org/wiki/IEC_62264

Figure 1.2, sums up the interactions that take place within the automatic control of a production plant.

The structure is made of 5 different levels:

- *Level 0: FIELD.* This is the shop-floor level of the production plant. All the physical assets, embedded with sensors, that can measure the data of the system and the actuators that control the process fall in this level. Communication protocols that enable interaction with the control system of sensors and actuators can be found in this level as well.
- *Level 1: CONTROL.* Here, Programmable Logic Controller (PLC) and Distributed Control System (DCS) can be found. These are the enablers of direct control of the variables of the field level.
- *Level 2: SUPERVISORY.* This level concerns the Supervisory Control And Data Acquisition (SCADA), a server that systematically and periodically takes data and processes them in order to memorize useful information. Workstations and Human-Machine Interface (HMI) fall in this category too.
- *Level 3: MANAGEMENT.* In this level, all the data acquired from the lower layers are analyzed and translated into useful information. Here, the work is done mainly by the Manufacturing Execution System (MES), a management tool that tracks and documents and enables the transformations happening in the production plant.
- *Level 4: ENTERPRISE.* The highest decision-making level of the pyramid is reached here. The Enterprise Resource Planning (ERP) implementation is included in this layer, making possible to track the highest level business resources, commitments and requirements.

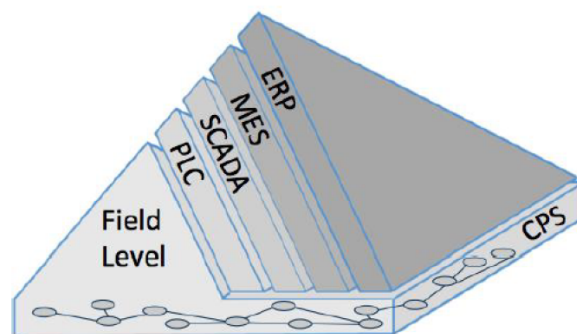


Figure 1.3. CPS-based automation pyramid

The connection between these levels, as the pyramid shape suggests, is hierarchical. This means that a certain control level can communicate only with the levels immediately next to it.

With the introduction of smart manufacturing, a new concept of the classical automation pyramid has been created, where the main character is a technological enabler of this new industrial paradigm: the *Cyber Physical Systems (CPS)*.

In fact, with the use of CPSs, the five automation levels of communication of the pyramid are no more hierarchical, but can be reached crosswise, like shown in Figure 1.3.

1.3 DIGITAL TWIN

According to what just said in the previous paragraph, a general idea of Digital Twin can be provided. In fact, the Digital Twin places itself in the broad CPS world. This digital tool is endowed with some peculiar characteristics according to its use and utility in manufacturing environment.

Generally speaking, an overall and comprehensive definition of *Digital Twin* does not exist [1]. In fact, no common understanding concerning this term can be found, since it is used slightly differently over disparate disciplines.

Whilst physical twins have been around for some time, the first definition of Digital Twin has been given by Michael Grieves in 2002, in the context of an industry presentation concerning product life cycle management. At that time, the Digital Twin was considered as a mere digital informational construct about a physical system, and linked to the physical asset in question [5].

At its origins, the Digital Twin describes a mirroring product, but in the years the concept evolved. In fact a central aspect of the Digital Twin became the ability to provide different information in a consistent format. Digital Twins have become more than just pure data; they include algorithms that describe their real counterpart and decide about actions in the production system based on this processed data [1].

Within the Industry 4.0 paradigm, in particular in the manufacturing field, the Digital Twin is reckoned to have a certain role. As a matter of fact it "*exploits sensed data, mathematical models and real time data elaboration in order to forecast and optimize the behaviour of the production system at each life cycle phase in real time*" [1] [6].

1.3.1 Digital Twin in Simulation

According to what just said, one of the main techniques that generate a Digital Twin tool is *simulation*. Although, simulation is not performed always in the same way. In fact, according to the kind of simulation used, a different nature and applicability is going to be given to the peculiar tool. In particular:

- *Simulation via algorithms.* In this case the simulation technique is a mere support to decision making within the industrial environment. In fact, based on a digital simulation model of a certain physical asset, by applying known and validated algorithms, the optimization cut can be found by performing a big number of simulation runs. The average behaviour of the object in a set of scenarios is studied, and the optimal solution can be found by applying known optimization algorithms.
- *Simulation via real-time data gathering.* This kind of simulation exploits sensors embedded in plants *e.g.* production lines. The objective is to use real data coming from the shop-floor level of industrial systems that are not coming from historical data sets or maintenance activities. On the contrary, data are gathered real time, so a direct approach in monitoring and optimizing this kind of data can be given by the simulation tool used. This is the basic nature of a *Digital Twin* as it is defined in literature.

Hence, in this work of thesis, the use of a Digital Twin is modelled and exploited in order to take advantage of data coming from the shop-floor level of automation. These data are synchronized to a specific modelling of machine error and malfunction states, in order to take optimization actions autonomously.

Further considerations on the role of Digital Twin are provided in Chapter 3, where attention will be placed also on whether there is room for its improvement in the research environment. Specifically the perspectives on a possible automated control functionality over a production plant that could be given to a Digital Twin are analyzed.

1.4 MANUFACTURING EXECUTION SYSTEM

Based on the normative IEC 62264 upon which the *Automation Pyramid* depicted in Figure 1.2 is conceived, the attention of this paragraph is dedicated to the explanation of the second level of the pyramid itself: the so-called **Manufacturing Execution System (MES)**.

As a general definition, the MES is nothing but a computerized system that is often used in manufacturing environment. The idea of MES might be seen as an intermediate integrative step between an Enterprise Resource Planning (ERP) system and, on the other hand, a Supervisory Control And Data Acquisition (SCADA) or process control system. Still, historically, the exact boundaries have fluctuated.

The conceptualization of a MES system finds its roots in the 1990s, when industry groups such as Manufacturing Enterprise Solutions Association (MESA) International have been created in order to address the complexity, and advise on execution of MES Systems.

The MESA International provides its own full and comprehensive definition of MES, that is:

Manufacturing Execution System (MES) delivers information that enables the optimization of production activities from order launch to finished goods. Using current and accurate data, MES guides, initiates, responds to, and reports on plant activities as they occur. The resulting rapid response to changing conditions, coupled with a focus on reducing non value-added activities, drives effective plant operations and processes. MES improves the return on operational assets as well as on-time delivery, inventory turns, gross margin, and cash flow performance. MES provides mission-critical information about production activities across the enterprise and supply chain via bi-directional communications. [3]

As it can be deduced by this definition, MES software tools, which are widely field-proven, can create benefit and added-value on several layers of manufacturing environment.

Summing up, the immediate advantages that MES provides to a company are:

- production data, usable by supervisors, operators, management teams;
- data from the facilities to feed the financial system;
- guidance for activities in production lines in order for them to meet their global goals;
- key piece information for quoting on real capabilities in supply chain management.

1.4.1 MES in Information System Architecture

Being the MES one of the major information types aimed at supporting manufacturing companies, it can interact with various manufacturing software system categories.

These software categories, which are all part of the Information System Architecture and can interact with MES like in Figure 1.4. Specifications of the common points with MES are provided in Table 1.1. The degrees of overlap vary by specific industry and implementation.

This whole dissertation proves the importance of MES as a central element of a manufacturing environment. In fact it can leverage the investments in all of the other manufacturing information systems, as well as enabling improvements in plant operations.

Software Category	Common points with MES
Sales & Service Management (SSM)	Labor Management
Supply Chain Management (SCM)	Scheduling activities
Enterprise Resource Planning (ERP)	HR function
Product and Process Engineering (P/PE)	Document control
Controls	Process management

Table 1.1. Common points of MES with the other software system categories of Information System Architecture

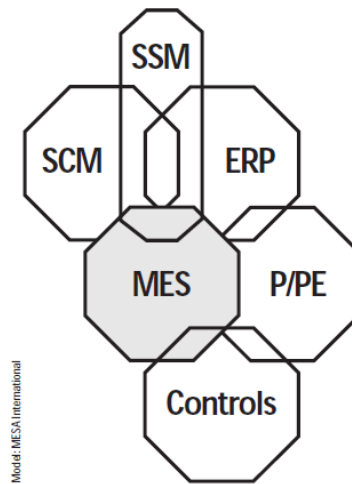


Figure 1.4. MES Context Model

1.4.2 MES Functionalities

Much like the other categories of Information System Architecture, the MES has not a single function. As a matter of fact, based on all the various activities that can be performed in a plant, MESA International has identified **eleven** principal functions for a MES that describe its capacities to support, guide and track each of the primary production activities [3].

1. **Operations/Detail Scheduling** — sequencing and timing activities for optimized plant performance based on finite capacities of resources.
2. **Resource Allocation and Status** — guiding what people, machines, tools and materials should do, and tracking that they are currently doing or what have just done.
3. **Dispatching Production Units** — giving the command to send materials or orders to certain parts of the plant to begin a process or step.

4. **Document Control** — managing and distributing information on products, processes, designs or orders, as well as gathering certification statements or work and conditions.
5. **Product Tracking and Genealogy** — monitoring the progress of units, batches, or lots output to create a full history of the product.
6. **Performance Analysis** — comparing measured results in the plant to goals and metrics set by the corporation, costumers or regulatory bodies.
7. **Labor Management** — tracking and directing the use of operations personnel during a shift based on qualifications, work patterns and business needs.
8. **Maintenance Management** — planning and executing appropriate activities to keep equipment and other capital assets in the plant performing to goal.
9. **Process Management** — directing the flow of work in the plant based on planned and actual production activities.
10. **Quality Management** — recording, tracking and analyzing product and process characteristics against engineering ideals.
11. **Data Collection/Acquisition** — monitoring, gathering and organizing data about the processes, materials and operations from people, machines or Controls.

This kind of functions provide the core information base to run almost every kind of plant. Also, in order to meet the same objectives, a mix of the aforementioned functionalities might be used.

The most powerful aspect of MES is that it combines both global business rules and local operations best-practices into plant-wide view of not only what is happening, but what should be happening to meet the objectives. Users and engineers can configure this set of functionalities to meet the corporate and plant objectives [3].

1.4.3 *MES Benefits*

In history, MES is the manufacturing software that created the highest benefit yield among all the other ones [7].

Based on the statistics gathered by MESA, the main benefits of MES are the *reduction* of [7]:

- manufacturing cycle time by an average of **45%**;
- data entry time of at least **75%**;

- Work In Progress (WIP) by an average of **24%**;
- paperwork between shifts of an average of **61%**;
- lead time by an average of **27%**;
- paperwork and blueprint losses an average of **56%**;
- product defects by an average of **18%**.

MES provides most industries a wide range of operational benefits, even for companies that have other systems in place. Several MES solutions or functions may contribute to the strength of a given benefit, and benefits may also be increased by appropriate process improvements. Benefits accrue in different sequences, based on the functionality chosen, project focus, integration to other systems, and the plant's driving needs [3].

1.4.4 MES Communication

Being one of the levels of the aforementioned *Automation Pyramid* (Figure 1.2), the MES must be endowed with communication means to exchange data the other levels of the pyramid itself, like ERP, PLCs or SCADA.

Furthermore, the various communication protocols that are up to be showcased, enable also a communication means with external software tools or engineering tools that might control the MES remotely as well.

Like in Figure 1.5, to communicate with an external tool, MES should adopt a certain **communication protocol** that, in turn, uses a defined **language**.



Figure 1.5. Communication paradigm between MES and external tools

Among an industrial point of view, the main communication protocol that seems to be mostly used is the **Industrial Ethernet** [8] [9]. This tool, based on standard IEEE 802.3, defines the physical layer and the data link layer model. As a consequence, the standard IEEE 802.3 specifies characteristics of communication interface and a method of managing the access to transmission medium; Industrial Ethernet falls exactly in this category [8].

Besides, many communication Ethernet protocols are recognized inside industrial practices, and were created and used in many solutions. They are gathered in Table 1.2 [8]. Some of the protocols are going to be later explained, since they are the ones used in the work of thesis. For specification about the terminology used in Table 1.2, refer to Chapter 6 where, in the section about technologies, the use of such protocols is explained.

Protocol Name	<i>Description</i>
Profinet IO	cyclic communication through MAC [8]
ISO	acyclic communication through MAC [8]
ISO-on-TCP	acyclic communication through IP [8]
TCP/IP	reliable acyclic communication through IP [8]
UDP/IP	acyclic datagram through IP [8]
M2CML	Manufacturing to Control Markup Language (acyclic) [10]

Table 1.2. Communication protocols used in literature to exchange data with MES

Aside from communication protocols, to reach a MES software from other tools, standards about the used languages exist worldwide.

This is because the way messages and data streams are encoded can either be *standardized* according to specific normatives or *proprietary*, according to the production house that designs and models the MES.

Some of the most used languages are:

- **eXtensible Markup Language (XML)** [10];
- **Structured Query Language (SQL).**

Based on these rules, any external tool can provide messages to the MES, in order to reach SCADA, PLC and, in the end, the Field level of the Automation Pyramid. This grants decision-making ability to whichever tool the user decides to adopt.

This element is crucial for the proposed work of thesis and will be later discussed. In fact, the thesis implementation is established on communication between manufacturing software tools like MES and external software and engineering tools *e.g. Matlab & Simulink*.

LITERATURE REVIEW

The aim of this chapter is to define and describe the motivation and procedure of literature review adopted for the work of thesis.

The review process has been performed to give a thorough and exhausting explanation about the techniques and technologies used in the Industry 4.0 environment.

Subsequently, the results of the performed inquiries are going to be given.

2.1 TOPICS, MOTIVATIONS AND AIMS

The reason why the literature review has been performed is to contextualize the work of thesis inside the research environment.

Based on the introduction on the Digital Twin concept done in Chapter 1, there is a particular flow of activities regarding data that are usually carried out with the use of this kind of tools. This is what is shown in Figure 2.1.



Figure 2.1. Sample data flow

Hence, the urge to find some reference about this procedure in literature grew. So, overall, the work of literature review started with one main goal: find specific **definitions, synonyms, relationships** of the following terminology, later defined as "*key words*":

- **Big Data (BD);**
- **Data Analytics (DA);**
- **Data Mining (DM);**
- **Artificial Intelligence (AI);**
- **Machine Learning (ML).**

In general, a spread use of these terms has been done through the years, among several fields of science and technology. The main goal

is to make order and clearness in the use of these terms and in the specific relationships that may or may not exist among them.

It is to be noticed that the study of the use of these terms has not been performed by analyzing literature coming from all the fields of knowledge. On the contrary, the inquiry was limited to these so called "*fields*":

- Industry;
- Production;
- Manufacturing;
- Factory.

The scope of the limitation is to narrow down the countless amount of existing knowledge in the main databases of the web, focusing the analysis to the relevant fields for the research plan.

Besides, particular attention will be given to the definition of the enabling technologies and applications starting from the searched *key words*. This will provide a strong context for the use of Digital Twins, and the data-based techniques used by these systems within the industrial field.

2.2 LITERATURE REVIEW METHODOLOGY

The work has been done following a *systematic literature review* methodology. As a matter of fact the research hasn't been performed by randomly reading papers. On the contrary, a systematic research has been performed on the main abstract and citation databases of the web. This process consists in narrowing the research results by filling up the research prompts of the websites one at a time. The result is going to be a "*funnel-like*" research process. The purpose of this narrowing process is to find a limited set of papers or citations that might be useful under the review process perspective.

The adopted methodology can be divided into two distinct phases:

1. the *research* phase;
2. the *analysis* phase.

The *research* phase started with a set of researches made on *Elsevier's* abstract and citation database: *Scopus*¹.

The number of inquiries performed is five. As it is shown in Table 2.3 (placed at the end of the current Chapter), the process consisted in filling up all the prompts of the research toolbar one by one.

The table has to be read like this:

¹ <https://www.scopus.com/>

- each **column** represents a single inquiry. Every column is called *SCOPUS- x* , where " x " stands for the identity number of the specific research, *e.g.* *SCOPUS-3* is the third research. A further partition has been carried out with two *sub-columns* that represent the *field* searched, and the subsequent *number of articles* that came out of the browsing;
- each **row** features a single prompt filling. For example, for *SCOPUS-1*, at first the words "*Data Analytics (DA)*" were typed in the research bar, and this gave **26 472** resulting articles. Then a restriction on time was imposed (*year* prompt), obtaining **26 386** articles. Next, the limitation about the *subject area* needed to be performed. Here, by selecting the *engineering* field, **6 601** articles came out. Then the selection of the *key words* of the research plan was executed. It is to be noticed that *SCOPUS* naturally selects a batch of key words, based on the research that is being performed at that time. Hence, all of the needed key words naturally appeared in this set. So, by selecting *BD*, *DA*, *DM*, *AI*, *ML* (dedicated section of the thesis for reference of the acronyms), a total of **3 220** citations resulted. Then the final restrictions were imposed, by filling out the blank in the "*Search within Results*" bar. Here, one by one, all of the *fields* of the research were typed in. So, by putting *Industry*, **835** articles resulted; the imposition of the *Production* field led to **283** papers; the filling with *Manufacturing* gave **191** articles; in the end a final batch of **51** publications emerged by entering the *Factory* field. A maximum number of 60 publications were considered as the upper bound for a single inquiry on *Scopus*. Hence, this set of papers could be considered as a reasonable amount to be studied in the next analysis phase. All of the other researches were carried out similarly to the one just described. For reference, check Table 2.3.

It has to be noticed that a series of acronyms were used in Table 2.3. These are explained in the relative section.

After the fifth research, it was obvious that the platform was providing always the same articles, so the research phase was reckoned to be completed. The total number of articles and citations retrieved from all the inquiries was **154**.

The *analysis* phase started with the study of the **titles** and **abstracts** of the articles, in order to evaluate whether they were useful or not to the research aims.

This lead to a further skimming of the articles. Subsequently here is a list of some of the reasons why the discarded articles were not feasible for the research.

- The main reason was the *inconsistency* in terms of *field of application* with respect to the manufacturing and industrial environment. For example, if an article was relevant in terms of *key words*, but the field of application was different from the *fields* point of view, it was discarded. Here some examples of fields that have not been considered relevant: social networking; social structures; customer satisfaction; costs; demand prediction; etc.
- Some articles were too specific in the description of the implemented algorithms to perform *data mining* et similia. This means that the article explained the *computer science* point of view of the issue. For example they were written by researchers of Computer Science departments etc. So they were not considered pertinent.
- Sometimes the focus of the articles was too close to the economic and corporate point of view rather than the operative industrial environment.
- Some articles couldn't be downloaded because of permission issues.

This led to the discarding of 110 papers. Accordingly, from 154, the number of **44** documents has been reached.

The analysis phase proceeded with a **full-length read** of the remaining papers, looking for interesting knowledge that could be used for the objectives. This phase also led to another cut of papers, again, mainly because of lack of consistency in terms of field of application. This happened because, even though the article could have been considered interesting by solely reading title and abstract, when proceeding with a deeper analysis, the *key words* and the research *fields* were analyzed in a superficial or not relevant way. So, this kind of papers was discarded too. Thus, 16 papers have been scrapped, leaving a final batch of **28** papers coming out of the research inquiries on *Scopus*.

A final analysis phase, always linked to the full-text read of the body of the articles, consisted in the research of peculiar *citations done by the authors of the articles* themselves. The work basically proceeded by finding the passages in which particular information about the used terms were given, and looking for the article that the author quoted in that specific line. With this activity, a richer amount of articles could be used as a reference, to contextualize the work into the research environment. This procedure made possible to add further **10** articles to the already existent batch.

So, in the end a final number of **38** articles (**28** coming out of the research inquiries and **10** quoted by the authors) has been selected. These have been subsequently studied more in detail with full length read and focused analysis.

2.3 LITERATURE REVIEW RESULTS

In this section an overview of the selected papers is going to be given. The structure of the results is going to be presented following the subsequent outline.

1. The starting point is the **analysis of the papers** based on the aims of the research objectives explained in the first section of the chapter. For a better explanation, two tables are going to be used.
2. Secondly, the **definitions, synonyms, relationships** of the *key words* in the application *fields*, based on the information detected in the analyzed articles, is going to be provided.
3. Eventually, a compendium of the **enabling technologies** and prototypical **applications** in the manufacturing environment, as in literature, are going to be showcased.

2.3.1 Analysis of papers

Two tables are presented:

1. The first table (Table 2.1) with only the papers coming out as results of the *Scopus* inquiries;
2. the second one (Table 2.2) with articles that have been quoted as reference by the authors of the first batch of papers.

Table 2.1 needs to be read like this:

- each row stands for a single **article** or **publication**.
- there are two columns that provide information about the **author** and the **year** of publishing of the article.
- there are five columns with several **acronyms**, one for each *key word*. Refer to the dedicated section of the thesis for the meaning. If in there's an "x" on a row corresponding to a key word, it means that that specific key word is mentioned and analyzed in the article.
- the **SOA** (*State Of the Art*) column gives information about the presence of a description of the overall existing background in terms of application of the *key words* in the manufacturing field. In other words, a list of existing technologies, terms, surveys, knowledge *et similia* present in literature and showcased in the article, is searched in this point. The articles of this type are those who helped the most in finding unambiguous definitions of the *key words*. If there's an "x", it means that such a study has been conducted in the article.

- the last column, called **App.** (*Applications*) gives information about enabling technologies, case studies, prototypes or experimental settings that have been implemented by the authors of the article. This is different from the *State Of the Art* section because, while the latter is based on the knowledge already existing in the scientific field, the *Applications* section studies the original contribution to research of the analyzed article. With an "x" on the row, the article has this kind of study.

Table 2.2 is very similar to the first one, and needs to be read like this:

- each **row** represents a single article;
- the **source** column gives information about the article that cites this specific reference, then the latter is described with **author** and **year** of publishing, in the respective columns;
- all the other columns have the exact same meaning as in Table 2.1.

2.3.2 Definitions, Synonyms, Relationships

Based on the articles reported in Table 2.1 and Table 2.2, it can be noticed that all of the *key words* are deeply connected to the environment regarding the so-called **smart systems**. This kind of assets can be considered as one of the main scientific concepts that affected the world during the last years, in many application fields. Regarding this, it is to be noticed that during the research process in *Scopus*, many articles popped out, about lot of fields, not strictly related to the manufacturing environment. This is being told to report the magnitude of the "smart" phenomenon worldwide.

However, smart systems are a rampant trend; these are nothing but one of the main expressions of the overall **Industry 4.0** paradigm. This policy, that has become a global trend during the last decades, started in Germany as a high-technology manufacturing concept. The rapid growth of this kind of technologies that are continuously interconnected with electronics, enables the manufacturing growth. This fast advancement in these fields let to the definition of this new paradigm that has been acknowledged worldwide as the *fourth industrial revolution* [11].

Briefly, the main scope of these industrial and manufacturing philosophies is the overall *automation* and *interconnection* of all of our resources, often in *real time* when possible, trying to include all of the "actors" that are related to the production process. All of these connections and integration processes aim to realize flexibility, adaptability, and efficiency and increase effective communication between producers and consumers [12].

Author	Year	BD	DA	DM	AI	ML	SOA	App.
Peres [13]	2018		x			x	x	x
Wang S. [14]	2018			x			x	x
Giusti [15]	2018		x					x
Lenz [16]	2018		x				x	x
Zheng [4]	2018		x		x		x	x
Gunasekaran [17]	2017	x	x				x	x
Suryajaya [18]	2017	x						x
Cattaneo [19]	2018	x	x	x	x	x	x	
Woo [20]	2017		x			x		x
O'Donovan [21]	2018	x				x		x
Verma [22]	2017		x	x			x	
Trunzer [23]	2017	x	x	x				x
Schuh [24]	2016		x				x	
Nikolic [11]	2017	x		x	x		x	x
Zhong [25]	2017	x	x				x	x
Liu [26]	2018	x	x	x				x
Accorsi [27]	2017			x		x		x
Kartal [28]	2016					x	x	x
Peña [29]	2016			x				x
Wang J. [30]	2018	x	x	x	x	x	x	
Flath [31]	2018	x	x	x		x	x	x
Caggiano [32]	2018		x		x	x		x
Ge [33]	2017		x	x		x	x	
Miškuf [34]	2016	x		x				x
Riedel [35]	2014	x	x	x	x	x		x
Shin [36]	2014	x	x					x
Dutta [37]	2018	x	x			x		x
Nedelkoski [38]	2017	x	x			x		x

Table 2.1. Analyzed articles from Scopus inquiries

Obviously, intelligent technology, like any production politics, has a series of **enabling technologies** that play the active role in making the realization process possible. There's many of them that are going to be better described and analyzed in the next paragraph. Yet, right now the focus is going to be given to the so-called **Big Data Analytics**.

Big Data Analytics, alternatively referred to as *Big Data Analysis*, basically describes the overall environment in which all of the analyzed *key words* take place [30] [31]. This term in literature is often confused and used alternatively with **Big Data**, but a slight difference is perceivable. **Big Data Analytics** can be considered as the overall process of grasping knowledge from disparate data. On the contrary **Big Data** is strictly the data, the huge amount of information of different structure and nature that comes from *continuous interactions among resources, people, (...)* [21]. So, if the first is more general, the

Source	Author	Year	BD	DA	DM	AI	ML	SOA	App.
[14]	Choundhary [39]	2009			x			x	
[24]	Gröger [40]	2012			x			x	x
[40]	Oh [41]	2001			x			x	
[40]	Wang K. [42]	2007			x			x	
[40]	Batanov [43]	1993		x				x	x
[25]	Thoben [44]	2017	x	x				x	
[27]	Li H. [45]	2014		x					x
[30]	Harding [46]	2006			x	x	x	x	
[46]	Shahbaz [47]	2005			x				x
[34]	Lee [48]	2014	x	x				x	x

Table 2.2. Citations by authors of researched papers

latter stands strictly for the data, that are reckoned as *the only possible source of information and knowledge* [11]. Obviously this kind of information needs processing to be useful for different purposes. Also, *Big Data* are very different from traditional data sets, since the latter are structured, whilst the first are completely random and unstructured [26].

Next comes the **Data Analytics** term. This in literature has been often described as the next step of *BD*, basically a more specific and more technical version of it [4] [17]. Therefore, *DA* includes all of the operative steps of the information extracting process.

In some cases, both *BD* and *DA* are gathered in a single big cluster of activities called **Big Data Analytics**, that is the general name that is given to the technology enabler [25]. As cited by Zhong et al., the *Big Data Analytics* environment can be considered the core of collaboration between humans and robots, with the implementation of dedicated algorithms [44].

Also it is very popular and common among articles to define *DA* as the act of translation of data into knowledge to gain a competitive yield, especially from the business point of view [13]. Secondly, *DA* is a means to trigger the decision making process phase [4].

The next *key word* that is going to be analyzed is **Data Mining**. This is generally considered as one of the most specific tasks inside the *BDA* environment. As cited by Wang et al., *DM* is a step of the *Knowledge Discovery in Databases (KDD)*, that involves the application of algorithms for extracting hidden patterns and relationships among data [39]. By doing that we ensure that the results are potentially useful and valuable information [11]. In general, *DM* can be considered as the applied and more specific side of *DA*, that is always referred as the general overall process. Hence, *DM* is limited to the algorithm implementation part of the activity, the operative step [23].

Artificial Intelligence, in turn, is proven to be a term used more transversely in literature. In fact, *AI* is just a *computer science* applica-

tion field, where specific programs are coded in order to be able to solve problems and make decision autonomously. Here *intelligent systems* are created, and, except for some outliers in literature, the overall terminology seems to agree with this meaning of the term [11].

In the end **Machine Learning** is studied. This term, differently from *AI*, is usually defined as the system's capacity to improve its performance and to learn without specific programming. Actually it's not completely different from *AI*, in fact it can be considered as a particular field of application of that.

One of the main characteristics of *ML*, that is usually shared by multiple authors inside literature, is that it can actually not be distinguished and separated from *DM*. As Accorsi et al. declare, *ML*, together with *DM*, play on the pattern recognition of historical datasets, that are used to enhance the performance of the machine. The machine, in turn, is subject and protagonist of a factual learning process [45]. Furthermore, *ML* can be seen, together with a work of statistical analysis, as a logical modeling procedure [20].

Inside the manufacturing environment, lots of **learning techniques** are cited in multiple papers of the inquiries. Here some: Bayesian Networks; Artificial Neural Networks; Support Vector Machine; Linear Regression; Linear Classification; (...).

All of these techniques enable many applications inside the production systems environment. For instance, multiple frameworks for *predictive maintenance* (failures, failure modes, ware) can be implemented [45]. Also, it's possible to create tools to generate *real time supervision* of resources [13]. But the analysis of these technologies is going to be more thorough in the next paragraphs.

2.3.3 Enabling technologies and applications

At this point a showcase of the main applications and technologies used by the researchers who published the articles will follow. This kind of study takes place after having looked for peculiar applications in the articles of the *App.* column of Table 2.1 and Table 2.2, and after having identified and selected them.

2.3.3.1 Enablers

The first step of this analysis is to provide the **enablers** that are used by smart systems and, in retrospective, by Industry 4.0 paradigm².

- **Cyber Physical Systems (CPS).** The overall paradigm of all the applications and case studies is, of course, the *smart manufacturing* one. The first and main scope of this manufacturing policy is

² N.B.: The definition of *Big Data Analytics*, one of the enablers of the Industry 4.0 paradigm, has already been showcased in the previous section.

to enhance performance levels of manufacturing systems themselves and to integrate the *physical* resources with a *cyber* configuration, creating the so called *Cyber Physical Systems (CPS)*. The prerogative of such a system is to be designed in order to have the "*three Cs*":

- **computational** ability;
- **communication** interfaces;
- **control** systems.

In all of the applications found in the papers, these constraints seem to be respected overall.

- **Information and Communications Technology (ICT)**. This enabler refers to the overall growth and need for *interconnection* and *communication* among all of the actors of the, in this case, manufacturing field. Furthermore, the stress is also imposed on the visual communication aspect, including in the ICT paradigm all the assets that allow a real-time data display *e.g.* Human-Machine Interface (HMI).
- **Cloud Computing**. This is a definition that refers to the ability to control a set of resources, like hardware and software tools etc. via *remote servers*. Files and information in general can be kept on a cloud-based storage system, rather than on on local storage devices. Thus, the use of this technology caused a overall enhancement of performance in manufacturing enterprises worldwide.

2.3.3.2 Applications

Probably the most common application of the *Big Data Analytics* enabler is **maintenance** in general. More specifically it is possible to talk about both **predictive maintenance** and **Condition Based Maintenance (CBM)**.

Talking about **predictive maintenance**, this is the field about which the most numerous applications were found inside the studied papers [11] [44] [34]. The whole concept of a *knowledge-based* predictive maintenance paradigm was conceived during the 90s, even before the *BD* etc terms were even born [40] [43]. Several examples of models for the implementation of predictive maintenance have been carried out in the studied articles: predictive planning platforms for both energy consumption and machining time [20]; data science toolbox creation for a predictive information system, including a case study application for *Bosch* [31]; *Prognostic and Health Management (PHM)* for numerous kinds of assets like industrial robot [48]; *Predictive Model Markup Language (PMML)* that are encoded models in factory operations with a description of predictive analysis models in the fog com-

puting paradigm [21]; specific applications of proactive and prescriptive maintenance management models [22].

About **condition based maintenance** and **fault detection and analysis**, the applications space from high-speed packaging machines to inter-department operations [16] [19] [40] [42] [27] [32].

Being in the smart manufacturing environment, there's also the need for **monitoring and performance** tools and indicators [16]. About these needs, several examples can be extracted from the papers: finding abnormal electricity consumption patterns coming from constant monitoring [14]; remote performance monitoring system for a *FESTO* prototypical production line [15]; general energy efficiency study [26] [29]; condition based monitoring of specific aeronautical tools [32]; prediction framework for sustainability performance and power consumption tailored for turning machines [36].

Of course, all of these applications are based on data-driven **decision making models** that are performed **real-time** to comply with the smart manufacturing paradigm [18] [22] [27] [34]. Some examples coming from the papers are: real-time processing module [13]; flexible architecture implementation that allow a near real-time processing for automated production systems [23]; data-driven decision making for predictive planning [20]; sensor-based inefficiency detection that triggers decision making [29].

Naturally, the smart manufacturing paradigm applies in many other fields of the industrial environment, like the **quality and design** area [16] [44]. Here some examples: quality analysis and optimization in real time [40] [41]; data mining for product design constraints and active control on quality [47].

Some other generic examples that don't fall strictly in the ones previously reported, affect the managerial and industrial field more in general, like: **logistics** applications [17] [44]; **inventory classification** [28]; **scheduling** [14]; **supply chain** management [16] [17].

2.3.3.3 *Technologies and algorithms*

Lastly, a brief compendium of the main **enabling technologies** that have been used by the researchers is going to be provided. These are not to be confused with the *enablers* shown before. The enabling technologies exposed at this point are nothing but the **algorithms** and **statistical analysis** that enable the use of technologies like *DM* in the field. In this case the latter is the industrial and business one. Also, the enablers are going to be associated to the related field of application in the manufacturing environment.

- **Clustering and Regression classifiers**: they have been used to create a real-time processing module where data are generated by a digital model of the physical resources [13].

- **Artificial Neural Networks (ANN)** [27]. Largely used technique, it has disparate uses, like: energy monitoring and re-scheduling frameworks [14]; decision making models [20]; sustainability prediction models [36].
- **Bayesian Networks (BN)**. In the papers many uses have been found, for example the decision making one [20].
- **Support Vector Machine (SVM)** used, for example, in decision making as well [20].
- **Regression Tree Models** [27]. Used for performance enhancing frameworks in assembly lines [38].

It is trivial that the witnessed technologies are just a small set of the many available in the application field. Some papers found in literature have a comprehensive compendium of all of this kind of techniques that, for the sake of synthesis, have not been reported in this work of review [19] [33].

	SCOPUS-1		SCOPUS-2		SCOPUS-3		SCOPUS-4		SCOPUS-5	
PROMPT	FIELD	# articles	FIELD	# articles	FIELD	# articles	FIELD	# articles	FIELD	# articles
Bar	DA	26472	DA, DM	5195	DA, AI	2228	DA, ML	2813	DA definition	424
Year	2003 - 2019	26386	2003 - 2019	5191	2003 - 2019	2227	2003 - 2019	2809	2003 - 2019	421
Subject Area	Engineering	6601	Engineering	1290	Engineering	620	Engineering	801	Engineering	107
Key Words	BD, DA, DM AI, IOT, ML	3220	BD, DM, DA AI, ML	1029	BD, DM, DA AI, ML	577	BD, DM, DA AI, ML	621	BD, DM, DA	46
Search within Results	Industry Production Manuf. Factory	835 283 191 51	Industry Production Manuf. Factory	219 61 38 21	Industry Production	127 39	Industry Production Manuf.	139 42 30	Industry	13

Table 2.3. Summary of researches performed on Scopus

DIGITAL TWIN FOR OPTIMIZATION AND CONTROL

In Chapter 1, a thorough explanation of the Industry 4.0 environment context has been given, with specification about the possible and practical uses and advantages of specific tools, such as Digital Twin and Manufacturing Execution System.

In this Chapter, the focus deflects on specific applications of the aforementioned technologies and enablers to carry out **optimization** processes inside specific manufacturing environments. In fact, failures, error states and fault occurrences are a practical issue in factories. Hence, solutions to cope with these problems must be taken into account.

Specifically, the case of assembly lines is going to be analyzed, since the resource that validates this work of thesis is the prototypical assembly line sited in *Industry 4.0 Laboratory* at Politecnico di Milano.

The term *optimization* must be meant in a general way, including all of the techniques that can enrich the application control technologies in manufacturing environment. As a matter of fact, in this Chapter, starting from the description of a set of scenarios, the focus will switch to the importance and the possibility of autonomy of control of production. Specifically, the analysis of autonomous simulation models (Digital Twins) designed to perform such activities, will take place.

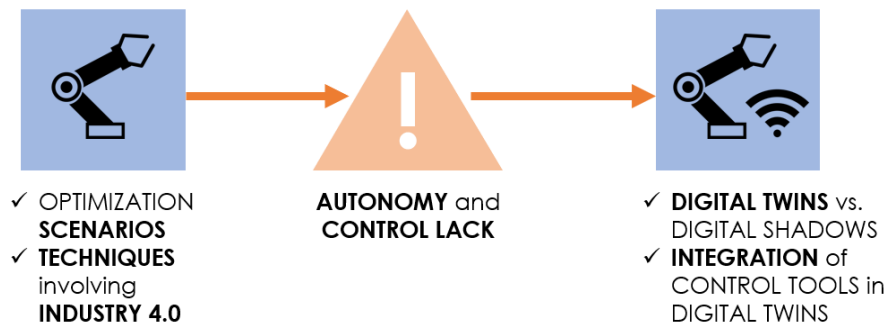


Figure 3.1. Literature study progress

Then, an analysis of the possible *control* and *decision making* models that can be integrated into a Digital Twin paradigm are going to be discussed. The main reason of this process is to close the control loop

and endow the monitoring tool (Digital Twin) with techniques and enablers in order to have a return in active control of the production.

Also, a dissertation about the level of connection of digital assets (*i.e.* Digital Twins) with the physical ones will be provided. This leads to the dichotomy between digital *shadows* and *twins*, where only the latter are endowed with the possibility not only to monitor, but to control the physical resources.

Lastly, the Chapter will analyze the proposal of a practical way to easily and immediately integrate the Digital Twin with Manufacturing Execution System, in order to guarantee a responsiveness of the tool to shop-floor events. A literature context of proposals of integrations and additions is provided as well.

A graphical reference of the literature study progress regarding this chapter is given in Figure 3.1

3.1 OPTIMIZATION SCENARIOS

According to the Industry 4.0 paradigm, manufacturing world is growing and changing fast. This new industrial paradigm gives birth to the so-called *smart factory* concept that transforms production facilities into highly digitalized and connected environments.

Anyway, the interest of companies often lies in simple operative adjustments in order to achieve the simplest and most profitable advantage: *optimization*.

This term, though, should not be taken lightly, since its meaning spaces broadly among industrial and manufacturing environment. As a matter of fact, the need of companies to reach an *optimum* springs from the occurrence of several industrial *scenarios* that, on the contrary, are the main root of inefficiency.

Basically, an industrial situation that needs streamlining can fall in these categories:

- efficiency and effectiveness of resources;
- quality requirements;
- human interactions with machinery;
- planning and scheduling of production orders.

All of these scenarios may, indeed, cause downtime, errors, events that create issues for a smooth operation of the production line.

Besides, when dealing with *smart factories*, the best possible option is to pace the management of this kind of fallacy to the use of smart devices and enabling technologies.

The struggle in coping with this kind of problems is no news in the manufacturing environment. In fact, the novelty lies in the added value provided by the endeavor in using Industry 4.0 enablers.

Here, the *four* possible optimization scenarios are shown.

Lean Manufacturing: Waste reduction & Time optimization

The first scenario is rooted in one of the first and most popular optimization policies conceived: *lean manufacturing*.

Notoriously, the lean manufacturing paradigm reckons on policies of pursuing of perfection by creating the premises for a production process without any of the seven dead wastes: overproduction, waiting, transport, extra processing, inventory, motion and defects [49]. In an automated production line, these problems needs to be coped with, if the company aims at being competitive in the marketplace.

Cottyn et al. propose a synergy of the *lean* paradigm with the Industry 4.0 one [49]. The proposal is the integration of a MES to the production system in order to maintain the process improvements provided by lean policies, enforcing a controlled and standardized way of working.

The solution creates an alignment of smart technologies with the searched optimization solutions, dealing with the error scenarios with a lean paradigm. Therefore, even if merely theoretical, the possibility to enable a constant monitoring, reduction of scraps and reduction of waiting time by using a smart device is proposed [49] [50].

Human interactions with machines

With modern manufacturing concepts, the interaction of humans with machines is a factor that cannot be avoided. Firstly, the human main responsibility is to control and check manually on machines in order for them to work properly. Then there are some activities that the machine can't do by itself, for instance substitution of a tool, or filling of a magazine for production purposes.

Thus, this factor is crucial for a proper working of a manufacturing plant, and sometimes faults occur. In fact the operator, being a human, is not a 100% reliable asset and might fail or just take some time to solve a predicament. That is why this scenario is considered. Humans may be one of the factors for a not perfectly optimized production site.

So, in order to have a stronger yield on production management, solutions that contemplate the use of smart tools can be considered. Even in this case, a MES-based measure can effectively regulate the behaviour of employees within the production procedures. The efficiency enhancement is proven, and the enterprise internal management is optimized [51].

Re-scheduling & Reworking

The occurrence of certain error states, may lead to a re-scheduling of the activity or to a reworking need, due to quality standards that must to be respected.

In fact, due to the complexity of the multi-objective optimization and the large amount of unforeseeable shop-floor events, measures are needed and need to be effective. Some proposals of application of event-driven rescheduling have been done, for example, to avoid excessive buffer occupancy in metal sheet production lines [52].

Disassembly

Another way to cope with errors and badness of quality requirements, in order not to scrap elements, might be the proposal of disassembly plans, if the production line can be reconfigured for this aim.

Frameworks to create and incorporate disassembly methodologies from all levels into different phases of a product lifecycle have been proposed [53]. Although, this kind of framework lacks a reactive nature.

It is to be noticed that all of the just described scenarios, since the optimization proposals contemplate the use of smart tools, may also have positive outcomes in data-based techniques. In fact, since software tools like MES produce and store big quantities of data, several analytic methods to deal with the latter can be conceived, in order to give a greater economic yield to the company [50] [51].

Anyway, all of the studied cases lack of an *event-driven autonomy perspective*, that might be provided by a Digital Twin-based simulation control tool.

3.2 DIGITAL SHADOW VS DIGITAL TWIN

Based on the observations done in the previous paragraph, the only way to have a real-time optimization, based on constant monitoring, is by making use of a Digital Twin. The description of this Industry 4.0 enabler has already been performed in Chapter 1.

According to the aforementioned policies about optimization, a new breakthrough could be given with the introduction of tools like Digital Twins. What is missing for a full application of these optimization decision-making models with a proper "*smart*" meaning, is the *link between physical assets and the digital word*. Such a connection can be provided by the use of Digital Twins. In fact, by creating a twin in the digital world of the physical asset, this gap is filled.

Nevertheless, another missing link comes up at this point. In fact, being this technology at an early development stage worldwide, the application field still lacks some crucial aspects.

In literature, many definitions of Digital Twin has been given, and a straightforward and definitive definition still does not exist (cf. Chapter 1). Kritzinger et al. provide a new perspective to fully understand how a Digital Twin should be defined, and it's based on its **level**

of **integration** within the physical world. According to these levels, three subcategories are conceived [1]:

- **Digital Model** (Figure 3.2): it is the digital representation of an existing or planned physical object that does not use any form of automated data exchange between the physical object and the digital one. Since all the communication happens in a manual way, a change of state of the physical object does not affect directly the digital one, and vice versa.

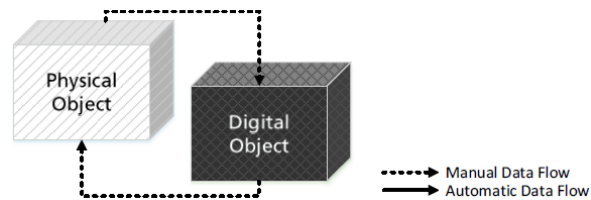


Figure 3.2. Data Flow in a Digital Model [1]

- **Digital Shadow** (Figure 3.3): based on the definition of Digital Model, if there further exists an automated one-way flow between the state of an existing physical object and a digital object, one might refer to such a combination as *Digital Shadow*. If the state of the physical object changes, then the state of the digital object changes as well, but *not* vice versa.

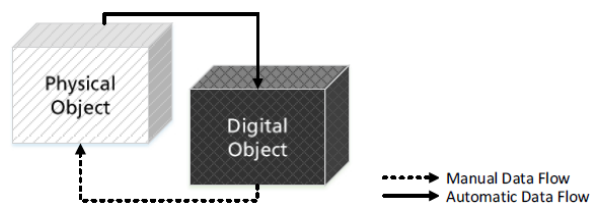


Figure 3.3. Data Flow in a Digital Shadow [1]

- **Digital Twin** (Figure 3.4): if further, the data flows between an existing physical object and a digital object are fully integrated in both directions, one might refer to it as *Digital Twin*. In such a combination, the digital object might also act as controlling instance of the physical object. There might be also other objects, physical or digital, which introduce changes of state in the digital object. A change in state of the physical object leads to a change in state of the digital object *and* vice versa.

According to this classification some observations are worth doing. In fact, based on the current literature and research development, the majority of applications and case studies fall in the use of mere *Digital Shadows*. This is due to the fact that digital models have been used for monitoring means, in order to give a broader understanding of what is going on in the physical world.

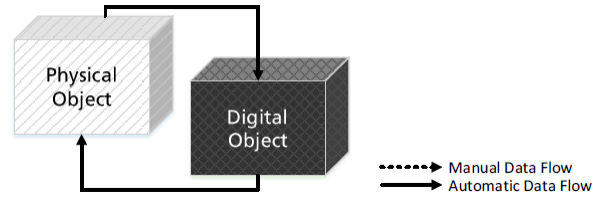


Figure 3.4. Data Flow in a Digital Twin [1]

The link that was mentioned before, is the lack of concrete and tangible applications of Digital Twins, as Kritzinger et al. mean it [1]. Automated and autonomous control can be provided to the physical resource with the implementation of such digital tools. Anyway, there is consensus in research concerning the conceptual basis of the new definition of Digital Twin. There is a tremendous benefit that this new digital tool would bring for industrial applications, but there is a strong lack of case-studies which apply this concept in practice [1].

Summing up, the two big lacks in literature, among optimization techniques, fall in the subsequent categories:

1. missing link between physical and digital world to allow optimization techniques application within industrial environment;
2. lack of applications of Digital Twins (Kritzinger et al. definition [1]), meant as autonomous digital tools that enable an autonomous control decision-making functionality to the currently developed models. The communication of this kind of digital tools is *bi-lateral* with respect to the physical world.

3.3 CONTROL MODELS AND AUTONOMY

Based on the premises done so far, the proposal of a deeper integration between digital and physical sides of an industrial asset is at stake. This embedding procedure enables the creation of a proper Digital Twin with autonomous *bi-lateral* communication between its physical and digital faces.

Even if models of this kind have not been widely implemented and validated in the research field, a big endeavor has been done by researchers to find a way to integrate a Digital Twin real-time simulation tool with **control models** in order to close the control loop. These tools can be based on data, on mathematical models, on tailored rather than generic algorithms. Still, the outcome will remain constant: the digital tool, endowed with control models, has also an **autonomy and control** feature, in order to react to basic shop-floor events. The latter are nothing but the scenarios mentioned in the previous paragraphs. Thus, another capability of the autonomous Digital Twin would be optimization too.

3.3.1 *Autonomy*

Together with the digitalization of every stage of manufacturing, productivity might reach whole new levels. In fact with a mix of *connectivity*, *modularity* and **autonomy**, Digital Twins can have a deeper control in the whole product lifecycle framework, especially in the production phase [54]. As it can be seen in Figure 3.5, the core elements of the Digital Twin scope are the aforementioned; in this paragraph, the stress will be given to *autonomy*.

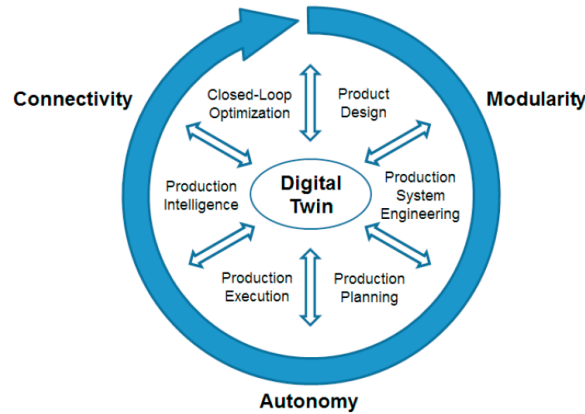


Figure 3.5. Notion of the digital twin in the lifecycle of a product [54]

The challenge is to create *autonomous* systems that are able to control the production process and react to variations during the course of action. Thus, autonomous systems are the key enablers for the new level of flexibility demanded by the latest industrial automation processes [54].

Examples of autonomy that might be provided by Digital Twins exist in literature, but they are still a mere proposal of application that needs experimental validation.

Future endeavor is addressed in order to combine data gathered during service runs and simulations models in order to [54]:

- have systems that support operators and planners during normal operation;
- propose maintenance plans by forecasting models.

3.3.2 *Control models*

The integration of a multitude of models to Digital Twins is possible to provide them with a stronger control and decision-making yield.

Some proposals include the integration of Digital Twins with the shop-floor level in a strong way. In fact, with a synthesis of physical assets, virtual assets, service system and shop-floor data, a big constant monitoring and control framework is proposed [55].

Proposals like this are, in fact, very powerful, since the aid of these integrations is among various levels of the product lifecycle, not only on the production side. Some proposals of examples are [56]:

- resource allocation;
- corrective actions based on optimization strategies;
- proposal of real-time production strategy and planning.

Other models fall in the **corrective maintenance** field. For instance, by using engineering data of the system and envisioning visual assistive technologies, fault ascription techniques and corrective proposals using the digital twin is proposed [57].

Next, models that include the use of Big Data to create **predictive models** in Digital Twin-based frameworks are studied. Specifically, the use of these tools is proposed for several application fields:

- product design [58];
- product planning [58] [59];
- management of shop-floor data and events [59].

3.4 INTEGRATION OF DIGITAL TWIN WITH MES

In the previous paragraph, a synthetic overview of all the possible integrations of Digital Twins to predictive and Big Data-based models is given. Being this whole world at an early development stage, these integrations remain mere proposals, at least within simulation models that gather data in real time from industrial systems.

A first step towards the implementation of these models is analyzed in this section, and it lies in the proposal of **integration of Digital Twins to Manufacturing Execution Systems**. Such an integration endows the Digital Twin with a direct control decision-making functionality within industrial systems. This capability of the Digital Twin to react directly and autonomously to shop-floor events can be seen as an initial step towards the integration of all of the other aforementioned control models that act on a wider scale.

In literature, some references hint a work of integration of a Digital Twin with MES.

Liu et al., for instance, developed a Digital Twin-driven methodology for rapid individualised design of automated flow-shop manufacturing systems. The future open issues regard possible integrations of a MES to the digital twin model in order to generate detailed work instructions associated with process plans. Also, such an integration would build a more comprehensive model that provides a more robust decision support [2].

Analyzing also other steps of the asset lifecycle, Digital Twins can be used also for a rapid approach to design production lines from the beginning. Those systems aim to have control ability over the whole product lifecycle management; this includes also operations and shop-floor management issues. Proposals come from the research environment to integrate MES to the Digital Twin to cope with operational issues on production lines [60].

Further investigations have been carried out, specifically trying to understand how to combine Digital Twin and tools like the MES in order to improve the *smart factory* paradigm.

For instance, to enrich connectivity, pro-activity and agility of production, Digital Twins can be combined with automated tools in order to improve the "*smart factory*" environment [61]. Besides, machine control via real-time monitoring is reckoned to be a critical milestone to enable distributed smart factories and smart manufacturing. Accordingly, a considerable endeavor is invested in these research fields [62].

It is to be noticed that such integrations can be done by allowing communication between the Digital Twin and MES software. Hence, as already anticipated in Chapter 1, a MES is usually endowed with communication means and protocols in order to be reachable from other assets within the production plant. This technique has been used in this work of thesis, and the integration has been performed by making use of *communication protocols* like shown in Chapter 7.

RESEARCH DESIGN

In this Chapter, the whole literature work done so far is going to be analyzed. The focus will be given to the investigation about the presence of *gaps* in the research context. Besides, a set of objectives is provided in order to fill the gaps found in the research environment with the proposed work of thesis.

4.1 RESEARCH GAPS

In Chapter 1 an historical context about the constantly growing application of Industry 4.0 technologies and enablers has been given. In particular, the use of Digital Twins and MES has been analyzed, together with the industrial context in which both of these tools are located.

A systematic literature review about the methods that Digital Twins and other enablers use to process data has been performed (Chapter 2). Accordingly, the study noticed a set of lacks about the connection between physical and digital assets, specifically about the autonomous decision-making control capability of the latter by making use of Industry 4.0 enablers with shop-floor optimization purposes (Chapter 3).

Hence, the study led to the definition of *four* gaps.

- **Gap 1:** The majority of applications in literature are *Digital Shadows*, meant as digital models where an autonomous data flow takes place only from the physical asset to the digital one, and *not* vice versa. There is a strong lack of case studies about Digital Twins, meaning digital models that have autonomous data flow from the digital side to the physical one *and* vice versa [1].
- **Gap 2:** There is a strong lack of autonomy of Cyber Physical Systems in performing shop-floor actions. To be profitable, the CPSs world — where Digital Twins are placed — should be endowed with a more autonomous decision-making degree.
- **Gap 3:** Optimization models and solutions discovered in literature are found to be possibly based on the integrated use of automation tools like MES. Still, the use in the CPS environment — where Digital Twins are located — is still widely lacking.

- **Gap 4:** The integration of a MES software with a Digital Twin model has been proposed in literature to have a more relevant control and decision-making ability on industrial production environment. Though, still no applications regarding this procedures have been carried out.

4.2 RESEARCH OBJECTIVES

Given the just stated research gaps, the presented work of thesis proposes to fill them up with the modelling of a real-time simulation tool.

Provided the literature gaps, a *main research objective* for the work of thesis is declared. Based on the definition of this principal objective, *two sub-objectives* are provided.

The development of all the objectives — both "*main*" and "*sub*" — has a strong literature base. Still the classification has been introduced to show the dependence of the *sub-objectives* on the *main objective*. Indeed, the sub-objectives can be seen as a possible development based on the crucial and indispensable fulfillment of the main objective.

- **Main Objective:** Create a Digital Twin, able to allow communication between the digital and physical side in *both directions*.

The digital models found in literature are nothing but *Digital Shadows*, which are able to gather data from physical assets, but without any sort of control or decision-making ability over the machinery (Figure 4.1a) [1]. With the completion of this objective, thanks to the MES integration functionality to be embedded in the Digital Shadow, communication becomes *bi-lateral*, like in Figure 4.1b. Thus, the Digital Shadow turns into a *Digital Twin* [1].

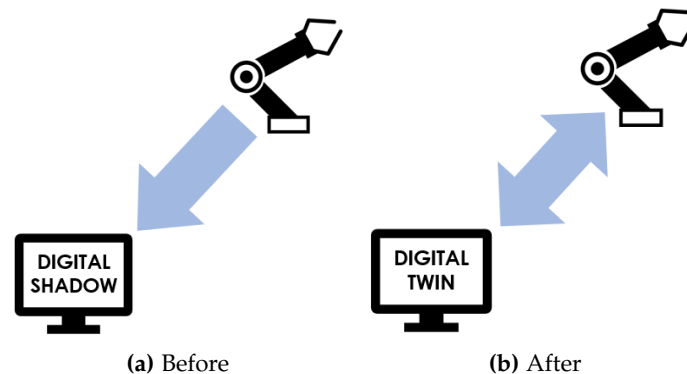


Figure 4.1. Enhancement of the Digital Shadow to a Digital Twin [1]

- **Sub-Objective 1:** Optimization and better performance of the assembly line together with an autonomous decision-making control ability.

Optimization measures have been carried out within the laboratory environment to validate the proposed Digital Twin model. These concern:

- **error states management;**
- **human - machine** interaction;
- **re-scheduling;**
- reactive **disassembly.**

- **Sub-Objective 2:** MES functionality improvement in terms of *Process Management*, based on the MESA International normative.

Process Management: "*directing the flow of work in the plant based on planned and actual production activities*"[3].

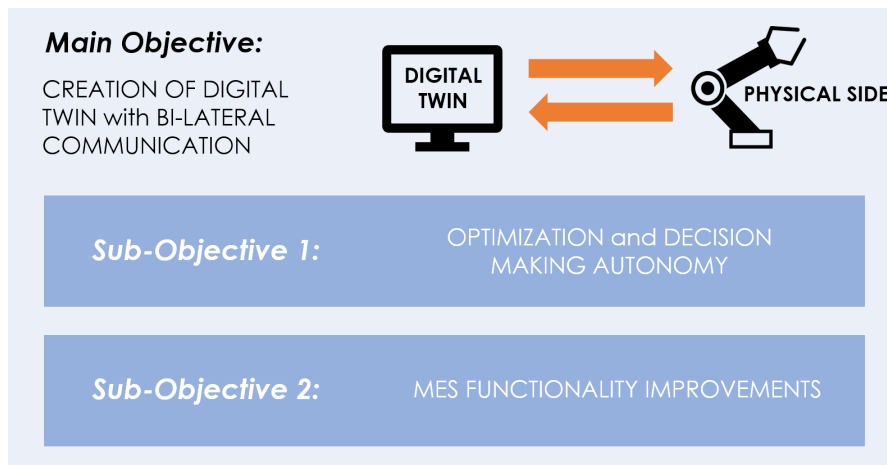


Figure 4.2. Graphical representation of the objectives

A graphical representation of the development of the objectives is given in Figure 4.2. Then, a schematized representation of the development stages of the work in order to fulfill the objectives is depicted in Figure 4.3.

4.3 PRACTICAL OBJECTIVES AND APPLICATION CASE STUDY

Starting from the development phases of Figure 4.3, in Table 4.1 a reference about the specific practical steps carried out in the work of thesis, together with the application case studies in *Industry 4.0 Laboratory* is given. Besides, reference about the methodology adopted to perform the steps and the relative location in the body of the thesis is given.

For the sake of completeness, a background on the already implemented models in the *Industry 4.0 Laboratory* at Politecnico di Milano must be given.

A simulation model of the production line in the laboratory has already been modelled and validated. Adopting the terminology of Kritzing et al. [1], the functioning of this *Digital Shadow* introduces this kind of operations:

- connection of the simulation model to the sensors on the line via OPC UA communication protocol;
- gathering of energetic data from the PLCs of workstations, with relative energy consumption performance evaluation;
- via a combination of the values of the sensors, provide the **machine state** of the workstations in real-time;
- implementation of a Graphical User Interface (GUI) that sums up all of the previous features of the model.

The study of this tool (Digital Shadow) gave the foundations of the practical development stages of the work of thesis (cf. Table 4.1).

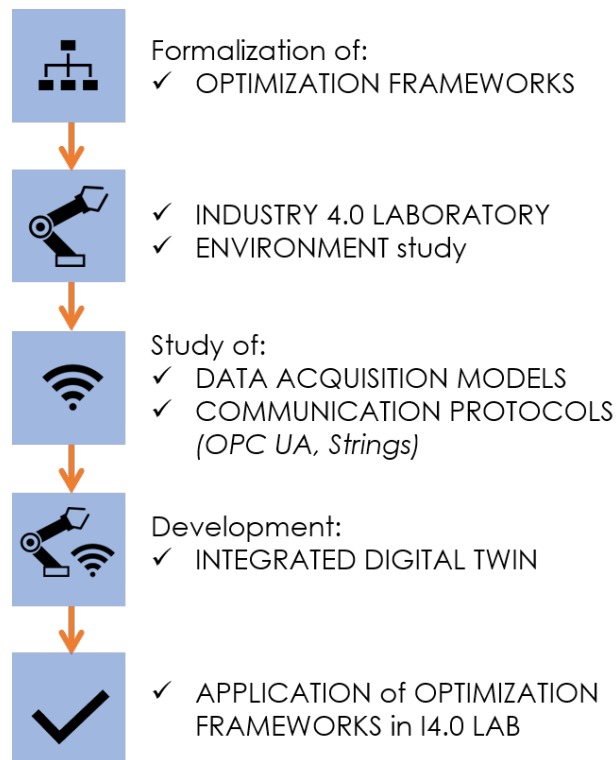


Figure 4.3. Development phases of the work of thesis

Step	Method	Chapter
Proposal of optimization frameworks	Modelling of operational frameworks and Digital Twin prototypes	5
Study of practical environment	Introduction to Industry 4.0 Lab	6
Real-time data acquisition	<i>Matlab & Simulink</i> data acquisition with OPC UA	6.5
MES function study	Introduction to MES in I4.0 Lab	6.6
MES communication study	MES communication protocol in I4.0 Lab environment	6.8
MES integration	Communication protocol integration in <i>Matlab & Simulink</i>	7
Error states model detection	Experimental classification of error states	8.1.1
Application of optimization frameworks	Modeling of tailored Digital Twins in I4.0 Lab environment	8
Validation of the model	Showing the results that come from integration of MES to existent Digital Shadow	9.1 9.2

Table 4.1. Practical development steps of the work of thesis

PROPOSED MODEL DEVELOPMENT

In this chapter two general **optimization frameworks** are modeled and proposed.

This kind of frameworks are put forward to fulfill the *optimization scenarios* unveiled in Chapter 3 using *smart simulation tools* like **Digital Twins**. Specifically, the frameworks also give a general perspective on a methodology that can be adopted in a general manufacturing environment to fulfill the research objectives defined in Chapter 4. In particular the creation of a Digital Twin that is able to communicate with its physical counterpart in *both directions* is proposed with the definition of these frameworks.

Then, the optimization aims that can be reached by the use of a Digital Twin that is tightly integrated to its physical counterpart, as already stated, are:

- **error states optimization;**
- **human-machine** interaction;
- **re-scheduling** and **disassembly**.

Subsequently, in paragraph 5.1, a framework to generally deal with error states in a manufacturing system is proposed, while in paragraph 5.2, a similar kind of framework is given to reactively schedule a disassembly order for pieces that do not comply with given standards.

These activities are carried out by analyzing the possible interactions between:

- *physical* level;
- *digital (simulation)* level.

This kind of frameworks are conceived to prove the possibility of *bi-lateral* communication between the two aforementioned levels of an industrial resource.

5.1 ERROR STATES MANAGEMENT FRAMEWORK

This first framework proposes the integration of physical and digital sides of an industrial manufacturing equipment with the aims to optimize the possible occurrence of **error states** that block the production flow.

5.1.1 Nature of error states

At first, some observations about the nature of these error states must be given.

First of all, the error states considered in the proposed framework are those that **block the production** in the physical asset. Hence, a procedure to solve this kind of errors must be performed in the facility. This kind of operations are often carried out *manually* by the operators in charge of monitoring the production flow.

At this point, a categorization of two possible kinds of errors that can be encountered is provided.

- the **type A error** concerns the fault states in which the intervention of an operator is *mandatory*. The kind of errors of this nature may involve, for instance:
 - filling of magazines;
 - operations to be manually performed on software tools;
 - substitution of tools for wear reasons;
 - (...)
- the **type B error**, in turn, is defined as an error state that can be fully solved using automated digital solutions, like:
 - sensors overwriting;
 - communication protocols;
 - (...)

Secondly, error states can be categorized according to the **impact** that they cause in terms of downtime while they are being solved. The impact can be either **high**, **medium** or **low**.

5.1.2 The error states management framework

Provided the possible error states, the proposed framework is shown in Figure 5.1. As the reader may notice, the solving of the error states is performed in an integrated way, exploiting both physical and digital parts of the machinery.

The framework must be read like this:

1. When an order arrives in a certain workstation, the checking phase to verify if the processing conditions are met is performed. If the order can be processed, then it is worked. If not, an **error state** is taking place.
 - If the error is of **type A**, then the simulation tool steps in to help the operator notice that there is an error state going on in the physical asset. When the operator solves the error, the simulation tool proceeds at finishing the operations needed to free the station by the stuck order.
 - If the error is of **type B**, the simulation tool intervenes right away in solving the error state and in clearing the workstation.
2. Once the error state is solved, then the flow of operations can resume.

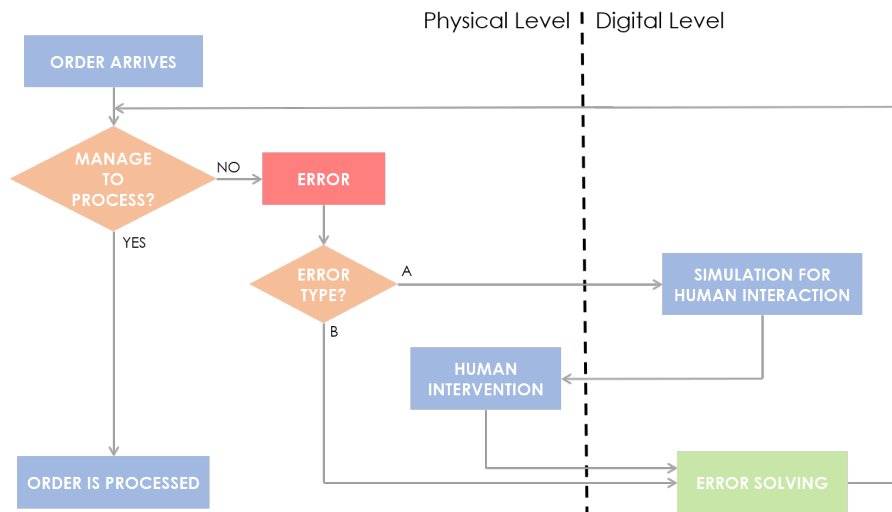


Figure 5.1. Proposed framework of integrated management of error states with a digital simulation tool

Based on the running of the proposed framework, *optimization* can be found in two separate aspects:

- the **downtime** relative to the solving of the error state is *reduced*, since the Digital Twin ensures its fixing in a shorter time.
- the **human-machine** interaction is *enhanced*, since the simulation tool takes measures to help the operator notice the occurrence of the error state.

5.1.3 Simulation model proposal

For the application of the proposed framework, the use of simulation tools can be considered to create a Digital Twin capable to tackle this kind of operational issues.

The work of thesis proposes a model for a Digital Twin like in Figure 5.2, that can be encoded in *Simulink* environment, with the use of functions that perform this kind of activities:

- A function that *reads the sensors values* embedded in the production system and that might eventually change their value, named "**Detector**". For this work of thesis, the use of *Level-2 Matlab S-Functions* has been adopted, since the Digital Twin has been modeled on Simulink. The use of this kind of functions is explained in Chapter 6.
- A second function that take as an input the sensors values and combine them in order to identify a set of *trigger states* that, in the analyzed case, coincide exactly with the *error states*. This function can be called "**Solver**" since, according to the specific error state, takes autonomously operational measures to solve the error state in question. Eventually, the function closes back the loop with the "**Detector**" function, to be able to control the actuators and sensors on the line once having solved the error. One of the measures that can be taken by the "**Solver**" function might be the use of communication protocols to send commands to the MES software. This method is the one adopted in the work of thesis.

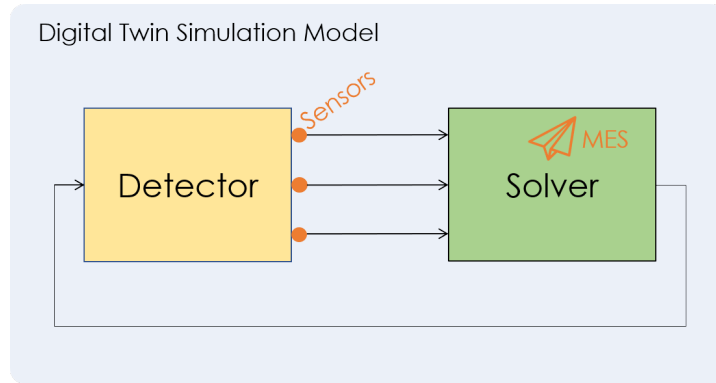


Figure 5.2. Simulation model basic functions for application of error states framework

5.2 REACTIVE DISASSEMBLY FRAMEWORK

This second framework proposes an integration of physical and digital sides with the objective to optimize the production process in

terms of **reactive disassembly** in cases in which compliance with respect to certain standards is not respected.

For the application of this kind of framework, the production facility must be endowed with:

- a workstation in which an **assembly** process can be performed.
- a workstation where the **check** on compliance and/or quality can be carried out.
- a workstation in which the configuration of a **disassembly** process can be done. It can either be the same as the assembly station, if the flexibility of the resource is high, or also a second workstation dedicated to the occasional disassembly process.

5.2.1 The reactive disassembly framework

In this case, the framework — in Figure 5.3 — has a simpler way of working:

1. The order, already assembled, arrives at the station in charge of *checking compliance*. If conformity is verified, the process continues, otherwise it does not.
2. When compliance is not verified, the simulation tool aborts the current workplan and takes action by reactively scheduling a *disassembly* plan on the same piece.

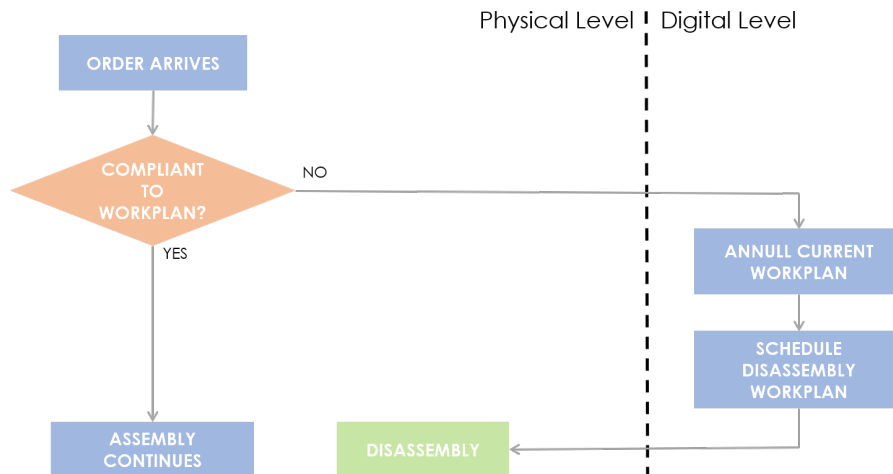


Figure 5.3. Proposed framework of integrated management of disassembly with a digital simulation tool

For this framework, *optimization* lies in:

- the autonomous **reactive scheduling** of a **disassembly** plan, whether certain compliance conditions are not met.
- an overall **scrap reduction** with possibility of materials recovery.

5.2.2 Simulation model proposal

Even in this case, the modeling of a tailored simulation-based Digital Twin is proposed to deal with the reactive scheduling of disassembly processes (cf. Figure 5.4).

The function to be embedded in the Digital Twin are these:

- A "**Detector**" function that, like in the previous framework, is able to monitor and overwrite the values of the embedded sensors in the facility.
- A function that processes the values coming from the "*Detector*" and identifies the conditions in which the reactive scheduling of a disassembly process is necessary. Provided its application field, the function can be called "**Disassembler**". Also in this case, the possibility to perform these tasks can be enabled by creating a communication paradigm with the MES, like in the proposed work of thesis.

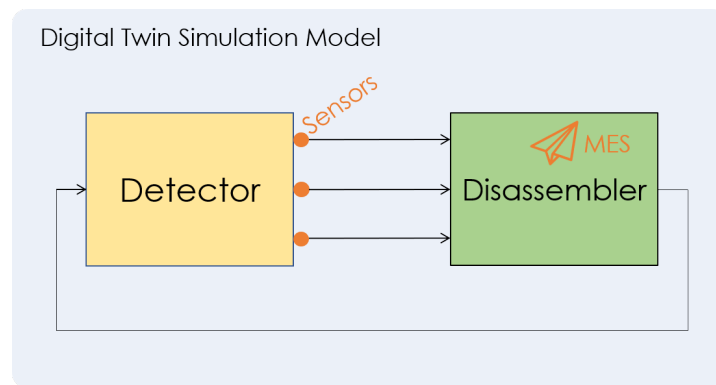


Figure 5.4. Simulation model basic functions for application of reactive disassembly framework

The application of the proposed frameworks, together with the implementation of properly tailored Digital Twins, is shown in the next Chapters. The development context, the exploited technologies and the results coming from the applications of the integration model are also provided.

INDUSTRY 4.0 LABORATORY — ENVIRONMENT & TECHNOLOGIES

In this chapter an overview of the *Industry 4.0 Laboratory (I4.0 Laboratory)* at *Politecnico di Milano* is going to be given. This chapter has been written since the application of the optimization frameworks proposed in Chapter 5 has been performed in the I4.0 Laboratory environment.

The Chapter is divided into the description of the **environment** (paragraphs 6.1, 6.2, 6.3, 6.4) and of the used **technologies** (paragraphs 6.5, 6.6, 6.7 and 6.8). Notice that in paragraphs 6.6, 6.7 and 6.8, the full explanation of the use of MES on I4.0 Lab is given, together with the technologies and communication protocols exploitable to exchange information with it from external sources.

6.1 LABORATORY RESEARCH OBJECTIVES

The I4.0 Laboratory was a specific commission of the *Manufacturing Group* at the *Department of Management, Economics and Industrial Engineering*¹ of Politecnico di Milano.



Figure 6.1. Location of the I4.0 Lab at PoliMi

As Fumagalli et al. said in 2016, whilst different and innovative technologies emerge nowadays, research and education are pushed to conceive new needs on the smart manufacturing field. Hence, the need to address this context in a significant way grew. So, the

¹ Dipartimento di Ingegneria Gestionale (DIG)

availability of a physical setting in an academic environment of this modern manufacturing concept is the prerogative to promote leading edge research and educational activities [63].

As a matter of fact, the I4.0 Laboratory, that is now in full effect active at Politecnico di Milano, allows the field application of several kinds of research and development activities, even if limited in a small scale perspective. This is because the resources available at I4.0 Lab are strictly didactic and not feasible for serial production.

The possible activities to be performed at I4.0 Lab fall in these categories:

- **research** projects, and related field activities;
- **theses** development;
- **didactic training** for academic courses;
- **communication and consulting**.

Furthermore, it is to be noticed that, in order to adequately build engineering and management competences in the I4.0 Lab, a set of major components needs to be considered in a holistic way within the manufacturing ecosystems. These are: the human factor; the product; the process/plant. Furthermore, the I4.0 Lab wants to address these special aspects in a flexible and scalable environment. [63].

Summing up, the I4.0 Lab aims to create a physical and tangible entity that enables the merging of research activities and innovative manufacturing management and planning approaches. The silver lining dwells in the possibility to apply everything in a "real-like" environment [63].

6.2 PRODUCTION LINE

The production line has been configured and installed with a mix of resources and machinery by *Festo*, *Mitsubishi Electric*² and *Siemens*³.

The objective of the line is to assemble a simplified mobile phone prototype.

Under a comprehensive point of view, the composition of the production line is *modular*. As a matter of fact, it is made of **7 workstations**, each one with a different task and function to assembly the final product prototype. A graphical representation of the production line layout, with a number for each workstation, is provided in Figure 6.2. Also, in Table 6.1 a description of the specific assembly tasks of each workstation can be found.

In turn, switching to a more technical and architectural point of view, each workstation is composed of an aggregation of two separate *CP* modules:

² <http://it.mitsubishielectric.com/it/>

³ <https://www.siemens.com/it/it/home.html>

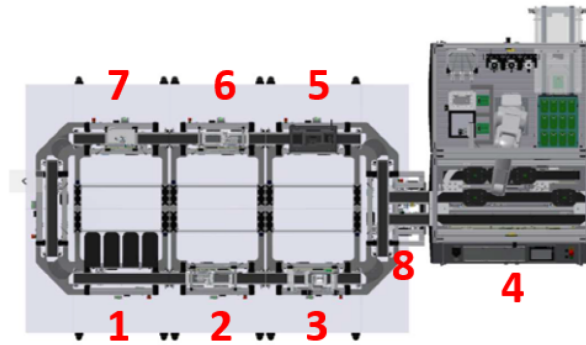


Figure 6.2. Layout of the line, with a number for each workstation.

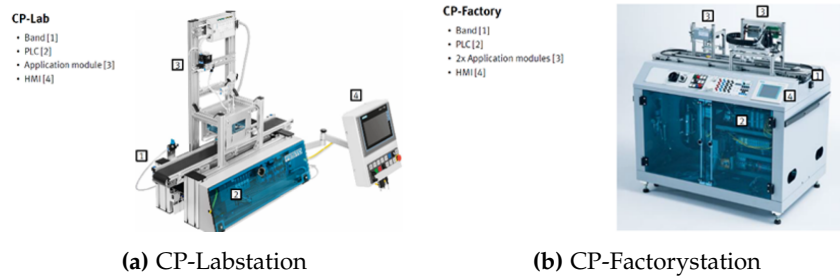


Figure 6.3. Automated process structure

1. The **CP-Labstation** module (showed in Figure 6.3a). It is easier to reconfigure since it has rollers under the station and a single cable connection, but its size is very limited, so the allocation of only one application module is allowed. This module is installed in the *I4.0 Lab* in workstations 1,2,3,5,6,7 (cf. Figure 6.2).
2. The **CP-Factorystation** module (showed in Figure 6.3b). Its dimensions are bigger with respect to the CP-Labstation module, as it is shown in Figure 6.3b. Hence the use of multiple application modules is permitted in the same station. Still, it is more difficult to reconfigure. Within the production line, this module is the one installed in position 4 (cf. Figure 6.2).

The application modules are put together to form a production line in the *I4.0 Lab* as showed in Figure 6.2. It is to be noticed that the production line follows the steps to produce the cellphone prototype, whose components are illustrated in Figure 6.4b.

Both the *CP* modules are equipped with:

- band;
- Programmable Logic Controller (PLC);
- Application module;
- Human-Machine Interface (HMI).

Number of the station from Figure 6.2	Station Type	Working Step
1	Manual station	The operator can interact with the machine and withdraw the finished piece
2	Front cover magazine	The front cover is placed in an empty pallet
3	Drilling machine	The drilling operation is executed in the cover positioned on the pallet
4	Robot Assembly cell	The PCB is placed in the front of cover of the phone and the fuses are embedded in the PCB
5	Camera Inspection	The camera inspection checks if the PCB and the fuses are well positioned in the cover
6	Back cover magazine	The back cover is positioned on top
7	Press module	The back cover is pressed to close the piece

Table 6.1. Function of each workstation in the production line

Also a *CP-Bridge* is used in position 8, as shown in Figure 6.2. Besides the line is not equipped with buffers, since the only handling system is a conveyor that goes through all the workstations. In the system, 8 carriers move around the line. These carriers are stopped in each module to *read/write* their ID information via a RFID technology.

Thus, the production line is connected to two computers that have this function:

1. one provides a connection to the **MES**, that enables the launch of production orders and the control of the production itself;
2. another allows the monitoring of **energy consumption** with specific applications.

Both computers store the recorded historical values on an *Access* database. it is possible to configure this database to store even more sensor values.

Lastly, all of the resources (CP-Lab, CP-Factory, computers) have a unique IP address that makes the connection to the *I4.0 Lab* possible.

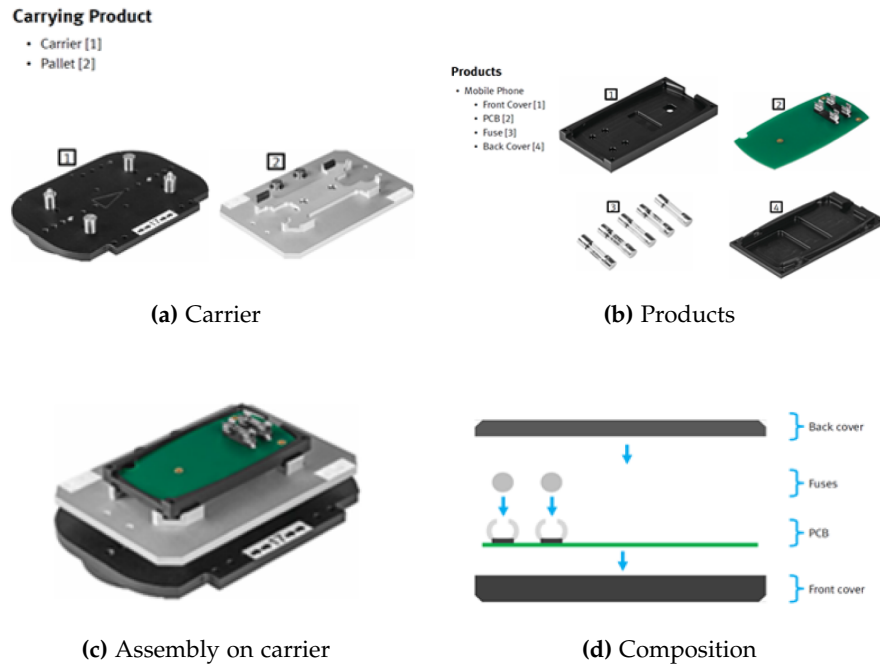


Figure 6.4. Product features

6.3 PRODUCT

As aforementioned, the line is designed to produce a **mobile phone prototype**. Like in Figure 6.4a, there's a carrying system that is made of a *carrier* and a *pallet*. Each carrier has a unique ID number, whilst the pallet is needed for mere production support.

As shown in Figure 6.4b, there are four separate parts that compose the final product: a front cover; a Printed Circuit Board (PCB); fuses; a back cover. The assembly of the parts is shown in Figure 6.4d. Thence, the product, assembled by the workstation, is placed on the pallet that is moved around by the carrier as in Figure 6.4c.

Overall, the main standard production for a part, that is generated by the MES computer, follows this sequence of operation: 2 - 3 - 4 - 5 - 6 - 7 - 1 (for reference, check the plant scheme in Figure 6.2). Lots of different production types can be programmed and launched from the MES installed in the laboratory. The amount of combinations to assemble different products is very large and is identified with a production code number. Hence, according to the user's needs, a different kind of product can be assembled *e.g.* production with 2 fuses, production with no fuses, only drilling etc. Besides, once the production is launched on the MES and is being physically executed on the line, an interface on the MES follows and monitors the progress of each production work along the whole line.

6.4 WORK PROCESS

As already mentioned in the previous sections, each of the seven workstations carries out a different task on the piece that is being assembled on the carrying product (Figure 6.4c). Once the carrier arrives to a specific workstation, the steps to assign the work to it and let it continue its production plan are exactly the same for each workstation. This is because each station is embedded with the same *sensors* and *actuators* in the working position, as it is shown in Figure 6.5. The presence of the aforementioned in the working position allows the communication between the piece that is being processed and the MES that is controlling the production orders.

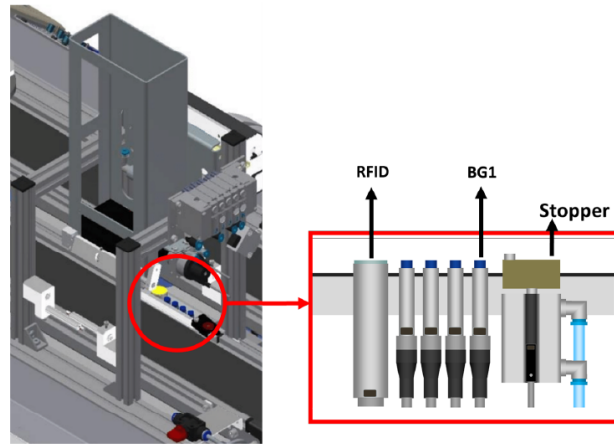


Figure 6.5. Detail of the working position for the Front Magazine station

Accordingly, to understand how the work is executed in each station, and which are the functionalities of the sensors and actuators in Figure 6.5, an example is going to be given. Specifically, for the *Drilling station* (workstation number 3 in Figure 6.2) the communication between the MES, where the work steps are assigned, and the piece worked in the station is going to be analyzed.

For the *Drilling station*, there are three different possible working programs, identified in table 6.2.

Program	Description
1	Drill left side of the workpiece
2	Drill right side of the workpiece
3	Drill both sides of the workpiece

Table 6.2. Possible working programs of the drilling station.

The procedure adopted to *identify* the workpiece and to *assign* the work to it is the following:

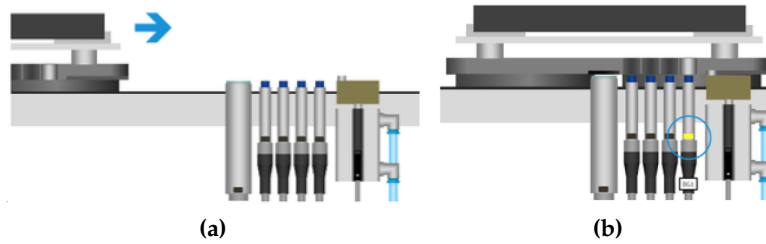


Figure 6.6. Working assignment at Drilling station: first step

1. The stopper is waiting for the carrier in Figure 6.6a. At this point, the first inductive sensor, called BG1, shows if the carrier has arrived, then it is stopped in the working position, as in Figure 6.6b
2. The RFID sensor reads the information coming from the RFID-tag, and the code 1 is shown if the piece is ready to be worked (cf. Figure 6.7).

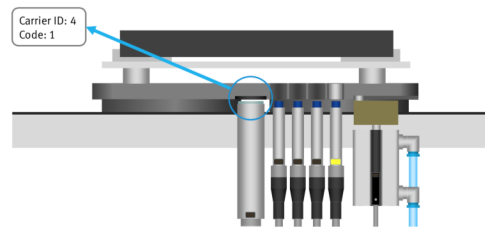


Figure 6.7. Working assignment at Drilling station: second step

3. Now, the *transition table* that can be accessed through the HMI, is checked. Here, the code 1 denotes the starting condition, meaning that the piece is waiting to be worked (cf. Figure 6.8). Hence, the code 1 is associated with the 3rd program, that means that the drilling will be carried out for both sides (cf. Table 6.2 for program numbers for the drilling machine).

Carrier ID: 4
 Code 1

No.	Start condition	Application execute	Prog. No.	Parameter	End condition
Init	none		0	0	0
1	1	✓	3	Program 3: Drill both	
2	0		0	0	0
3	0		0	0	0
4	0		0	0	0

Figure 6.8. Working assignment at Drilling station: third step

4. Since the identification showed that the piece is ready to be worked, and the program code has been assigned to the work-

piece, the application can be performed. The drilling will be done for both sides, as in the program showed in the previous step (cf. Figure 6.9).

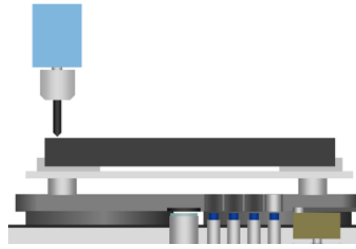


Figure 6.9. Working assignment at Drilling station: fourth step

5. At this point, the *transition table* can be checked in order to wait for the "*end condition*". At the end of the working process within the station, the code changes from 1 to 2. The number 2 stands for "piece worked" (cf. Figure 6.10)

Carrier ID: 4
Code: 1

Carrier ID: 4
Code: 2

No.	Start condition	Application execute	Prog. No.	Parameter	End condition
Init	none		0	0	0
1	1	✓	3	0	2
2	0		0	0	0
3	0		0	0	0
4	0		0	0	0

Figure 6.10. Working assignment at Drilling station: fifth step

6. The end condition is written back to the RFID-tag of the work-piece (Figure 6.11).

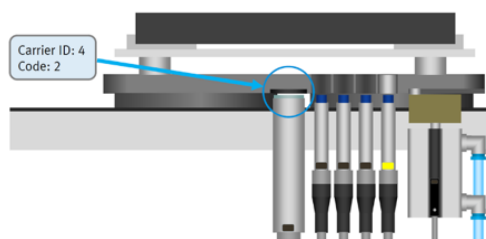


Figure 6.11. Working assignment at Drilling station: sixth step

7. The stopper is released, and the carrier is allowed to proceed to the next workstation. The station can be considered *idle* until a new carrier enters in the station, and the BG1 sensor gives an input to start the process once again (cf. Figure 6.12).

To have a more thorough perspective of the *I4.0 Lab*, the tasks performed for each station are the following (cf. Figure 6.13):

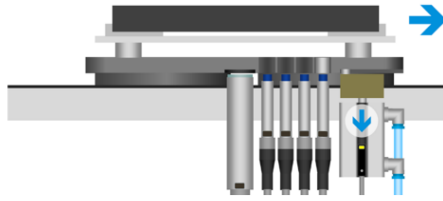


Figure 6.12. Working assignment at Drilling station: seventh step

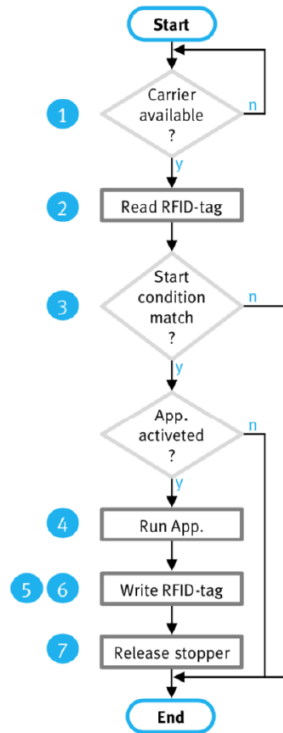


Figure 6.13. Working steps

1. Wait for the carrier arrival: BG1 sensor evaluates the availability of the carrier;
2. Read the RFID information from the RFID-tag: Carrier ID and stard condition of the status code;
3. Check the *transition table*:
 - if there's a raw piece to which the *start condition* is assigned, the application is enabled, and steps 4 - 5 - 6 - 7 in the figure are performed;
 - if there's no raw piece, or the workpiece has already been worked (cases in which the *start condition* is not assigned), the application is not enabled and the workflow goes directly to step 7.
4. Run the correct application, depending on the workstation;

5. Check the *end condition* of the actual row in the *transition table*;
6. Write the *end condition* back to the RFID as a status code;
7. Release the stopper, allowing the carrier to go through the next task.

6.5 OPC UNIFIED ARCHITECTURE (UA)

In the *I4.0 Laboratory* one of the ways that communication is provided, is a resource called **Open Platform Communication (OPC) Unified Architecture (UA)**. This is basically a *Machine-to-Machine (M2M) communication protocol*.

The OPC (usually referred to as *OPC Foundation*) is basically an "*industry consortium that creates and maintains standards for open connectivity of industrial automation devices and systems, such as industrial control systems and process control*"⁴.

This paradigm was firstly defined as a Microsoft DCOM standard base. Then, to enable the communication with non-Windows operative systems, different OPC standards were introduced, until the final solution that has been provided by the **Open Platform Communication Unified Architecture** specification. This specific solution was published as an IEC 62541 standard⁵.

Thus, the *OPC Unified Architecture* enables this kind of operations, that were considered as a main needs in the years immediate before the release of this kind of protocol:

- communication with systems and software tools like MES and Enterprise Resource Planning (ERP);
- show the value of some process variables during events generated by real-time processes.

So basically, the main focus of this protocol is the communication between industrial equipment and systems for data collection and control. The way OPC UA operates is by creating a *client/server* connection, like shown in Figure 6.14.

Having this kind of tool inside the *I4.0 Lab* helps to understand practically how such a tool might help under a manufacturing point of view. It is possible to exploit it in the laboratory using a various set of engineering tools, such as *MATLAB* and *Simulink*; like this the connection to the communication level granted by OPC UA is guaranteed and used.

Anyway, apart from the educational purposes of the laboratory that trigger the use of *MATLAB* and *Simulink*, many other purposes can

⁴ https://en.wikipedia.org/wiki/OPC_Foundation

⁵ <https://webstore.iec.ch/publication/21987>

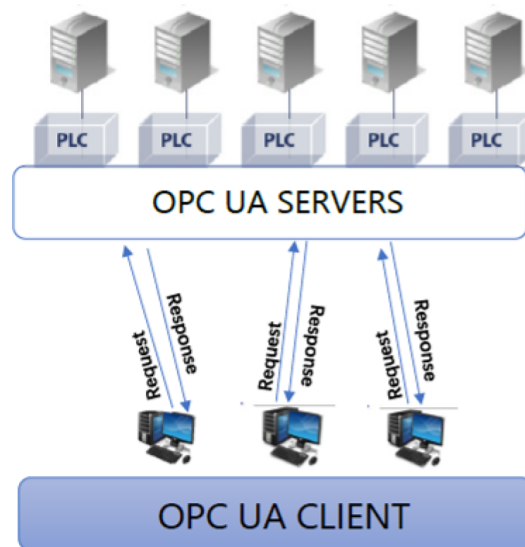


Figure 6.14. OPC UA client/server connection

be given to the OPC UA protocol. As a matter of fact, OPC UA is supported in several different languages and with different licences, such as `open62541`⁶, that is based on C language. Like this, many open source implementations of OPC UA that support both server/client creation have been implemented. Though, there are other implementations that support only a client instance, like the C-based `opcua-client`⁷, the one used in *MATLAB*⁸ and C++/Python based `uaf`⁹.

OPC UA supports two protocols: the binary protocol `opc.tcp://Server`; the protocol for Web Service `http://Server`.

The OPC information model bases its processing on a *node-based* network, in which the *node* is the basic concept of the whole architecture. These nodes actually build a meta model that creates a way to communicate a big variety of meta information. Hence, each node has a set of characteristics and attributes, and for each attribute there's a possible value or entry. OPC UA guarantees also a server address space (*OPC UA AddressSpace*) where all of the nodes are created and managed. Each node in this space belongs to a specific class, and can be used to represent any kind of information, including variable instances and types. The type of data built in OPC UA and supported by it is numerous [64]. For the specific use done in the Laboratory environment, refer to Table B.1 in Appendix B.

Besides, OPC UA gives a lot of importance to the secure communication between OPC UA Client and Server. Through the creation of a *Secure Channel*, the confidentiality of integrity and application authentication is guaranteed [64].

⁶ <https://open62541.org/>

⁷ <http://www.openopcua.org/>

⁸ <https://it.mathworks.com/products/opc/features.html##opc-ua-data-access>

⁹ <https://github.com/uaf/uaf>

There are a lot more information about all the functions and running mechanisms of OPC UA protocols, but for the sake of brevity they're not being reported. For details, refer to the bibliographical reference number [64].

6.5.1 OPC UA in I4.0 Laboratory

The OPC UA communication protocol just described, is also applied and used in the laboratory environment. Hence, the connection between the resource layer and engineering tools such as *MATLAB* and *Simulink* is granted. The aim of this asset integration is the possibility to read in real-time the values of the sensors that are embedded in the modular infrastructure of the production line. It is to be noticed that each module is equipped with a PLC, that is cabled directly to the line. Also, each workstation has two different kinds of PLC: one referred to the *operational* control; the other that enables the *energetic* monitoring.

All of the PLCs in *I4.0 Lab* use the OPC UA protocol. Thus, it is possible for the researchers and the laboratory users to access all of the sensors in the line in real-time.

The way to access the values of the variables relative to the facilities is by using the OPC Toolbox in *MATLAB*. OPC Toolbox¹⁰ provides access to live and historical OPC data directly from *MATLAB* and *Simulink*, based on the fact that they should conform to the OPC UA Standard. This extension allows the reading, writing and logging of OPC UA data from devices, such as distributed control systems, supervisory control and data acquisition systems and PLCs (that are exactly the ones in the laboratory).

All of the data can be accessed only by creating a connection with the server IP address of the specific resource. All of the IP addressed of the resources in the line are reported in Table 6.3.

Workstation	IP Address
Manual	10.48.134.10
Magazine Front	10.48.134.20
Drilling	10.48.134.30
Robot	10.48.134.50
Camera Inspection	10.48.134.60
Magazine Back	10.48.134.70
Press	10.48.134.80

Table 6.3. IP addresses of the *I4.0 Lab* workstations

So, by creating a connection with the specific resource, using its specific IP address, all of the sensors value can be found. Accord-

¹⁰ <https://it.mathworks.com/products/opc.html>

ingly, a set of operations can be performed on these values, achieving optimization of operations, control of the line and similar activities.

6.5.1.1 UA Expert

All of these data must be stored and accessed somehow inside the *I4.0 Lab* environment. Hence, the need of a software that could be used as an access tool for the variables values, by exploiting OPC UA communication protocol, was needed.

The choice fell on **UA Expert** (Unified Automation Expert)¹¹. This software is a full-featured OPC UA Client, installed on both the computers in the laboratory. Its general purpose is to support OPC UA features like *Data Access*, *Alarms & Conditions*, *Historical Access* by granting access to the values of the sensors embedded in the line by using the OPC UA communication protocol. The use of UA Expert can be separated in two distinct interfaces:

- the *common framework* that includes general functionality like certificate handling, discovering UA servers, connecting to UA Servers *etc.* As a matter of fact in this general window, to access a specific resource and all of its sensors, the *server* needs to be reached, in order to build a OPC UA connection platform.
- the *Data Access* panel. Once established the connection via the server IP address, this kind of plugin is generated, like in Figure 6.15. Here, the possibility to (multi-)select UA Nodes in the Address Space window is given. Into the *Address Space*, in the left-hand side detail of Figure 6.15, a hierarchical classification of all the nodes the relative sensors is given. Thus, by selecting subsequently different levels of this list, the final node can be reached. Then, by dragging and dropping the nodes in the *Data Access View*, depicted in the top side of Figure 6.15, a subscription and real-time monitoring of them is given. The sampling rate and the subscription interval can be changed by right-click into the Data Access View. An important possibility is given in this interface since, for each node, under the *Value* column, the value of that sensor in real-time is given. Here, by modifying manually the value, it is possible to change it by *overwriting* a new one in order to give external control to that sensor value. The value that is being imposed externally must be compliant to the data-type of the specific variable. The data types that are supported are *scalar*, *array*, *matrix*, *boolean* *etc.*

In general, the use of this software allows the access to the sensors values and their real-time reading and, if needed, overwriting. Basically, every sensor has a specific *address* and *path* to be followed in

¹¹ <https://www.unified-automation.com/products/development-tools/uaexpert.html>

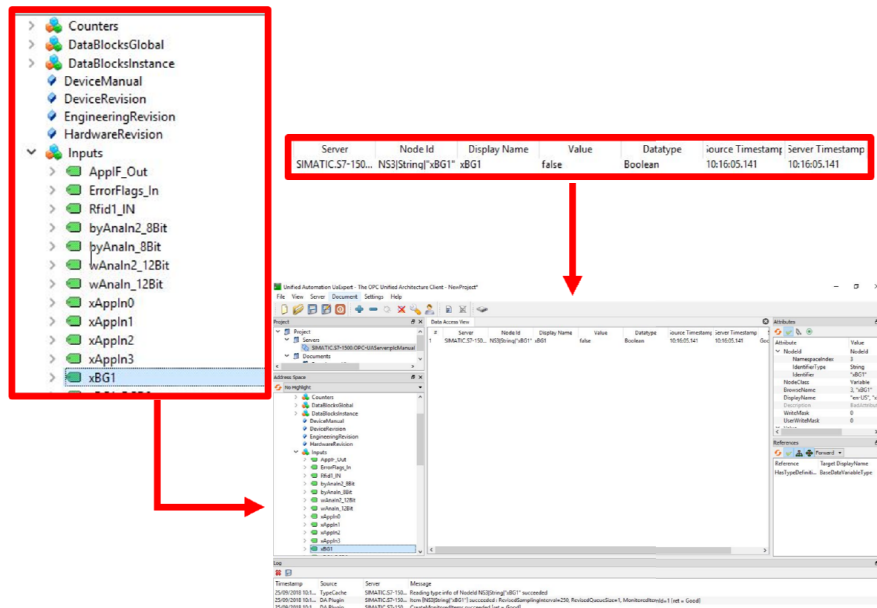


Figure 6.15. UAExpert software interface with detail on Data Access View (top side) and Address Space (left-hand side)

order to access it externally. So, in order to have control on these values, the connection and integration via a bigger and more powerful engineering tool is planned, like the *MATLAB & Simulink* environment.

6.5.2 OPC UA in Matlab environment

In the *I4.0 Lab*, for this kind of integration, the aforementioned software, *UA Expert* is used. Secondly, in order to access and process the sensors values on *MATLAB* using the OPC UA communication protocol, a set of steps needs to be followed:

- create a *connection* to the server of the station of interest;
- create the *client* and connection of the just created client to the server;
- track the path of the specific sensor (*node*) within the resource;
- replicate the address by using interlocked *MATLAB* variables that reach the specific node;
- eventually *operate* on the outcoming value.

In Table 6.4 the command steps to reach the `iErrNo` variable, is explained. Also, in Figure 6.15, a visual representation of the graphical interface used in *UA Expert* to reach a sample variable.

Finally, the variable that is stored in the node `ddg` in the last row of Table 6.4 can be read by using the code: `readValue(ddg)`.

Command	Meaning
serverList = = opcuaserverinfo('10.48.134.20:4840')	Connect to the server via the right IP address
uaClient = opcua(serverList)	Creation of the client
connect(uaClient)	Connect the created client to the server
f = getNameSpace(uaClient) b = findNodeByName(f, 'plcMagazineFront','-once'); c = findNodeByName(b, 'DataBlocksGlobal','-once'); ddd = findNodeByName(c, 'iErrNo','-once'); dde = findNodeByName(ddd, 'HMI','-once'); ddf = findNodeByName(dde, 'ISTB','-once'); ddg = findNodeByName(ddf, 'iErrNo','-once');	Use the address found in UA Expert to create the sequence to reach the selected variable

Table 6.4. Steps to find a variable via OPC UA, on *MATLAB*

6.5.3 OPC UA in Simulink environment

Once having found a way to read real-time values from the laboratory servers, the next step is to try to read and extract the exact same values using *MATLAB functions* in *Simulink* environment. As a first attempt, the use of normal *MATLAB* functions to read values coming from OPC UA was tried. But this resulted in a big delay, that compromise the real-time feature of the *Simulink* simulation model. Hence, the use of *Level-2 MATLAB S-Function*¹² was taken into account.

The Level-2 MATLAB S-Function allows to use the MATLAB language to create some custom blocks that are endowed with *input* and *output* ports. This kind of accesses are capable to handle any kind of signal produced by a Simulink model, including matrix and frame signals of any data type.

The added value is that such a function has an internal block that references a certain MATLAB function inside a Simulink model. The MATLAB function itself comprises a batch of *callback methods* that the Simulink engine invokes when updating or simulating the model. These callback methods perform the actual work of initializing and computing the outputs of the block defined inside the S-Function.

So, basically, a function built "*block-wise*" allows the call of specific functions only when needed. For instance the part that must be executed only at the start of the run on Simulink is performed only once, at the beginning. Like this, the big delay issue that needed to be faced by using classic MATLAB functions is solved.

The *blocks* that build the structure of the Level-2 MATLAB S-Function are three (cf. Figure 6.16), and their operation is here described.

- **SETUP block.** In this part of the function, the number on *Inputs* and *Outputs* of the function are defined, together with their Data Type (different data types are listed in the Appendix B).

¹² <https://it.mathworks.com/help/simulink/sfg/writing-level-2-matlab-s-functions.html>

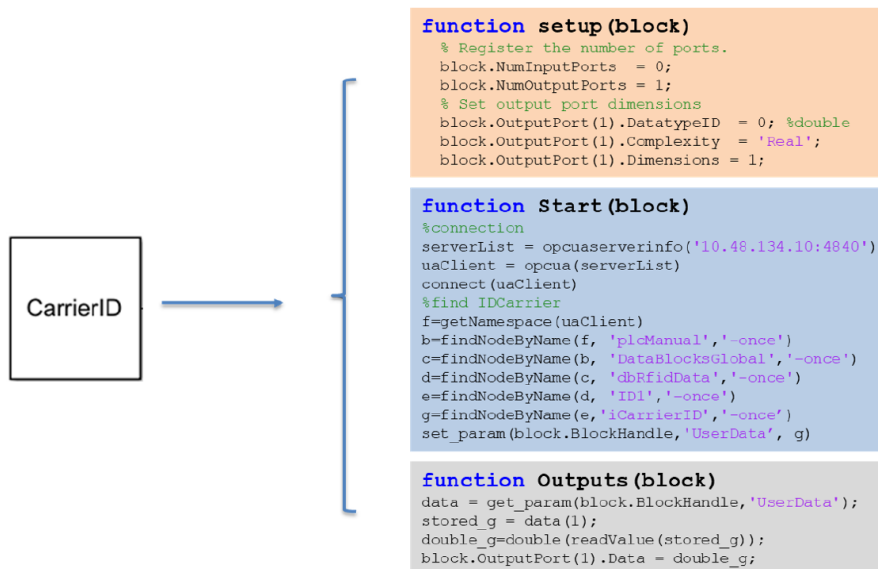


Figure 6.16. Level-2 MATLAB S-Function code to read the CarrierID value from the server

This block can be featured with only inputs, only outputs, or both, according to the operational needs of the model. Since the model is created as a discrete event simulation, this method also includes the specification about the sampling time required for the reading of the values of the sensors.

- **START block.** This is the most interesting part of the function for the simulation purposes of the work. As a matter of fact, this method allows to perform the connection to the server only when the start of the simulation on Simulink takes place, based on the code in Table 6.4. Then, the addition of the command "set_param(block.BlockHandle,'UserData', LAST NODE)" makes it possible to store the variable address in the Level-2 MATLAB S-Function block memory.
- **OUTPUT block.** This block calculates the S-Function outputs by getting them from the "BlockHandle" memory block. Then, with the "store" and "double", the variables are stored and transformed in a double DataType. Notice that in the example of Figure 6.16, double is the chosen DataType. Then, when the output is set, the real-time value of the selected variable is set with the code line "block.OutputPort(n).Data = double_g".

So, basically, the use of this function allows the real-time reading and subsequent use and manipulation of the values of the sensors embedded in the line. In fact, these variables are used to control and monitor the field level of the production line thanks to the relative building of a simulation real-time model on Simulink.

The *I4.0 Laboratory* is endowed with a software on one of its two control computers: the **MES**. Specifically, a version of it was provided by the production house of the line, *Festo Didactic*. This version is called **MES4**¹³ and it is featured with many functions that enable the *planning* field of the automation pyramid.

Substantially, MES4 is a specially prepared Manufacturing Execution System with a peculiar design for Industry 4.0 learning platforms. It is featured with a database that is open and can be written by external programs and read via Structured Query Language (SQL) commands by external programs. Furthermore, the individual controllers can communicate with the MES4 via *TCP/IP* communication protocol.

The training content that MES4 is provided with are various. Some of them are:

- Define and edit order workflows and process plans;
- Read orders and update status;
- Write allocation of the goods carriers to the order;
- Create warehouse data and material buffer;
- Overall Equipment Effectiveness (OEE), PLC and malfunction report generation, including graphic representation.

As just said, MES4 has been conceived to perform a big amount of activities. Though, only a few of them have been exploited to create the integration with Digital Twin in the work of thesis. The functions explained are the ones that can be substituted by making use of the command-based communication protocol on the Digital Twin. The specific exploited functions are going to be showcased in the next paragraphs.

6.6.1 Order creation & tracking

The MES4 software has been conceived to manage many activities, including:

- order creation;
- order tracking.

Both of these activities are performed by making use of the RFID technology, previously mentioned in the Chapter. In fact, thanks to

¹³ <https://www.festo-didactic.com/int-en/learning-systems/mps-the-modular-production-system/mes4.htm?fbid=aW5oLmVuLjU1Ny4xNy4xOC41ODUuNTM3NjA>

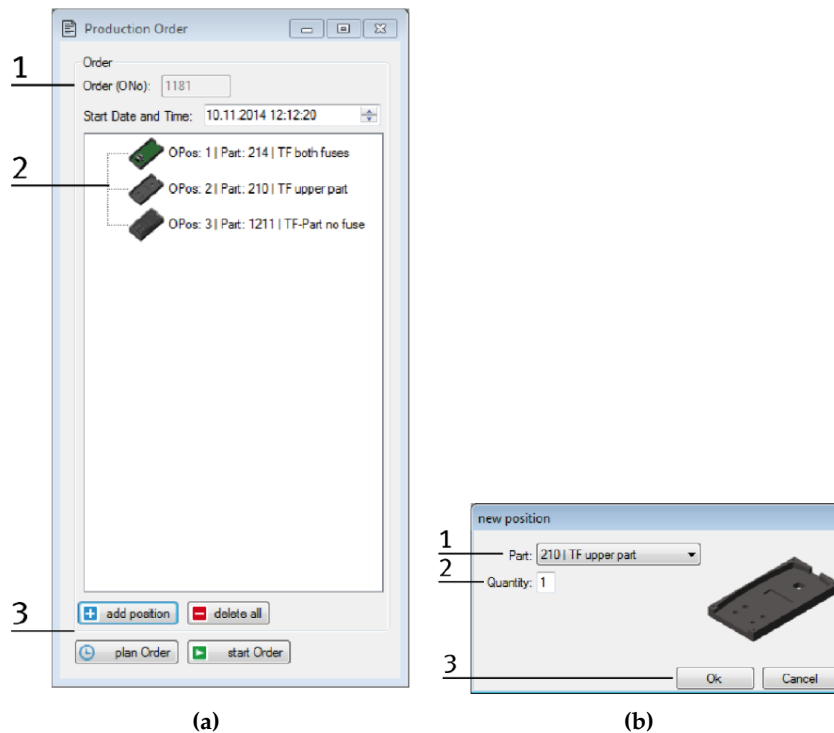


Figure 6.17. MES4 interfaces to create a production order and launch it in the production line

the storing of information in the specific carrier that is performing tasks along the line, the MES is able to manipulate the data and process information to have high monitoring and control yield.

To create and launch orders inside the line, the MES4 interface is the one depicted in Figure 6.17.

Specifically, the Figure 6.17a explains how the order can be created by the user. In position 1, the specification of the number of the order that is about to be created is given. In position 2, all the different order positions are displayed. Then, in part 3, the order can be launched by clicking on the "Start Order" button.

In Figure 6.17b, the interface to create new order positions is showed. This window comes from the "Add Position" button of Figure 6.17a. In this interface, part 1 shows the possibility to choose among workplans, part 2 defines how many units the user wants to produce. Everything can be confirmed as shown in part 3 of the figure.

Another important feature is the monitoring window, called "Actual Orders", that is shown in Figure 6.18.

The orders are shown in rows, in fact for each single order a row is created in the interface. For each row, many information about the state of the order are provided. Also, the row itself is coloured differently according to the production phase of the order. This colour paradigm is explained in Table 6.5.

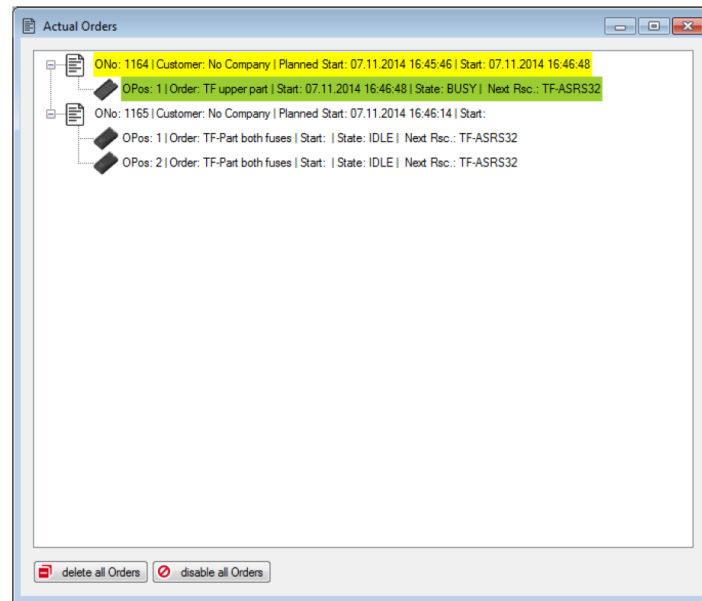


Figure 6.18. The window of MES₄ that shows the processing of orders along the line

Colour	Meaning
White	The processing of the order has not started
Blue	The order is finished
Yellow	The order started and it is in progress
Green	The order is in busy state
Red	The order has been annulled

Table 6.5. Meaning of the colour of a row in the *Actual Orders* window of MES₄

Also by clicking on the buttons positioned in the lower part of the window in Figure 6.18, the possibility to annul or disable the order is provided.

6.6.2 PCB Box handling

The possibility to control the items that are stored inside specific utilities is provided by MES₄ as well.

Since the *FESTO* line does not provide embedded sensors in the structure to monitor the presence of elements inside *Utilities*, the intervention of MES was needed. Inside *Industry 4.0 Lab* the only magazine specifically identified as an utility by the MES is the *PCB Box*. Inside this box, a set of PCBs is stored, ready to be assembled by the Robot station. In fact, the box itself is located inside the Robot Factorystation cell.

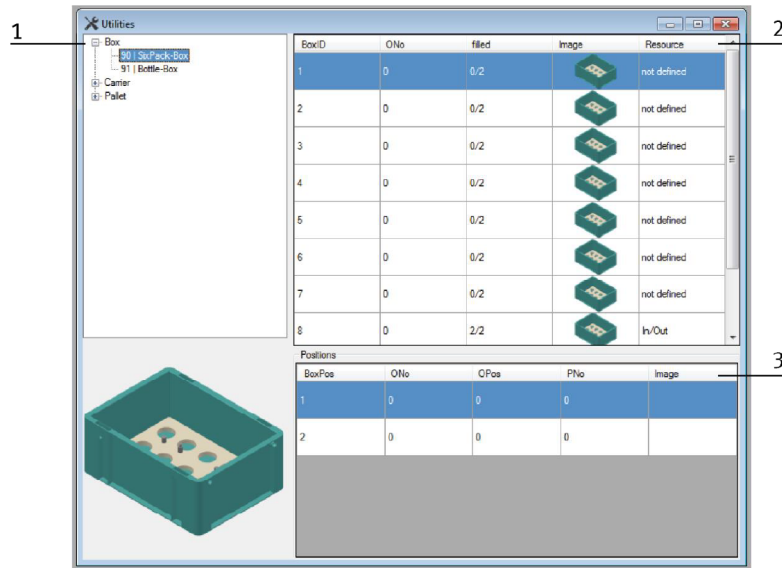


Figure 6.19. The *Utilities* interface of MES4

As it is shown in Figure 6.19, a proper interface for Utilities management is embedded in MES4. In part 1 of the figure, a list of all the utility types present in the facility is provided. Once the type is selected, the specific utility can be selected in part 2 of the figure. Lastly the management of all the elements storable inside the specific utility is enabled in part 3 of the figure. It is here that all the PCBs of the PCB box must be stored.

It is to be noticed that, if the PCBs are not stored inside the MES interface of the PCB box, the Robot does not proceed with the assembly. This is a crucial issue that has a solving proposal in Chapter 8.

6.6.3 PLC Simulator

With this interface, the simulation of communication between MES and PLC can be performed. In fact, the SQL data bank showcased in Appendix A is needed.

The request must be chosen, the green marked fields need to be filled (Figure 6.20), and the "Send" button must be clicked. Hence, on the bottom-side of the window, the specific PLC values coming from the database are returned, if the query is designed to demand them.

With this procedure, the exact working of all the strings in Appendix A is performed, but on this MES4 interface. As a matter of fact, before moving on to the implementation of the communication protocol on *Matlab & Simulink*, this *PLC Command Simulator* interface has been widely used to understand the data exchange that can be performed with the PLC database.

This is the interface of MES4 that is more linked to the communication with the Digital Twin that this thesis is proposing.

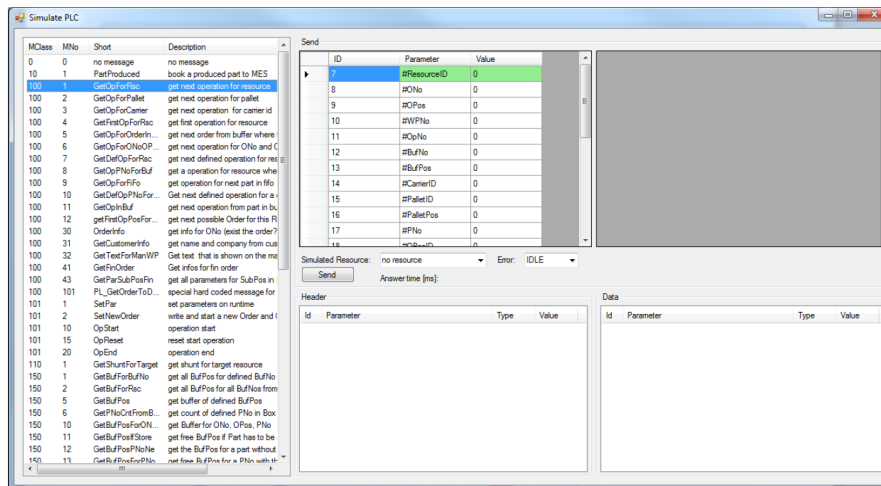


Figure 6.20. The MES4 interface that simulates information exchange with the PLC database

6.7 TCP/IP COMMUNICATION PROTOCOL

One of the means used in the work of thesis is the so-called **TCP/IP** internet protocol suite. The main objective of this technology in the proposed thesis, is the possibility to enable communication between external sources (*e.g.* engineering tools like *Matlab*) and the MES of the assembly line sited at the *I4.0 Lab*, the **MES4**.

This kind of suite is nothing but a conceptual model and set of communication protocols that are used in the Internet and in similar computer networks. The whole package is commonly known, by synechdoche, as the base foundational protocol of the suite, that is called **Transmission Control Protocol (TCP)** and the **Internet Protocol (IP)** ¹⁴.

The TCP/IP was first conceived and implemented by the research and development team of the Defence Advanced Research Projects Agency (DARPA) in the late 1960s in order to ensure and preserve data integrity, as well as maintain communications in the event of a catastrophic war [65]. By the summer of 1973, Cerf et al. worked on a redefinition of the protocol, in which the use of an inter-network protocol would hid the local network protocols [66]. The *Transmission Control Protocol* protocol was implemented and first published in 1974 [67].

The protocol proposed by Cerf et al. provides *end-to-end data communication* that specifies how the data should be packetized, addressed, transmitted, routed and received [67].

Overall the TCP/IP protocol architecture is based on several layers: Application, Transport, Internet and Link. The first mentioned are more *user-oriented*, whilst the latter can be considered as more

¹⁴ https://en.wikipedia.org/wiki/Internet_protocol_suite

transmission-oriented. As a matter of fact, the TCP/IP protocol as it is meant in the application field of the work of thesis must be intended as the **Internet Layer** protocol, the one that enables inter-networking via its routing function. Anyway, throughout the years, the TCP/IP communication protocol was proven to be the enabling technology that gave birth to the web infrastructure that later would've been called *the Internet*. This is to justify the magnitude of this protocol, that, as a matter of fact, is used worldwide in a huge amount of application fields.

In general the TCP/IP protocol is very easy to apply in many computerized systems. As a matter of fact it doesn't presume any kind of specific hardware or software environment. The only basic requirement is that hardware and software in analysis are capable to send and receive packets on a computer network. This remarks the easiness of implementation and use of the protocol.

6.7.1 *Fields of application*

Internet protocols are now, *de facto*, widely recognized as the main network standard. Many computers, nowadays are sold with all of the TCP/IP technologies in their standard delivery configuration. Also, the worldwide experiment of these protocols in all computer application fields guarantees the durability and efficiency of this network communication stack [68].

In most embedded systems used in robotics, industrial automation, machinery, and other industrial operations, communication can be either be done via *Ethernet* or via connection-based protocols like the TCP/IP one. This creates a client/server model for data retrieval through the sending of requests, and the reply of the client.

Grossly, the main advantage of the TCP/IP communication protocol is the possibility to transmit data in *real-time*. And this feature explains its extremely wide number of fields of application.

Focusing to the *manufacturing environment*, the TCP/IP protocol is widely used. One of the main applications is to use it to implement a Manufacturing Message Specification (MMS), that enables the programming of Application Programming Interface (API) for integrating non-MMS compatible manufacturing devices into the standard MMS communication Environment [69].

Some other applications focus on the development of inter-machine communication in transport line systems. So, an improvement of the working of the plant for de-manufacturing purposes is proven [65].

6.7.2 Communication with MES

In the *Industry 4.0 Laboratory*, the TCP/IP technology, within the work of thesis proposed, has been used mainly to guarantee a communication protocol from external software tools to the Manufacturing Execution System (MES).

Using this kind of communication, the possibility to communicate from an external software to the MES, and eventually to the Programmable Logic Controller (PLC) is granted.

Before proceeding with any other analysis, it is to be noticed that MES software packages are subject to standards, but basically every production house can choose to adopt or occasionally design an *ad hoc* kind of MES for its industrial applications. Hence the specific software tools and the related communication protocols will vary according to the operative environment.

In *Industry 4.0 Lab*, that is the context in which the work of thesis has been executed, the assembly line is an asset provided by the German company *FESTO*. However, the MES that is embedded in the production line is proprietary; it is, in fact, *MES₄*, previously introduced in the current Chapter.

Accordingly, all of the next assumptions about the MES software are to be referred to *MES₄*.

Inside the *MES₄*, communication can be of two distinct types (Figure 6.21):

- the **cyclic communication** that runs to detect the state information of the stations;
- the **acyclic communication** that is performed occasionally and arbitrary to elaborate the requests coming from the PLC.

Both forms of data transfer deliver data at speeds much faster than human can deal with. Clearly this is a major advantage, but there are some drawbacks that need to be taken into account.

Based on this distinction, a more *in-depth* analysis of these alternatives is going to be carried out.

6.7.2.1 Cyclic Communication

This kind of communication is based on TCP/IP protocol. The main objective of this kind of communication is to send *status information*.

In a cyclic connection paradigm, the data are delivered on a guaranteed or highly predictable schedule. As a matter of fact, information are constantly updated and transmitted respecting a predictable pace i.e. sampling time, production cycle *et similia*. Furthermore, in a cyclic network, usually the communication form of *master & slave*¹⁵ is used.

¹⁵ [https://en.wikipedia.org/wiki/Master/slave_\(technology\)](https://en.wikipedia.org/wiki/Master/slave_(technology))

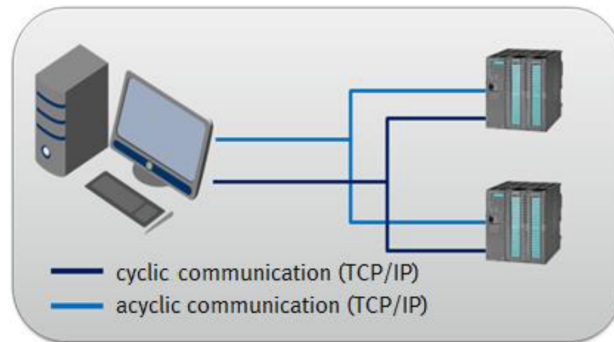


Figure 6.21. Alternative communication types between MES (left) and the PLCs (right)

This means that only one device is allowed to speak at a time, and answers are always returned, even if just to confirm that the message was received.

In the application field of *MES₄* software, the PLC sends several messages made of bytes of zeros cyclic. These messages must be of less than 5 seconds, otherwise the MES breaks the connection.

6.7.2.2 *Acyclic Communication*

On the other hand, this kind of communication philosophy is still reliable for machine and facility control. Information is just delivered in a less deterministic way *e.g.* on the order of milliseconds, up to several seconds.

When accessing a peculiar Variable-Frequency Drive (VFD) with a single PLC, this different behaviour with respect to cyclic communication will never be noticeable. On the contrary, sometimes there's the perception of a quicker and more responsive data transmission.

This kind of communication paradigm can be also reckoned to be anything that "*can wait*" communication-wise.

Acyclic networks might be accounted as a more chaotic way to communicate, but actually one of their main benefits is the ability to "*cheat*" the communication formalities. Therefore this can lead to an overall better speed and performance in transmission of data.

For the *MES₄* such a communication protocol has been implemented to control the assets remotely and at an arbitrary time. Each station or controller can:

- **get** information about the database;
- **write** values to the database.

An example of the workflow of information between the MES and a station PLC is pointed out in the next few lines. In this peculiar case, information about the production steps are asked by the station, in order to be updated about the next tasks. The sample steps are:

1. The station *asks* the MES whether there is an active order for this station.
2. The MES *sends back* the actual data (e.g. order number, process parameters). If no job is available, the data fields will be 0.
3. The station sends information about the *start* of the operation.
4. The station sends information about the *end* of the operation.
5. MES sends back the relevant data about the *next step*. This kind of information can be easily stored in the Radio-Frequency Identification (RFID) of the singular carrier.

This kind of approach allows to send this sort of requests to the MES that, in turn, will process it and return them with a response.

The requests can be carried out into two separate ways:

1. **byte** oriented;
2. **string** oriented.

Based on this distinction, on the *MES₄* platform a peculiar communication form can be adopted. This is the **string based** communication protocol that is shown to be an *acyclical* communication arrangement.

Being a very important enabler of the thesis project implementation, a dedicated section (6.8) is going to be presented.

6.8 STRING BASED PROTOCOL

This kind of protocol, as aforementioned, exploits the TCP/IP communication technology. The main objective of this means is to *connect to the MES software* and to *communicate with it in order to control it remotely*, without having to use it directly on the computer in which the MES is installed in the laboratory environment. Specifically, in the *I4.0 Lab*, this string-based protocol enables the communication with *MES₄*.

Basically, according to the distinction done in Chapter 1, the strings used in the work of thesis are a Structured Query Language (SQL), that relies on the TCP/IP protocol to communicate and exchange information with the MES.

The core nature of this communication protocol consists in its flexibility and applicability in several simulation and engineering tools. As a matter of fact, code can be written and launched in many software packages both regarding the assets of the line or external ones.

6.8.1 Definition of string

The main concept to be explained, talking about this kind of communication paradigm, is the **string** one.

In computer science, the traditional definition of string is a "sequence of characters, either as a literal constant or as some kind of variable"¹⁶.

A string can be generally considered a data type *de facto*. In fact it is often implemented as an array data structure of bytes or words, with the specific aim to store a sequence of elements. Those elements are usually *characters*, and to store them in the string a character encoding tool must be used.

Clearly, according to the encoding software used, a different way to create, encode and read a string is provided. Being *MATLAB* the selected engineering tool for the communication, a brief explanation of how this kind of data type is created and stored is provided.

The main difference among *data types* is the one between the **char** (*character*) and **str** (*string*) type. As it is shown in Table 6.6, there's a difference between the two data types.

Data Type	Syntax	Extension
char	a = 'hello' ;	1×5 char
str	b = "hello" ;	1×1 str

Table 6.6. Character and string extensions in MATLAB

- In the first row of the table, a **character** (char) data type syntax is shown. Here, the data is stored by writing the characters between apostrophe marks. The result is a *char* data type stored in the *Workspace*, with an extension equal to the number of characters typed between apostrophes.
- In the second row, a **string** (str) data type syntax is shown. Here, characters are written between quotation marks, giving as a result a 1×1 element in the *Workspace*. All of the characters, in fact, are clustered in a one dimensional feature.

Based on this difference, an annotation must be done. As a matter of fact, in the adopted communication protocol, even though the main term used to define the adopted data type is **string**, only **character** data types are used in the encoding software packages. This clarification is due to the fact that some people mean and expect a particular object when talking about *strings*, and misunderstandings might have happened. Hence, in this particular application, the word *string* must be meant as a general indicator of an *object that contains character elements, and not integers, double or other numerical items*.

¹⁶ [https://en.wikipedia.org/wiki/String_\(computer_science\)](https://en.wikipedia.org/wiki/String_(computer_science))

Also, by adopting the word *string* for both string and character element extension, no bugs will be faced. This is because the only data type adopted is the *character* one. Hence, the *string* term is just a "*perceptive*" adjustment that has been adopted by the *Festo* board when defining the MES communication protocol, and the author decided to comply with this rule.

This may force the overall terminology cohesiveness, especially in terms of computer science. But, by respecting the aforementioned definition of string, no misleadings should happen.

6.8.2 Communication with MES₄

Based on the assumptions done in the previous sections, the communication protocol is going to be showcased here.

The communication with the MES is done based on the action that the user wants to carry out. As a matter of fact the communication paradigm can be split in two:

- to **send** information to the MES;
- to **get** information from the MES.

These two procedures involve several actors and, even though they have some passages in common, they need to be explained thoroughly in the subsequent paragraphs.

To **send** information to the MES means that the user wants to operate on the line in order to have, eventually, the possibility to take a *planning* decision that could maybe affect the *field level*. To have a better explanation of this process, the reader is invited to check the *pyramid of automation* paradigm that follows the IEC 62264 normative (cf. Figure 1.2 in Chapter 1).

Based on this concept, the process called as *send* in this work is nothing that basic exploitation of the MES tool at *planning* level. This kind of decision, by descending the hierarchies of the automation pyramid, will eventually reach the *field level*.

Hence, by employing a communication string, the user is able to avoid the use of the physical MES resource. In fact they'll be sending information to the server by means of another engineering tool, used as a platform to communicate with the MES.

In this work of thesis, the aforementioned engineering tool is *MATLAB*, so all of these communication steps are being performed on this software.

The phases of the *send* process are the following:

1. *create* a TCP/IP object in order to connect to a remote host by means of a remote communication port. Here, the creation of a TCP/IP echo server is performed. Basically the MES is the host

in this framework, so information about its server IP address are going to be employed.

2. *connect* the TCP/IP just created object to the host.
3. *send* information to the host, in this case the MES. Information is going to be written and printed in string form.
4. the MES *receives* the string with the relative information that the user wrote.
5. the MES *communicates* with the downstream levels of the automation pyramid, in order to perform the requested action.
6. the PLC *receives* information and *sends* it to the field level, so as to carry out the field-level action, finally.

So, the sending procedure is easy, since the only task is to control the field level.

The process to **get** information from the MES is more complicated, since it contemplates the closing of the "loop" of the communication stream.

The concept of getting information from the MES is also different with respect to the sending process. Here, the user has a different decision-making drive. As a matter of fact they are not interested in sending information to the MES in order to control the field level. On the contrary, the user wants to send information to the MES only to get back information that would give them decision-making advantage.

So, for the *get* information stream, the steps are exactly as for the *send* option. It is to be specified that the information sent via string to the MES is, of course, not meant to take field-level measures. On the contrary, the string is written in order to *ask* about a specific information that the user is interested in. Then, once having performed all of the steps as in the send option, the following are:

1. the PLC, once having *received* information from the upper levels of the automation pyramid, *sends back* information to the MES.
2. the MES *receives* the information and *stores* it in the TCP/IP object. The information is always stored in a string form.
3. according to the engineering tool used, information will need to be *extracted* from the location in which it is archived.
4. Thus, information will need a *decoding* phase, using peculiar functions to parse the string.
5. Once decoded and transformed in an accessible data array, information are *available* to be used in MATLAB environment.

This whole communication framework was deduced from documentation about the assets of the assembly line, in addition with a study of the functions of the TCP/IP protocol.

Thus an additional coding work to make all of this possible had to be performed both on *MATLAB & Simulink*.

Both the "send" and "get" information frameworks together with the coding implementation have been described in Chapter 7.

Based on the aforementioned working steps of the communication strings, an important observation needs to be done. As a matter of fact, the communication is carried out between an external source and the MES. But, as it can be noticed in the previous procedures, the MES itself can be reckoned as a means that allows data transmission towards lower levels of the automation pyramid (cf. Figure 1.2 in Chapter 1). Hence, the communication is *de facto* happening between the external source (*Matlab & Simulink*) and the the PLC databases, since the first, by using TCP/IP protocol, is capable to mimic all of the functions of the MES. This is important in order to properly consider all of the communication protocols that can be used to grant the communication with the MES, as anticipated in paragraph 1.4.

6.8.3 Syntax of the string for MES communication

The composition of the string syntax for the *MES₄* communication protocol follows a standardized structure that respects specific rules to be followed.

Basically, a string, in order to be sent to the MES and to be understood without errors, should be made of the basic elements reported in Table 6.7.

Item	Symbol	Function
Parameter	(variable)	Identifier that gives specification
Equals	' = '	Separator
Value	(variable)	Value of the parameter
Semi-column	' ; '	Separator
Carriage return	<cr> or \r	End the string

Table 6.7. String basic syntax rules

Clearly, the basic elements explained in Table 6.7 must be meant as a basic pattern that can be used repeatedly for the whole length of the string. As a matter of fact, there is a big set of parameters and values that give further specification for the string objective.

The *body* of a string consists in two separate parts:

- the **header**, that is the standard part. In fact, each one of the possible strings has this portion, with different ID numbers en-

coded in it, that have the aim to identify the specific string. As the name suggests, the header section must be at the beginning of the string.

- the **data** section, different for each string. Here, according to the peculiar requests that the string might contemplate, the typing of specific parameters, with a specific ID is contemplated. By specifying this kind of information, the string is ready to be sent to the MES, whether if it is for sending orders to the PLC or gathering information from it.

The "header" and "data" sections of the string needs to be structured as explained in Table 6.8.

As in this table, the so-called header has the possibility to identify the string with an identifier nomenclature that gives specification about a determined string and its relative function. Specifically, the IDs 3 and 4 of the Table 6.8 determine the *class* and the *number* of the string. The library of the possible classes and number of the string is reported in Appendix A. Hence, the combination of the class and number of the strings, as in the *MES4* normative, will give the possibility to send to the MES a big number of different messages with different aims, both for the "send" and the "get" interface.

On the contrary, the **data** part of the string is less standard and very sensitive to the specific application that the user wants to address to the MES server.

The structure of this part of the string, follows the exact syntax explained in Table 6.7, but with different parameters and values. The communication protocol allows a set of data exchange via these strings, and to perform the tasks, a combination of parameters needs to be specified. All of these parameters are relative to the specific classes and number of the queries. Each class has a number of queries that are built to carry out a specific task. The aforementioned information are reported in Appendix A.

Furthermore, in Table 6.8, all of the possible additional parameters are given. Those are to be typed right after the header sequence, separating each parameter with a semi-column and anticipating it with a hashtag '#'.

Each parameter, as in Table 6.8, has its own specific function. These fields need to be filled up according to the needs of the peculiar string, both for sending to the MES and getting information from it.

Based on the information provided in Table 6.8, an example of the form of a generic string with a generic scope. For example, if the user is interested in checking if there is an order for the resource 61, the query to be sent to the MES is:

```
' 444;RequestID=0;MClass=100;MNo=1;ErrorState=0;#ResourceID=61\r'
```

(1)

Furthermore, it is to be noticed that these syntax rules are valid for both the strings that the user sends to the MES and the ones that are received by it. In particular, the string received by the MES has a structure altogether similar to the string in Equation 1, with the relative differences according to the application case. Thus, this received string needs a decoding process for the user to be able to exploit the information stored in the text.

This activity was one of the objectives of the MES integration in engineering and simulation tools, and was dealt with in Chapter 7, the dedicated one.

	ID	Parameter	Description
Header	1	Communication Number	It defines the protocol of the response 444 for the PC 445 for the Robot
	2	RequestID	ResourceID of the sending station
	3	MClass	Message Class
	4	MNo	Message Number
	5	ErrorState	Error coder, if 0, there's no error
Data	6	DataLength	Length of additional data
	7	ResourceID	ResourceID of the request
	8	ONo	Order number
	9	OPos	Order position
	10	WPNo	Work Plan number
	11	OpNo	Operation number
	12	BufNo	Buffer number
	13	BufPos	Position inside the buffer
	14	CarrierID	Carrier number
	15	PalletID	Pallet number
	16	PalletPos	Position on the pallet
	17	PNo	Part number
	18	OPosID	Unique number of the order position
	19	StepNo	Step number in the workplan
	20	MaxRecords	Maximum number of records to be shown
	21	BoxID	Box number
	22	BoxPos	Position inside the box
	23	MainOPos	Main order position
	24	BeltNo	Belt number
	25	CNo	Customer number
	26	BoxPNo	Part number of the box
	27	PalletPNo	Part number of the pallet
	28	Aux1Int	Free additional parameter
	29	Aux2Int	Free additional parameter
	30	Aux1DInt	Free additional parameter
	31	Aux2DInt	Free additional parameter
	32	MainPNo	Main part number

Table 6.8. Structure of the header and data sections of the string

MES INTEGRATION

In this chapter the model that has been conceived to integrate specific MES functions inside peculiar engineering tools is going to be explained. For this work of thesis, the choice of computing environments fell on *MATLAB & Simulink*.

Referencing Figure 7.1, at first (*Step 1*) a way to send commands to the MES and get information by it, has been performed manually on *MATLAB*. Secondly (*Step 2*), the focus switched on developing the same functions on *Simulink*, where the *Digital Shadow* was already implemented. Using this tool, a way to automatically communicate with the MES from the Digital twin was conceived.

All of the steps of the implementation procedure are going to be provided, including the obstacles that were faced and solved in order to create a communication interface between the MES and the just named external engineering tools.

The next paragraphs will explain the steps of the process to integrate the protocol in *MATLAB & Simulink* environment. The order of the next sections is meant to be temporary-based. In fact the steps have been implemented sequentially in time.

So, the first step has been the integration in *MATLAB*, as in section 7.1. Next, the second step consists in the embedding of the communication protocol in real-time simulation models on *Simulink* (section 7.2). Besides, two sub-sections are to be mentioned for both integration processes on *MATLAB & Simulink*: firstly the implementation of sending information paradigm (subsections 7.1.1 and 7.2.1); secondly the realization of a model to get information from the strings (subsections 7.1.2 and 7.2.2). A reference to the sequence of the development steps is represented in Figure 7.1

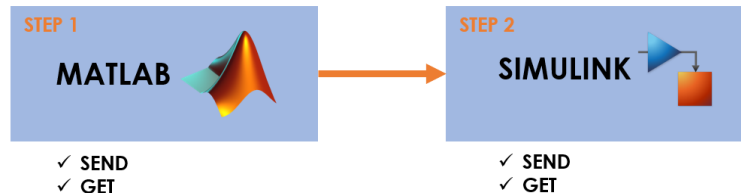


Figure 7.1. Steps of the development of integration work

7.1 INTEGRATION ON MATLAB

The integration of the communication strings in *MATLAB* environment has been performed as a mere first step of the whole work. In fact, the only added value of controlling the MES functions on *MATLAB* is the ability to run these tasks remotely.

All of the possible algorithms that an user can implement on *MATLAB* need to be run separately and every time the situation demands them. For the sake of completeness, a set of steps are provided in order to understand the use of a *MATLAB* algorithm to communicate with the MES. The manual nature of this process is also graphically synthesized in Figure 7.2.

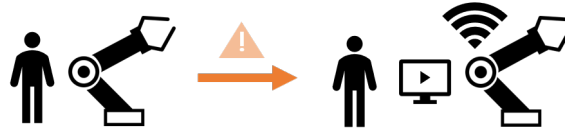


Figure 7.2. Manual sending of commands on Matlab from the user, when certain conditions are met

1. The user watches the assembly line to check whether it is more convenient to run the algorithm previously encoded to perform a certain function (left hand side of Figure 7.2).
2. When a certain condition is met, the user *runs manually* the *MATLAB* code with all the needed communication commands (right hand side of Figure 7.2).
3. Matlab connects with the MES server and the PLC, in turn, performs the task related to the specific machinery (right hand side of Figure 7.2).

This is far from the vision of an autonomous algorithm, but it gives the fundamental basis to understand the processing needs of communication protocols. As a matter of fact, provided the choice of the engineering tool, a deeper understanding of how to perform all of the steps to communicate with the MES grew (paragraph 6.8.2 for reference). This preliminary but necessary study, enables the following implementation of a stronger and more robust integration on *Simulink*, but this will be explained later.

Based on these premises, the work is going to be explained step by step.

7.1.1 Sending information

The procedure of sending information to the MES via *MATLAB* is one of the easiest and the most immediate to implement. In fact, the only steps to follow are:

1. create a TCP/IP object to connect to the MES server;
2. send the relative string to request the performing of a specific task to the MES.

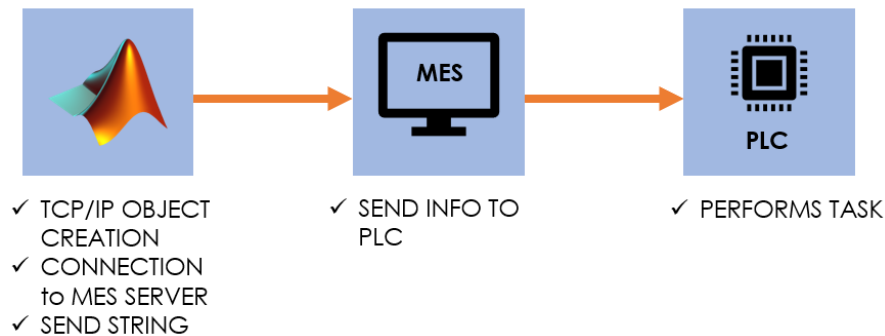


Figure 7.3. Operations performed by the "send" information model

As in the *MATLAB* library, the use of the `tcpip` embedded function was needed ¹. The sequence explained in Table 7.2 is fairly straightforward, and it is going to be applied also in other context within the integration work.

RemoteHost (IP address)	10.48.134.1
RemotePort	2000 or 2001

Table 7.1. Values of the IP address of the MES and the relative communication port

For the mere sending of information to the MES on *MATLAB*, the tasks performed are exactly the same as Table 7.2.

Based on Table 7.2, the steps are clear enough, except for some brief notations:

- The `RemoteHost` and `RemotePort` values are given in Table 7.1;
- in the dotted section of the code, the whole formulation of the string needs to be encoded, as explained in paragraph 6.8.3.

Based on these consideration, the use of strings can be done for whatsoever communication, using the standards on the library reported in Appendix A.

¹ <https://it.mathworks.com/help/instrument/tcpip.html>

Step	Code	Description
1	<code>t = tcpip('RemoteHost', RemotePort);</code>	Creation of TCP/IP object, connected to a remote host (IP address of the MES' server) with a remote port
2	<code>fopen(t);</code>	Connect the TCP/IP object to the host
3	<code>fprintf(t, sprintf('...'));</code>	Write in the TCP/IP object (t) the formulation of the string ('...') by creating a string variable (sprintf)

Table 7.2. Basic steps to send commands to the PLC via TCP/IP communication protocol in Matlab environment

```
t=tcpip('10.48.134.1',2000);
fopen(t);
fprintf(t,sprintf('444;RequestID=0;MClass=101;MNo=2;ErrorState=0;#PNo=1211;#AuxInt=1\r'));
```

Figure 7.4. Sample string use in Matlab to send an order to the MES

An example on Matlab code environment of a sample string to send an order to the MES is provided in Figure 7.4. A synthesis of the operations performed by the Matlab code in Figure 7.4 is provided in Figure 7.3.

7.1.2 Getting information

As explained in paragraph 6.8.2, the operation to get the information that the PLC sends back is more delicate, and needs more steps.

The basic flow to send information to the MES server, and so to the PLC, is exactly the same as the one reported in Table 7.2. The difference lies in the fact that the 3rd step should contemplate a string capable to "get" information.

A block-based sequence of the process to get information is given in Figure 7.5.

As anticipated in section 6.8.2, some of the queries listed in the tables of Appendix A have an additional feature. If the "send" type just transmits control data to the PLC, the "get" kind is able to interrogate the PLC about some specific information, storing it in a certain object in order for it to be retrieved by the user in a second moment. In this paragraph the explanation of the information retrieval procedure is showcased.

So, by applying the procedure in Table 7.2 with the a proper string, the TCP/IP object will be created and stored on the Matlab *Workspace*.

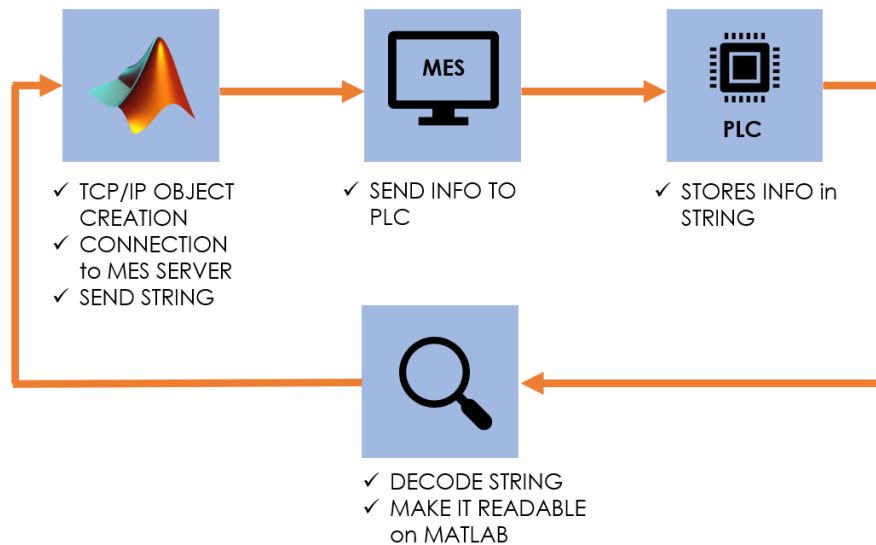


Figure 7.5. Operations performed by the "get" information model

This object is of the *scalar* class² category and contains a big set of information. As it is shown in Figure 7.6, the information set is big, and the most relevant data have been circled and explained in Table 7.3.

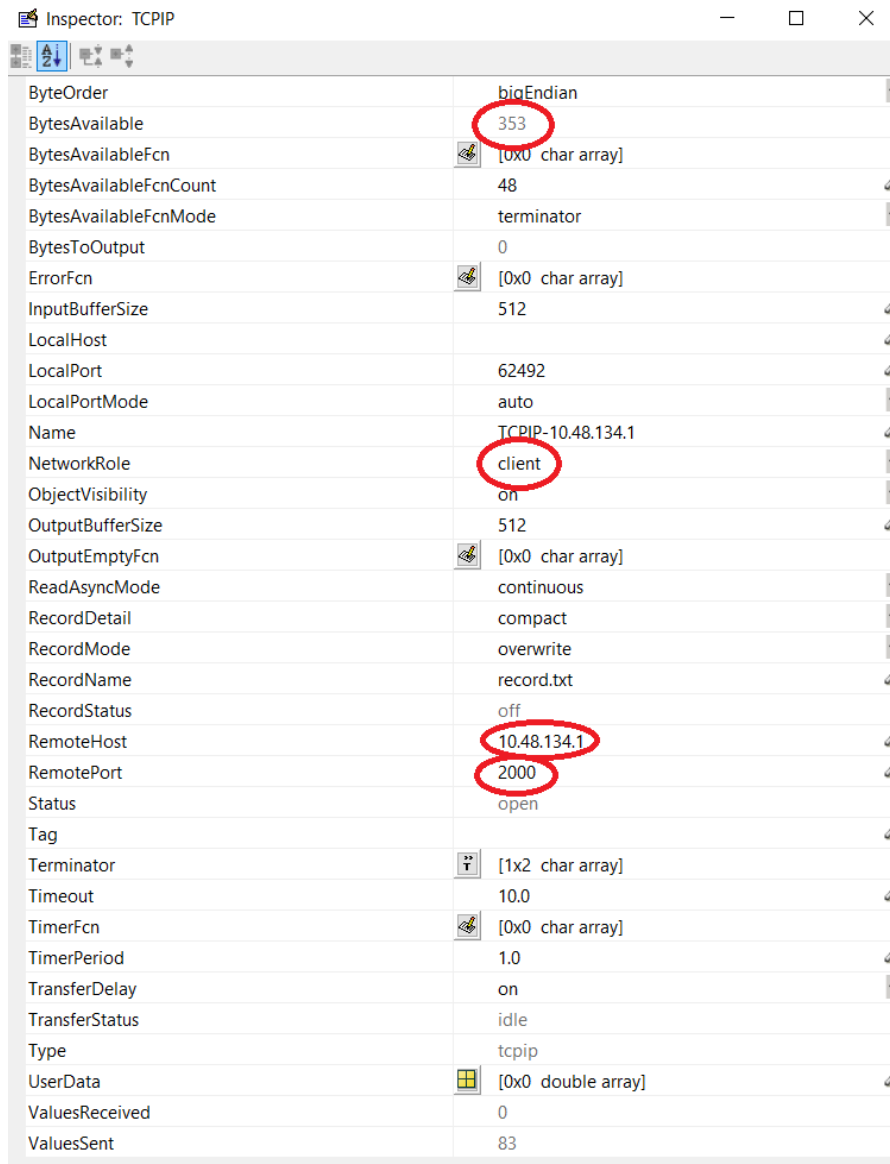
At this point, an observation is needed. The TCP/IP "t" object creation takes place both in the "send" and "get" frameworks, but for the first the object itself is of no use, since the concept of sending information to the PLC does not contemplate a procedure of data collection. Hence, the detailed description of the TCP/IP object has been neglected in the previous communication step.

FieldID	Description
BytesAvailable	The string is stored here
NetworkRole	It shows the role of the TCP/IP object in the established connection
RemoteHost	IP Address of the host of the connection
RemotePort	Connection port

Table 7.3. Data fields contained in the TCP/IP on Matlab Workspace

As it is shown in Table 7.3, the `BytesAvailable` field is stored into the Matlab structure. In fact, once the procedure in Table 7.2 is performed, for the strings which contemplate an interrogation of the PLC database, this particular field is going to be filled up with specific information.

² https://it.mathworks.com/help/matlab/matlab_prog/fundamental-matlab-classes.html



Property	Value
ByteOrder	bigEndian
BytesAvailable	353
BytesAvailableFcn	[0x0 char array]
BytesAvailableFcnCount	48
BytesAvailableFcnMode	terminator
BytesToOutput	0
ErrorFcn	[0x0 char array]
InputBufferSize	512
LocalHost	
LocalPort	62492
LocalPortMode	auto
Name	TCP/IP-10.48.134.1
NetworkRole	client
ObjectVisibility	on
OutputBufferSize	512
OutputEmptyFcn	[0x0 char array]
ReadAsyncMode	continuous
RecordDetail	compact
RecordMode	overwrite
RecordName	record.txt
RecordStatus	off
RemoteHost	10.48.134.1
RemotePort	2000
Status	open
Tag	
Terminator	[1x2 char array]
Timeout	10.0
TimerFcn	[0x0 char array]
TimerPeriod	1.0
TransferDelay	on
TransferStatus	idle
Type	tcpip
UserData	[0x0 double array]
ValuesReceived	0
ValuesSent	83

Figure 7.6. Data contained into a TCP/IP object in Matlab

This information is nothing but the data that the specific previously sent query requested to the PLC. So, the latter stored the requested information in this specific field of the TCP/IP object. Unfortunately information is not already readable, since the structure element shows only the number of bytes occupied by the string object inside the specific field of the TCP/IP object. An example of the information is given in Figure 7.7.

As it is shown in Figure 7.7, when calling the field of the TCP/IP object, no information can be extracted by this field. Anyway this will be proven to be the location in which information is stored. Therefore, the need of an extraction and decoding methodology grew.

```
>> t.BytesAvailable

ans =

353
```



Figure 7.7. Example of the calling of the *BytesAvailable* field inside the TCP/IP object "t"

Based on this, an **information extraction** procedure was conceived. This needed to be structured into two separate and consecutive steps:

1. a phase in which information is **read** from the structure field and **stored** into a Matlab object;
2. information is **converted** into a more practical Matlab object tailored for subsequent use of data.

These steps needed to be adapted and performed into Matlab environment. The designed procedure is briefly summed up in Table 7.4. Please notice that this information extraction procedure is meant to be applied only downstream to a query send to the PLC (procedure in Table 7.2). Besides, only after this, the TCP/IP object is created and can be manipulated with the steps in Table 7.4.

Step	Code	Description
1	<code>DataReceived = fscanf(t);</code>	Data is stored in the <code>DataReceived</code> $1 \times n$ 'char' object
2	<code>DataCellArray = = textscan(DataReceived, formatSpec,Delimiter,{ '...' });</code>	This function creates an array of cells (<code>DataCellArray</code>) where the <code>DataReceived</code> 'char' object has been parsed based on chosen delimiters

Table 7.4. Information extraction from a TCP/IP object previously stored in Matlab Workspace

Based on the information shown in Table 7.4, the full description of the information extraction steps is worth doing.

First of all the `fscanf(fileID)` function needs to be explained. In fact, the function is one of the Matlab embedded functions that is able to elaborate files. The basic description of the function can be a *reader of data from text files*. In this case, the file to be read is the TCP/IP object. This is why the `fileID` field in Table 7.4 is exactly the "t" object. The function itself identifies the data stored into the TCP/IP object, and reads and stores it into a $1 \times n$ 'char', where "n" is the size of the previously mentioned `BytesAvailable` field of the TCP/IP structure. Logically, each byte of `BytesAvailable`

corresponds to a single digit of the `DataReceived` 'char' variable, hence the size matching.

Data, according to the proposed algorithm, is formatted inside the `DataReceived` object that, if checked in Matlab Workspace, will be like in Figure 7.8. As it can be seen, if calling the just created object, a string comes out. As the reader may notice, the structure of the given string respects the previously studied *header/data* rules, as in Table 6.8 of Chapter 6.

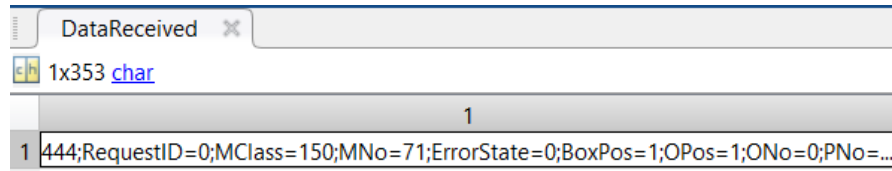


Figure 7.8. The `DataReceived` object in Matlab Workspace

It is to be considered that the information extraction process from the TCP/IP object using the `fscanf()` Matlab function takes about **5 to 10 seconds** to run completely, providing the value.

The second step includes the parsing operation *per se*. This phase is necessary for the user to be able to exploit information encoded in string form inside the query coming from the PLC database.

To extract information, at first an operation of simple call of the n^{th} string element was briefly considered, but quickly discarded. The call function was not suitable because information is not always placed in the same byte of the string *e.g.* a certain data is not always placed in the 15th element of a string that provides a certain information. This happens because the characters of the string coming from the PLC can vary, giving to the string different extension. Thus, positions of data change in the body of the string based on the different cases and runs. *Ipso facto*, a more robust decoding method was needed.

Thereby, the `textscan(fileID, formatSpec, Delimiter, {'...'})` function has been considered. As anticipated in Table 7.4, the function takes a formatted data stored either in text or string format, reads it and parses it following a pattern indicated by the user. The rules to be followed are hereby explained.

- `fileID` needs to be encoded with the string file that the user wants to parse. In this case, `DataReceived`.
- `formatSpec` is the field indicating the extension of the output objects of the function. Some examples of the encoding needed to specify peculiar extensions are given in Table 7.5
- `Delimiter` is the final field that expresses how the user wants to separate the output cell array of the `textscan` function. For every specification about a delimiter, the function breaks the string and creates another cell for the array. Anything can be

Code	Extension
'%f'	Floating
'%d'	Integer
'%s'	String
'%c'	Character

Table 7.5. Code to express specific extensions in Matlab function "textscan"

chosen as a delimiter, as soon as it is put in curly brackets, between apostrophes and separated by semi-columns. For the sake of clarity an example of application of `textscan` is given in Equation 2

```
textscan(fileID, '%f', Delimiter, {' ',';' '='}); (2)
```

In Equation 2, the `fileID` string is going to be parsed into separate cells containing float-point numbers. Furthermore, only commas and equals signs are considered as cells delimiters.

Once having explained all of the steps of Table 7.4, the user will have an array cell object called `DataCellArray`. Thus, by simply calling elements of the aforementioned array, the user will be able to know and exploit for whichever reason all of the information retrieved from the PLC string.

Based on all these considerations, a simple design process to extract information stored in TCP/IP objects coming from PLC queries, is provided. This kind of application works exclusively if code containing this function is run on Matlab, not on Simulink.

For the sake of completeness, a visual example of coding in Matlab environment is provided in Figure 7.9. Notice that in the figure, both strings and `Delimiter` field in `textscan` are samples, used for mere representation.

```
t=tcip('10.48.134.1',2000);
fopen(t);
fprintf(t,sprintf('444;RequestID=0;MClass=150;MNo=71;ErrorState=0;#MaxRecords=10\r'));
DataReceived = fscanf(t);
DataCellArray = textscan(DataReceived,'%f',Delimiter,{' ',';' '='});
```

Figure 7.9. Code to get information from a previously initialized TCP/IP query

7.1.3 The multiple string issue

During the use *Matlab* of for testing the functions of the various strings, a major problem was faced. In fact, if a certain amount of

strings was sent by using the same TCP/IP object and port connection that was created, the *MES*₄ crashed. This was symptomatic of the incapability of the MES server to handle more than **three** string transmission per time, by exploiting the same TCP/IP connection.

This represented a problem, called the "*multiple string*" issue. In fact, a user could be arbitrary interested in sending a big amount of data to the infrastructure, or to get a certain amount of information at the same time. Also, with this problem, the integration of a big information flow via TCP/IP strings in whatsoever real-time algorithm could not be performed.

This big issue was faced by using two separate Matlab functions:

- the `fclose(t)` function³. This permits to close the TCP/IP object "*t*" previously opened with the `fopen(t)` function.
- the `pause(n)` function⁴. This Matlab function stops the computational run of the script for *n* seconds.

At first, the solution was attempted by sending multiple strings within the same TCP/IP object, with a short `pause()` between the strings. This led to the send of only the first information string to the MES, so this solution was proven not to be effective.

The final solving technique lies in a combination of both the aforementioned functions. In fact the operation sequence must be this one:

1. follow all of the basic steps in Table 7.2;
2. close the TCP/IP object with `fclose()`;
3. pause the run for a short time by making use of `pause()`. The optimal pause time in order to have a sufficiently smooth run is **0.1 seconds**.
4. open another TCP/IP object following again the steps in Table 7.2, and then close again the object.
5. repeat this pattern for an arbitrary number of times.

An example of use of two strings in a single Matlab run is given in Figure 7.10.

Lastly, it is to be noticed that the multiple strings issue occurs also in *Simulink* environment. Thus, this way of coping with the problem can be used also in that environment, since algorithms are conceived and implemented in that setting as well.

³ <https://it.mathworks.com/help/matlab/ref/fclose.html>

⁴ <https://it.mathworks.com/help/matlab/ref/pause.html>

```

t1=tcip('10.48.134.1',2000);
fopen(t1);
fprintf(t1,sprintf('444;RequestID=0;MClass=101;MNo=2;ErrorState=0;PNo=1211;AuxInt=1\r'));
fclose(t1);

pause(0.1);

t2=tcip('10.48.134.1',2000);
fopen(t2);
fprintf(t2,sprintf('444;RequestID=0;MClass=101;MNo=2;ErrorState=0;PNo=1211;AuxInt=1\r'));
fclose(t2);

```

Figure 7.10. Sample use of multiple consecutive strings on Matlab

7.2 INTEGRATION ON SIMULINK

In this paragraph the process to integrate MES functions into a *real-time* simulation environment is going to be explained. The need to integrate these functions follows the primary thesis objectives: the integration of MES into a Digital Twin model.

The choice of the software fell on *Simulink*, the simulation tool embedded in *Matlab*. Hence, all of the considerations done by explaining the conceived process are meant for exclusive *Simulink* use.

This integration work must be seen as an evolution of the previous work of integration performed in *Matlab* environment. In fact the work of integration on *Matlab* was just preparatory for the highest level embedding work that has been performed on *Simulink*. In fact, integration on *Simulink* is nothing but the STEP 2 of Figure 7.1

Based on the design process later explained, the functioning of the Digital Twin is by far easier with respect to the running of the simple Matlab code. In fact the use of the integrated Digital Twin can be reduced to the next steps, that are visually synthesized in Figure 7.11:

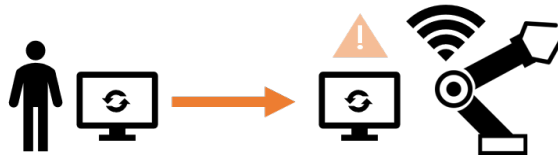


Figure 7.11. Automatic sending of commands on Simulink, using the Digital Twin model

1. the user runs the Digital Twin model on Simulink (left hand side of Figure 7.11);
2. the model itself solves autonomously the operational shop-floor issues that it is designed to tackle when certain situations occur (right hand side of Figure 7.11).

As for the *Matlab* case, an endeavor was made in order to integrate both "*send*" and "*get*" strings. The technical explanation follows in the next paragraphs.

Anyway there is a simple but big difference to point out between the working of both *Matlab* & *Simulink* with respect to the proposed assembly line framework.

- *Matlab* needs a code to be run so to activate the particular functions that are embedded in the code itself. Though, it is the user that decides autonomously whether to run or not the specific code for a specific function.
- *Simulink* is very different: once the simulation model starts, the user cannot stop or interfere with the running. So, this decision-making process must be modelled in the code structure.

For both *Matlab* & *Simulink* the implementation of semi-autonomous algorithms is possible, but — based on the nature of the Digital Twin *per se* in *I.40 Lab* environment — in this case the difference is substantial.

In fact, whereas for *Matlab* the code was simple and performed simple actions only by running it for few seconds fractions, for *Simulink* a bigger modelling work is needed, since the code of the *Simulink* functions must include all of the possible conditions in which the line can be at any given time. Based on these conditions, the code is conceived with the purpose to react to them and give optimization solutions to the shop-floor issue. These states can be called "*triggers*".

Hence the difference lies in this basic concept: for *Simulink* a set of "*trigger states*" of the assembly line must be detected and embedded in the code model. These triggers will allow the run of specific communication strings that interact with the PLC in order to give prompt problem solving methods. This is the only way for the integrated Digital Twin model to run and autonomously communicate with the PLC via TCP/IP objects.

Some examples of "*trigger states*" can be:

- error states of machines;
- breakdowns;
- the detection of a deviation from standards of production.

For a deeper understanding of the aforementioned procedure, a block-based representation of the designed *Simulink* model is provided in Figure 7.12. So, only when a trigger state occurs during the running of the model, the "*send*" and "*get*" models like in Figure 7.3 and Figure 7.5 are run.

Anyway some applications implemented in *I4.0 Lab* environment to validate this conceptual model are provided in Chapter 8.

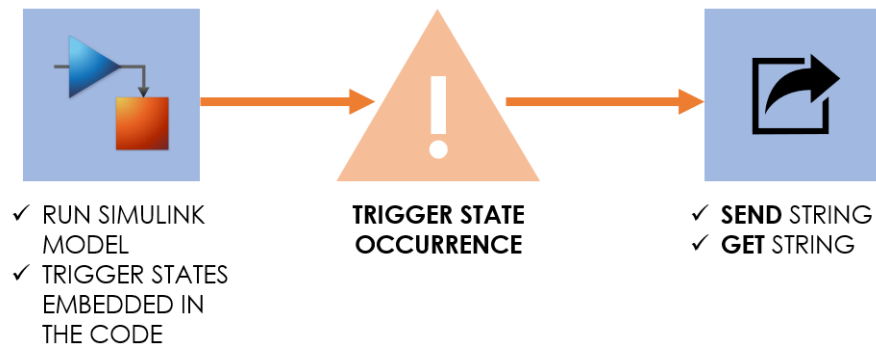


Figure 7.12. Operation of the Simulink model based on the definition of trigger states

7.2.1 Sending information

The procedure to send information to the MES, and so to the PLC via *Simulink* does not differ substantially from the *Matlab* case. As a matter of fact, it respects the rules explained in Table 7.2.

The procedure needs to be integrated into a real-time simulation model. Thus, to send this kind of information to the MES the use of one of the aforementioned triggering conditions is necessary. In fact, without a specific condition that is being detected by the Digital Twin reading the sensors, the information transmission could not be generated.

Besides, all of the coding inside Simulink environment has been performed by making use of customizable *Simulink Matlab functions* blocks, like the one in Figure 7.13. Like this the creation of real-time functions that can be properly coded is granted and used, since the studied block allows the creation of a specific Matlab function with inputs and outputs that can be used in the real-time model.

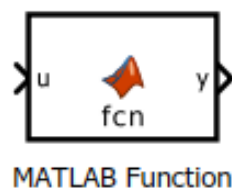


Figure 7.13. Example of a MATLAB function block inside Simulink environment

An example of a simple information sending to the MES is depicted in Figure 7.14. In the figure, only when the "if" condition is met, the string is sent to the MES.

```

if (xDisableMenu == 1 && xG1_BG50 == 0 && xG1_BG51 == 0)
    t=tcip('10.48.134.1',2000);
    fopen(t);
    fprintf(t,sprintf('444;RequestID=0;MClass=100;MNo=10;ErrorState=0\r'));

```

Figure 7.14. Code inside a Simulink Matlab function to send a communication string whether a particular condition is met

7.2.2 Getting information

As well as in pure Matlab environment, also for real-time Simulink models, the procedure to get data from the MES is more delicate.

Based on the aforementioned steps for Matlab environment, in order to get information from the MES, the user needs to send a string. The latter, sent following the steps reported in Table 7.2, will trigger the PLC to send back the requested information of the sent query. As in previous cases, information is stored in a TCP/IP object in a string form, that will need parsing and decoding in order for the data to be accessible to the model.

Anyway, data extraction is not so immediate as in pure Matlab environment, since Simulink models create some problems in code generation and parsing operations during simulation run. If such problems occur, in fact, simulation automatically terminates. So, the design of the model required some adjustments in order to properly work and parse the string received by the PLC.

The code used for running the real-time simulation on *Simulink* has the same basic structure of the one in Table 7.4, that was tailored for Matlab environment. The needed adaptations consist into two main areas:

- *initialization* of objects that are created with the decoding phase of the received string;
- *conversion* of data into objects that are parsable by *Simulink*.

As in previous cases, the information extraction is meant to be run only after the query send to the PLC, in order to have data to decode (Table 7.2 for reference). Therefore, once the creation of a TCP/IP object took place, the steps to be followed in order to properly parse and access information are explained in Table 7.6.

As it can be noticed, Table 7.4 and Table 7.6 differ among each other. The added steps are needed for the aforementioned adjustment process in *Simulink*.

Proceeding in order in the explanation of Table 7.6, the first and the third step are well known from the previous paragraph, and no further notations are needed.

The second and fourth step of Table 7.6, are strictly linked, and have been conceived to overcome a “call” issue that emerged during

Step	Code	Description
1	<code>DataReceived = fscanf(t);</code>	Data is stored in the <code>DataReceived</code> 1x1 'char' object
2	<code>DataMatrix = zeros(n);</code>	An empty array of n zeros is initialized
3	<code>DataCellArray = = textscan(DataReceived, formatSpec,Delimiter,{'. ...'})</code>	Creation of an array of cells where the string has been parsed based on chosen delimiters
4	<code>DataMatrix = = cell2mat(DataCellArray);</code>	The array of cells is converted into a matrix form

Table 7.6. Information extraction procedure from TCP/IP object, tailored for Simulink simulation run

the attempts. As a matter of fact, the array of cells that have been created in the third step of Table 7.6 are not perfectly suited for Simulink run.

It is known that the information extracted from the string coming from the PLC is nothing but a base for the creation of a decision making process. Hence, once that the code owns the knowledge of the string coming from the PLC, later it is going to use it to make other decisions like sensor overwriting or sending of other strings. The baseline is that the code must exploit and "call" the elements of the object in which the information is parsed and stored.

This is the point in which Simulink created issues. In fact the use of cell arrays in Matlab function blocks in *Simulink* environment was forbidden, at least in the format in which data was stored for the pure Matlab code use (like in Table 7.4). Here comes the solution that is provided and briefly explained in Table 7.6.

In the proposed solution, two additional steps are declared to overcome the aforementioned problem. In fact in step 2 of Table 7.6, the initialization of a zero vector with a length of n elements takes place. The length of the vector must be equal to the number of cells of the cell array element. This initialization is necessary in order to help in the problematic code generation in Simulink.

Then, in step 4 of Table 7.6, the conversion of the array cell into a matrix form is performed by making use of the `cell2mat` Matlab function. This step is also necessary because, as previously noted, the simulation algorithm must be able to extract and call elements of the array in which the storage of the string information takes place. Since in Simulink environment the cell array format is not suitable for this aim, a tailored matrix array has been created by converting the cells

array. Like this, the code can be designed properly and the elements of the array can be called in subsequent steps of the algorithm.

```
t=tcip('10.48.134.1',2000);
fopen(t);
fprintf(t,sprintf('444;RequestID=0;MClass=150;MNo=71;ErrorState=0\r'));
DataReceived = fscanf(t);
DataMatrix = zeros(136,1);
DataCellArray = textscan(DataReceived,'%f','Delimiter',{' ';' '='});
DataMatrix = cell2mat(DataCellArray);
```

Figure 7.15. Code inside a Simulink Matlab function to communicate with the PLC in order to get information

Also, a visual representation of the full Matlab code of the “get” designed framework is provided in Figure 7.15. It is to be noticed that in the figure both the string and the Delimiter field of the `textscan` are simplified.

7.2.3 Syntax adjustments

In order to have the Simulink real-time proposed model running smoothly, a couple of further technical measures are needed. The use of two peculiar Matlab functions have been carried out:

1. `persistent;`
2. `coder.extrinsic.`

Starting with the first point an analysis of the basic run of Simulink models is needed. Since the model runs all of the code simultaneously at every time stamp of the simulation, all of the code is involved, also the lines relative to the sending of strings to the MES and PLC.

An issue occurs whether there is the meeting of a triggering condition to which the sending of a string is linked. The problem arises when this condition remains on for a certain amount of time, and not for a single timestamp. If this happens, for all of the timestamps relative to this triggering condition, a string is sent to the MES. This results in a huge amount of string sending.

The problem has been solved by introducing a **persistent** dummy variable. This variable, at the beginning of the Simulink Matlab function block, is initialized to 0. Then, when the condition to send a given string is met, right after the string send, the dummy variable has its value modified to 1. This makes it possible for the portion of code relative to a certain function, in this case the sending of the strings, to be run only once during the simulation time. This happens because, for the nature of a persistent variable, the dummy unit is not re-set to zero at each run of the Simulink block. Hence, it is *persistent*.

An example of code of a persistent variable to trigger the send of a single string in a Simulink Matlab function block is provided in Figure 7.16.

```

persistent i
if isempty(i)
    i=0;
end

if (i==0)
    t=tcip('10.48.134.1',2000);
    fopen(t);
    fprintf(t,sprintf('444;RequestID=0;MClass=101;MNo=2;ErrorState=0\r'));
    i = 1;
end

```

Figure 7.16. Combined use of a persistent variable and strings in Simulink Matlab function block

The second problem is easier, and regards the declaration of the functions that are going to be used inside a given Simulink Matlab function block.

In order for the model to run properly, all of the functions must be initialized with a **coder extrinsic** specification. Thence, the model will work.

An example of the call of extrinsic Matlab commands inside a Simulink Matlab function block is provided in the codes of Appendix C.

This is the work of integration of a specific MES platform inside *Matlab & Simulink* engineering tools. By making use of this tailored coding model, an effective integration of MES functions inside a Digital Twin simulation model have been performed. Accordingly, the Digital Twin is endowed with a *reactive* nature to cope with certain conditions, enabling a simple autonomous shop-floor management framework.

Some relevant applications in *I4.0 Lab* environment have been carried out. These implementations, that validate the proposed approach, have been described in Chapter 8.

APPLICATIONS

This chapter aims at showing the application of the method of integration between the formerly developed Digital Shadow and the MES software of *Industry 4.0 Lab* at Politecnico di Milano (cf. Chapter 7). This applies in the field the **optimization frameworks** shown in Chapter 5, generating a full-bodied *Digital Twin*, able to create a *bi-lateral* communication between the physical asset and its digital copy. Hence, the control loop with the MES is closed.

The running of the proposed frameworks and integration method is shown in some applicative cases, that validate both the method and the frameworks. All of the applications yet to be proposed are based on the literature gaps explained in Chapter 4, putting forward the fulfilment of **sub-objectives 1 and 2**.

The first application proposal is explained in paragraph 8.1, and aims at applying the *error states optimization framework* showcased in paragraph 5.1 by classifying and finding a way to optimize a set of error states found for the assembly line.

Secondly, in paragraph 8.2, a Digital Twin model is tailored for the reactive scheduling of disassembly processes, applying the *disassembly framework* already explained in paragraph 5.2.

All of the proposed applications are performed by modelling *ad hoc* Digital Twins, integrated to the MES, in order for them to carry out the tasks by simply running.

8.1 ERROR STATES MANAGEMENT

This application of the integrated Digital Twin model consists in the proposal of solving a set of error states identified for the assembly line. The objective is *optimization* by reduction of the downtime generated by the aforementioned states.

The work progress, indicated in Figure 8.1, started with the identification and classification of error, mainly based on their nature. Then the calculation of their impact on the line in terms of downtime is performed. (cf. paragraph 8.1.1). Then, solution is proposed with the creation of integrated Digital Twins for every machine composing the line (cf. paragraph 8.1.2).

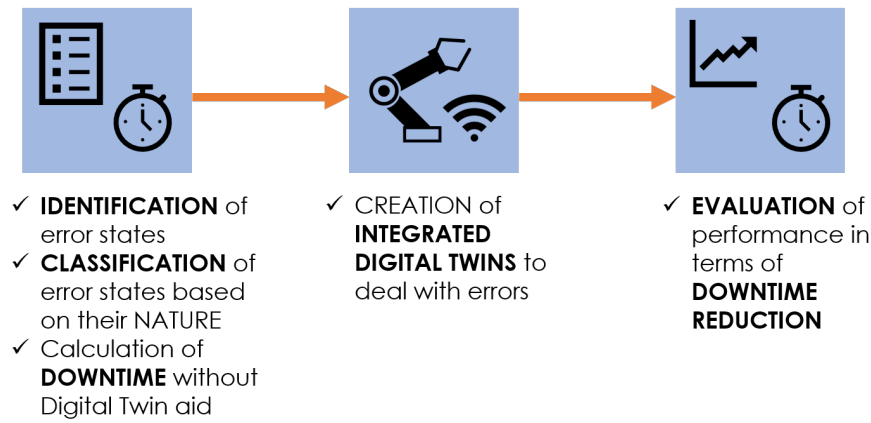


Figure 8.1. Working steps for the application of integrated Digital twin for error states management

8.1.1 Error states list and framework application

In this paragraph, the study to identify and classify a set of shop-floor level error states concerning the several modules that compose the assembly line of the laboratory is performed. Based on the definition given in Chapter 7, the error states are nothing but some trigger states that are identified and embedded, together with a solution method, in the code. Hence, this paragraph deals with the adaptation of the **error state management framework** explained in Chapter 5 in I4.0 Lab environment.

The work started with the determination of a set of error states that might affect the working of the assembly line. Then a timing of the mean downtime caused by this kind of errors was performed.

This specific task has been carried out by running the real-time simulation model that was already validated in the laboratory. The identification of a specific combination of sensors that enabled the identification of the error states is performed. By integrating the sensors value detection to the simulation tool, several runs of the tool itself have been done while manually triggering the error states on the line.

The error states, without the implementation of a Digital Twin integrated to the MES that is able to autonomously control the line, need to be manually solved by the operator in charge of monitoring the line. Accordingly, the work of classification of error states is aimed at finding an average downtime caused by the error states themselves, while they are being manually solved.

About this peculiar step, according to the *Central Limit Theorem* [70], a sample size of **30 observations** was considered sufficient to estimate the mean downtime for each error state, before and after the optimization measures provided by the Digital Twins.

The use of the integrated Digital Twin proposed by this work of the-
sis proves that an autonomous decision-making control and solving
of the error states of the line is possible, with a subsequent reduction
of average downtime, and enhancement of performance.

In Table 8.1, a list of the **11 error states** found in the assembly line
of the laboratory, is given.

Error ID	Machine	Description	Type
1	<i>Magazine Front</i>	Magazine empty	A
2	<i>Magazine Front</i>	Front cover already available	B
3	<i>Magazine Front</i>	Pallet not available	B
4	<i>Drilling</i>	Front cover not available	B
5	<i>Robot</i>	PCB not available	A
6	<i>Robot</i>	Pallet not available	B
7	<i>Robot</i>	Front cover not available	B
8	<i>Magazine Back</i>	Front cover not available	B
9	<i>Magazine Back</i>	Magazine empty	A
10	<i>Magazine Back</i>	Back cover already available	B
11	<i>Press</i>	Back cover not available	B

Table 8.1. Error states of the assembly line

The errors shown in Table 8.1 have been found with the aforemen-
tioned procedure based on sensors. The specific analysis of the pecu-
liar sensors to identify the error states is provided in the next para-
graph 8.1.2; there the implementation of the Digital Twins for each
module of the line is thoroughly explained.

Based on Table 8.1, some remarks are worth doing:

- at least *one* error state has been identified for each machine com-
posing the line, except for the *Camera Inspection* module, that
was proven not to create error states that generate a production
stop with subsequent downtime.
- the column named "*Type*" refers to the basic classification that
has been performed on the error states and that was already an-
ticipated in Chapter 5. This classification concerns the necessity
of **human intervention** to solve them.
 - The errors of **type A** are the error states that *need the in-
tervention of a human*, regardless of the way the problem is
solved. This kind of error can be optimized with the use of
a simulation tool, but the presence of a human being is nec-
essary. The reader may notice that this kind of error refers
to the predicament in which there is the lack of production
pieces in magazines.

For the specific case of *Industry 4.0 Lab*, this kind of error
occurs when the PCB box is empty, or the magazines rela-

tive to the front and back covers of the prototypical phone. It is trivial that a Digital Twin cannot solve this problem entirely, since the various magazines need to be filled by an operator. Anyway, an optimization margin still stands, and it will be proven in next paragraphs.

- the errors of **type B**, on the contrary, can be fully solved by making use of a controlling Digital Twin, since *no human intervention is needed* given the fact that no physical resource needs to be managed in these states. In fact, for this kind of errors, human intervention is needed only when there is no integrated Digital Twin running.

For the specific case of the laboratory, an error of this type occurs when, for instance, a piece necessary for the assembly is already assembled on the workpiece, when the carrier arrives at the workstation. This kind of error, when the Digital Twin proposed by the work of thesis is not running, is usually manually tackled by the operator in charge of monitoring the line by pressing a button of the HMI.

It is to be noticed that, based on the components of the modules of the assembly line, many other error states could have been identified. Unfortunately not all the embedded sensors in the line are programmed to be reachable using OPC UA protocol, so the errors relative to those peculiar sensors could not be considered. Nevertheless, all the other possible error states would have fallen in the "A" and "B" typologies. For instance, some error states could have been the lack of fuses in the fuse magazine, or the missing of one of the specific assembly tools of the robotic arm. Such errors would have been of type A, since they both represent the missing of a physical component strictly necessary to proceed with the production, so human intervention should have been necessary.

Based on the aforementioned classification a the solving technique framework can be applied in the laboratory context. For visual reference of the flow of activities, please refer to in Figure 5.1 in Chapter 5.

The chart is divided between *physical* and *simulation* level: the former explains what happens on the shop-floor level, the latter shows the actions performed by the Digital Twin. It is worth remarking that the actions performed by the Digital Twin on simulation level, are carried out by communicating with MES.

An explanation of the peculiar adaptation of the framework in Figure 5.1 follows:

1. When an order arrives to one of the laboratory workstation, it can be either processed without problems or there might be issues in carrying out the task.

2. In case the order cannot be processed, the **error state** is triggered. The solution of the error state depends on the error type (cf. Table 8.1 for reference to the error types in I4.0 Lab):
 - with **type A error**, the simulation model helps the interaction between the human and the machine. Then the human needs to solve manually the issues relative to the error. Lastly, once the human intervention is performed, the flow goes back to the simulation side, where the final solution of the error is done.
 - with a **type B error**, the simulation model solves the issue right away.
3. Once the error is solved, the algorithm tries to process the order once again. Since the error is solved, the order is processed with no problems.

8.1.1.1 Error states mean downtime without optimization

Based on what anticipated about the method used to gather data, in Table 8.2 the mean downtime caused by each error state is given.

Error ID	Type	Mean Downtime [s]	Impact
1	A	16.28	Medium
2	B	12.86	Low
3	B	13.23	Low
4	B	13.86	Low
5	A	66.75	High
6	B	14.89	Low
7	B	11.84	Low
8	B	14.39	Low
9	A	19.74	Medium
10	B	13.05	Low
11	B	13.81	Low

Table 8.2. Error states mean downtime without optimization

Given the values in Table 8.2, it is worth noticing that the classification mentioned in Chapter 5 of error states based on their impact on downtime has been performed.

- **Low impact** ($Downtime < 15s$): this category includes all the errors of type B, since they are the easiest to solve *e.g.* release the order by simply pressing a button on the HMI.
- **Medium impact** ($15s \leq Downtime \leq 20s$): this category concerns errors of type A that exclusively need the intervention of the operator to perform a single operation *e.g.* filling of the

magazine with new assembly parts. Hence, the downtime is not that lengthy.

- **High impact** ($Downtime > 20\text{ s}$): here fall the errors of type A that need the intervention of the operator to perform multiple operations. This leads to a considerably long downtime.

8.1.2 Simulation Models

In this paragraph, the description of simulation models (Digital Twin with MES integration), as already structured in the framework explanation of Chapter 5, is performed. Specifications about the sensors used to identify the error states are provided, together with the description of the strings used to intervene by using MES functionalities.

The integration of the MES functionalities activated by the string transmission, has been synthesized in the addition of a single Simulink Matlab function block. The function, as already anticipated in the framework proposal of Chapter 5, has been called "**Solver**" in the simulation models, since it actually solves the error states detectable in the assembly line by communicating with the MES.

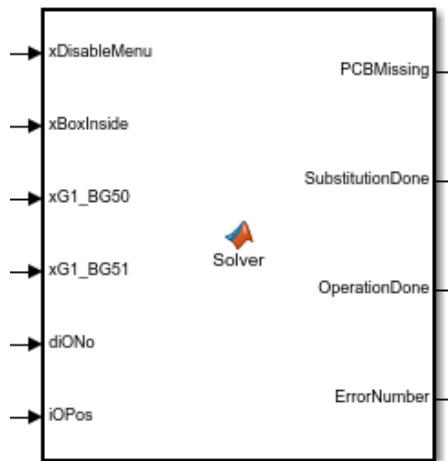


Figure 8.2. Example of "*Solver*" Simulink Matlab function block

The example of a "*Solver*" block that can be found in the Simulink models of the Digital Twin is in Figure 8.2. The block is linked both in input/output to the "**Detector**" function, that, in the application case in the lab, is a Level-2 Matlab S-Function:

- the *inputs* to the "*Solver*" are sensors that are read in real-time by the Level-2 Matlab S-Function using OPC UA. Hence, these values are outputs of the Level-2 Matlab S-Function.
- the *outputs* to the "*Solver*" are variables that indicate the completion of the operations performed by string transmission inside

the function itself. These variables are needed to trigger the overwriting of sensors of the line, so these values become inputs to the Level-2 Matlab S-Function.

The specific applications of the “*Solver*” function block are explained for each workstation in the next paragraphs.

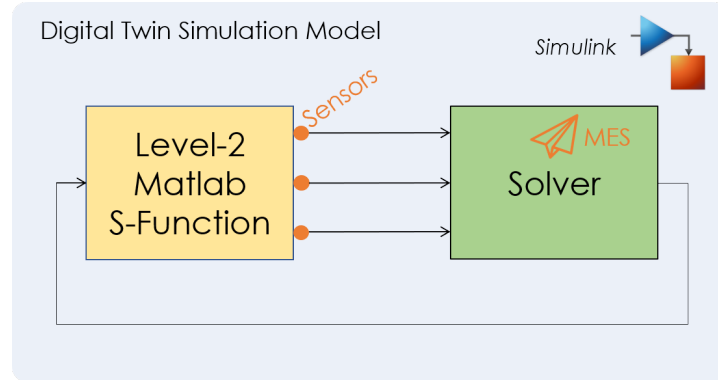


Figure 8.3. Structure of the Digital Twin on Simulink with the integration of the “*Solver*” function to connect to the MES

For the sake of clarity, a visual representation of the Digital Twin in Simulink environment, integrated with the MES functions thanks to the Solver block function is shown in Figure 8.3. The Solver block and the Level-2 Matlab S-Function are linked, and the outputs of the former become the inputs of the latter, just as anticipated.

Next, the reader will find the specific features of the Digital Twins that have been developed for each workstation of the assembly line that is affected by the occurrence of error states (cf. Table 8.1). Notice that the structure of the Digital Twins follows the main characteristics given by the model in Figure 8.3.

8.1.2.1 Magazine Front Station

This station, illustrated in Figure 8.4, assembles the prototypical cell-phone discharging the front cover directly on the pallet where the other assembly tasks will be performed in the text steps.

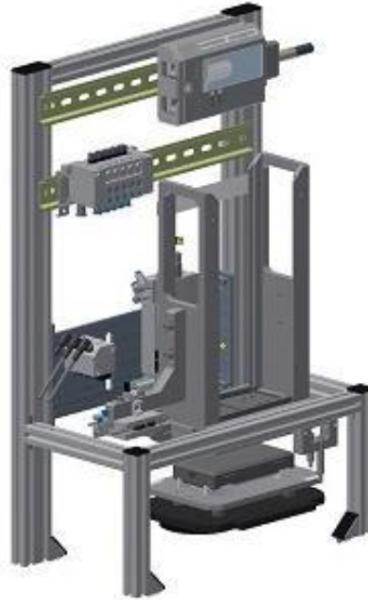


Figure 8.4. Magazine Front Workstation

The *input* sensors of the Magazine Front station "Solver" block are:

- *xCL.BG7*: it shows the presence of the pallet on the carrier.
- *xCL.BG8*: it shows the presence of the front cover on the pallet.
- *xCL.BG5*: it shows the presence of front covers in the magazine of the workstation.
- *iErrNo*: it is relative to the specific error type on the HMI display.
- *diONo*: it shows the *number* of the processed work order.
- *iOPos*: it shows the *position* of the processed work order.

The *outputs* of the "Solver" function for the Magazine Front are:

- *MagazineFilled*, used for the solving of Error ID 1.
- *HMIError*, is 1 if there is an error on the HMI.
- *OperationDone*, used for the solving of Error IDs 1 and 2.
- *ErrorID*, displays the current Error ID.

Error ID	xCL_BG7	xCL_BG8	xCL_BG5	iErrNo
1	1	0	0	8
2	1	1	1/0	2
3	0	0	1/0	1

Table 8.3. Sensors combination for error states at the Magazine Front

The possible error states of the Magazine Front are 3. They are identified thanks to the sensors combinations in Table 8.3 and the Matlab function `cop` with them like this:

- **Error ID 1** (type A): *magazine of front covers is empty*. When an order arrives at the working position, the production is stuck since there is necessity for the filling of the magazine. The simulation model waits for the operator to fill up the magazine. As soon as the magazine is filled, the *MagazineFilled* output changes value and, thanks to a feedback loop to the Level-2 Matlab S-Function, the *xRetry* sensor (that tries again the processing of the order) is automatically triggered. Since the magazine has been filled, the order is processed.
- **Error ID 2** (type B): *front cover already available*. When the order arrives the production is stuck, since the front cover is already on the pallet. The code, once this error state is met, sends to the MES a "OpEnd" string (**MClass = 101**, **MNo = 20**, cf. Appendix A) to the MES, that ends the current operation of the order identified by the *iOPos* and *diPNo* sensors, switching to its next operation on the workplan. As soon as this is done, the *OperationDone* changes its value, enabling, thanks to the feedback loop to the Level-2 Matlab S-Function, the *xReject* sensor, that clears the station.
- **Error ID 3** (type B): *no pallet on the carrier*. When the order arrives, the production stops since there is no pallet to discharge the front cover onto. The code sends an "EndOPos" string (**MClass = 101**, **MNo = 21**, cf. Appendix A), that aborts the order identified by the *iOPos* and *diPNo* input sensors. Once this is done, the *OperationDone* changes its value, and, just like in the previous error state, the station is cleared.

8.1.2.2 Drilling Station

This workstation performs a fake drilling of the front cover by making use of two drills. The task is performed by simulating the drilling of the holes two at a time. Possible operations contemplate the drilling of just two holes (left or right side of the case), or four holes (both sides of the cover).



Figure 8.5. Drilling Workstation

The *input* sensors of the Drilling station "*Solver*" function are:

- *xCL_BG4*, that detects the presence of the front cover on the pallet, when the pallet is in working position.
- *iErrNo*, relative to the specific error type on the HMI display.
- *diONo*: it shows the *number* of the processed work order.
- *iOPos*: it shows the *position* of the processed work order.

The *output* variables coming out of the "*Solver*" function are:

- *OperationDone*, that states when the operation of annulment of the order is performed.
- *ErrorID*, that displays the error state number, as in Table 8.1.

The body of the "*Solver*" function code deals with the error states of the Drilling workstation like this. For this workstation, there is a single possible error state. Accordingly, when the sensor-based conditions in Table 8.4 are met, the error state with **Error ID 4** (type A) is occurring, meaning that *no front cover is available on the pallet*. For this

workstation, the full "Solver" block script can be found in Appendix C.

The code sends the string "EndOPos" belonging to **MClass = 101** and **MNo = 21** to the MES in order to annul the order (cf. Appendix A). The string used is in Equation 3, and takes as inputs the *diONo* and *iOPos* sensors, to know which order needs to be annulled.

$$\begin{aligned} & \text{fprintf}(t, \text{sprintf}('444;RequestID=0;MClass=101;MNo=21; \\ & \text{ErrorState}=1; \#ONo=\%d; \#OPos=\%d\r', diONo, iOPos)); \end{aligned} \quad (3)$$

When this operation is performed, *OperationDone* value changes. The change of value of this variable becomes an input of the Level-2 Matlab S-Function, that triggers the overwriting of the *xReject*. This sensor value, once overwritten, allows the order to be rejected from the working position, clearing the workstation.

Error ID	xCL_BG4	iErrNo
4	0	4

Table 8.4. Sensors combination for error states at the Drilling

8.1.2.3 Robot Assembly Station

This station concerns the assembly of a PCB on the front cover of the smartphone. The assembly may include one or two fuses, according to the specific workplan. All of the activities are performed by the robotic arm, inside the dedicated robotic cell (cf. Figure 8.6), in which there are also two magazines, one for PCBs and one for the fuses.



Figure 8.6. Robot Assembly Workstation

The inputs to the "Solver" function of the Robot are:

- *xDisableMenu*: it indicates when an error is being displayed in the HMI.
- *xBoxInside*: it's 1 when the PCB-box is inside the robotic cell, at the position it is required for working purposes.
- *xG1_BG50*: it senses the pallet at the working position.
- *xG1_BG51*: it senses the front cover at the working position.
- *diONo*: it shows the *number* of the processed work order.
- *iOPos*: it shows the *position* of the processed work order.

The outputs to the "Solver" function of the Robot are:

- *PCBMissing*: it is the sensor that detects if there's lack of PCB necessary for the assembly process, in the PCB box. It is linked to the solving process of the Error ID 5.
- *SubstitutionDone*: it detects if the PCB substitution process in the has been performed by the operator. It is linked to the solving process of Error ID 5.

- *OperationDone*: used to solve Error IDs 6 and 7.
- *ErrorID*: it displays the ID of the error that is occurring at the workstation.

Error ID	xG1_BG50	xG1_BG51	xDisableMenu
5	0	0	1
6	1	1	1
7	0	1	1

Table 8.5. Sensors combination for error states at the Robot

Based on the sensors combination of Table 8.5, there are three detectable error states.

- **Error ID 5** (type A): *PCB not available*. When this error occurs, two kinds of operations need to be done by the operator: to fill manually the physical PCB box, and to fill the virtual PCB box on the MES. This operation is necessary since there are no sensors to detect the single PCB in the box, so to process the orders and keep track of resources, this method has been adopted. The Digital Twin, when this error occurs, performs these steps:
 1. When the error occurs, the *PCBMissing* variable changes value, and the Digital Twin interrogates the MES on whether there are empty slots in the virtual PCB box, thanks to the string "GetBoxPositions" (**MClass = 150, MNo = 71**).
 2. Automatically the Digital Twin ejects the PCB box from the robotic cell, in order to induce the operator to notice that PCBs are missing. At the same time the Digital Twin fills up the virtual PCB box slots on the MES by sending commands to it ("SetBoxPos" **MClass = 151, MNo = 71**).
 3. As soon as the operator fills up the physical PCB box, in the moment when the PCB box is put back in its working position, the *SubstitutionDone* value changes. So, the code automatically triggers the *xRetry* sensor, in order to continue the assembly work.
- **Error ID 6** (type B): *no pallet available*. When this error occurs, the Digital Twin sends the "EndOPos" string to the MES to annul the order (**MClass = 101, MNo = 21**). When this is done, the *OperationDone* variable changes value in order to trigger the *xExitCycle* sensor and release the order from the working position.
- **Error ID 7** (type B): *no front cover available*. The principle of working is similar to Error ID 6.

8.1.2.4 Magazine Back Station

This workstation assembles the smartphone by releasing the back cover on the previously dispensed front cover, whether the workplan includes PCBs or other components.

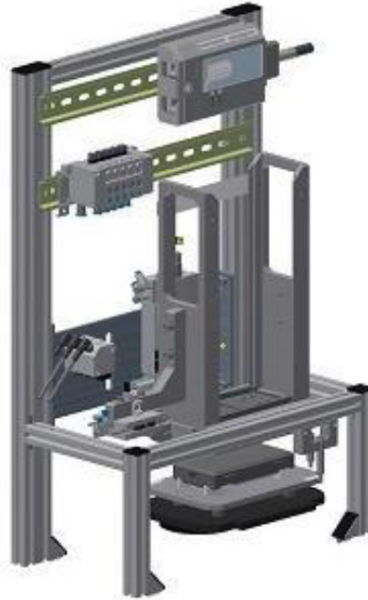


Figure 8.7. Magazine Back Workstation

The inputs to the "*Solver*" function of the magazine back Digital Twin are:

- *iErrNo*: it changes according to the error type on the HMI display.
- *xCL.BG5*: it senses the presence of back covers in the magazine.
- *xCL.BG7*: it senses the presence of the front cover on the pallet.
- *xCL.BG8*: it senses the presence of the back cover on top of the front cover.
- *diONo*: it shows the *number* of the processed work order.
- *iOPos*: it shows the *position* of the processed work order.

The outputs to the "*Solver*" function are:

- *MagazineFilled*: variable used to solve the Error ID 9.
- *HMIError*: it shows if there is an error on the MES display.
- *OperationDone*: variable used to solve Error IDs 8 and 10.
- *ErrorID*: it displays the ID of the error occurring.

Error ID	xCL_BG5	xCL_BG7	xCL_BG8	iErrNo
8	0	0	1/0	4
9	1	0	0	8
10	1	1	1/0	3

Table 8.6. Sensors combination for error states at the Magazine Back

Based on the sensors combination of Table 8.6, a batch of three error states is detectable, and respective solving paradigms are proposed.

- **Error ID 8** (type B): *front cover not available*. The production flow gets stuck since there is no front cover to discharge the back cover onto. So, the Digital Twin sends an "EndOPos" string to the MES (**MClass** = 101, **MNo** = 21, cf. Appendix A), that annuls the order that is being identified by *iOPos* and *diPNo*. After this, the *OperationDone* value changes and so the *xReject* sensor is overwritten by the Level-2 Matlab S-Function, and the workstation is cleared.
- **Error ID 9** (type A): *magazine of back covers is empty*. The production low gets interrupted since the magazine of back covers is empty. The Digital Twin waits for the operator to fill up the magazine. When this happens the *MagazineFilled* changes value, in order to trigger the *xRetry* sensor. Like this the order can be processed correctly and the production can keep on proceeding.
- **Error ID 10** (type B): *back cover already available*. The order cannot be processed by the workstation since the back cover is already present on top of the front one. The Digital Twin intervenes by sending a "OpEnd" string to the MES (**MClass** = 101, **MNo** = 20 cf. Appendix A), so the current operation is ended and the workplan sequence switches to the next planned operation. Once this operation is done, the *OperationDone* output changes value, and the *xReject* sensor is automatically triggered in order to release the carrier from the workstation.

8.1.2.5 Press Station

Lastly, the press station presses the front and the back covers previously assembled. It is the last possible step to be performed for the programmable workplans of the assembly line.

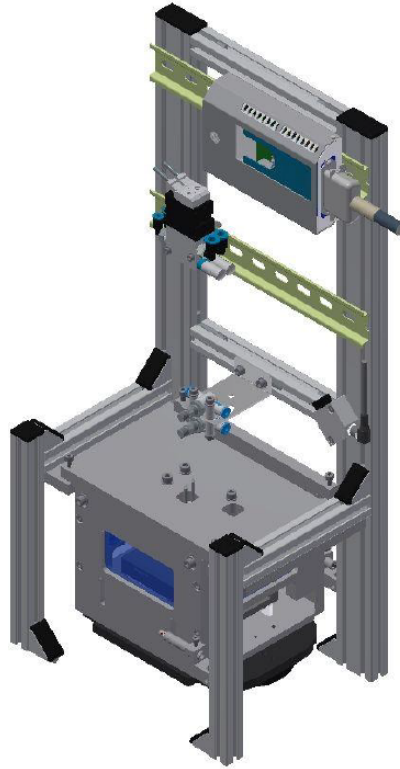


Figure 8.8. Press Workstation

The inputs to the "Solver" block of the Press Digital Twin are:

- *xHL.BG8*: it senses the presence of the back cover to be pressed.
- *iErrNo*: it tells the type of error displayed on the HMI.
- *diONo*: it shows the *number* of the processed work order.
- *iOPos*: it shows the *position* of the processed work order.

The outputs of the "Solver" are:

- *OperationDone*: it is used to solve the Error ID 11.
- *ErrorID*: it shows the type of error displayed on the *hmi*.

Based on the sensors combination of Table 8.7, there is only one detectable error state for the Press workstation, the **Error ID 11**, that occurs when *no back cover is available* in order for it to be pressed on the front cover. Once the condition is met, the Digital Twin sends the "EndOPos" (**MClass = 101**, **MNo = 21**, cf. Appendix A) string

Error ID	xHL_BG8	iErrNo
11	0	5

Table 8.7. Sensors combination for error states at the Press

to the MES, in order to abort the order that is being identified by *diONo* and *iOPos*. Afterwards, the output value of *OperationDone* is automatically changed, in order to allow the Digital Twin to trigger the *xReject* sensor in order to clear the workstation and release the carrier.

8.2 DISASSEMBLY PROPOSAL

In this paragraph, an automatized way to cope with quality issues is provided. In fact, in order to avoid the scrapping of a component that is not compliant with some standards, a disassembly work plan can be reactively scheduled. Basically, this part of the work aims at performing the tasks shown in Figure 8.9, that are the application of the **disassembly framework** previously mentioned in Chapter 5.



Figure 8.9. Disassembly tasks proposal to be performed by a tailored Digital Twin

8.2.1 Camera Inspection

In Industry 4.0 Lab, this kind of activity can be performed by properly modelling a Digital Twin tool based on the workstation whose task is to check compliance with quality standards: the **Camera Inspection** (Figure 8.10).

This workstation is placed right after the Robot Assembly cell, and checks whether the assembly work done so far is compliant with respect to the workplan requirements. The pallet passes through the workstation, where a camera, together with light beam, checks the aforementioned conformity.

The workstation does not stop the working process if these standards are not satisfied, but it is able to detect it using specific embedded sensors values.

The final aim of this station, as it is meant in the framework application, is to be able to reactively schedule a disassembly order that is going to be performed by the Robot Assembly. The latter, in fact, can be easily configured to perform several activities, both of assembly and of disassembly nature.

Accordingly, the requirements to apply the *disassembly framework*, as already explained in Chapter 5, are met:

- the workstation that performs the **assembly** work is at disposal: *Robot Assembly*;

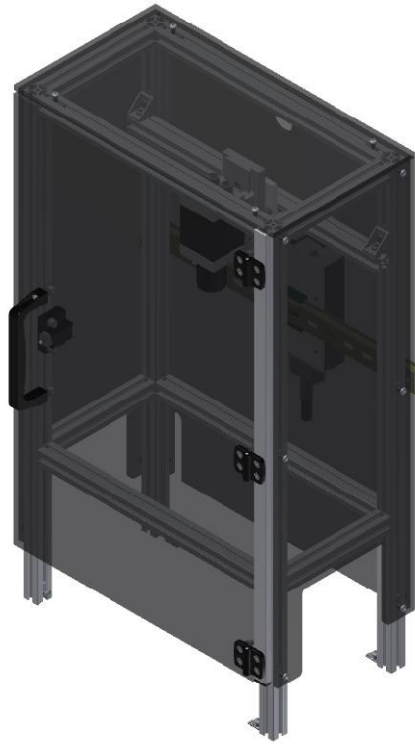


Figure 8.10. Camera Inspection Workstation

- the workstation relative to the **visual check** is available: *Camera Inspection*;
- the workstation that carries out the **disassembly** process is in the plant: *Robot Assembly*.

8.2.2 Simulation Model

The real-time Digital Twin proposed to perform this task is an extension of the Digital Shadow that has already been validated in the Lab. The blocks that have been coded and modelled in this work of thesis to perform this specific reactive-disassembly task are:

- the **"Detector"** function, that in Simulink environment translates in a *Level-2 Matlab S-Function* that works exactly like in the previous cases. It takes values of sensors coming from the Camera Inspection workstation and enables operations based on these values.
- the **"Disassembler"** function that takes some input sensors values, coming from the Level-2 Matlab S-Function, and sends commands to the MES.

The Simulink Digital Twin model is relative to the *Camera Inspection* workstation, and it is graphically shown in Figure 8.11.

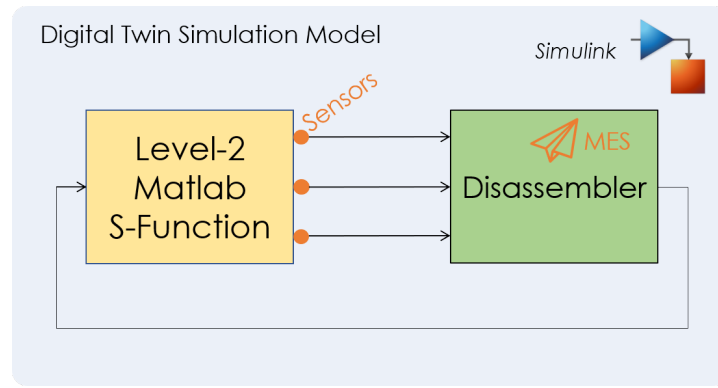


Figure 8.11. Structure of the Digital Twin on Simulink with the integration of a "Disassembler" function to connect to the MES

Like in the previous cases, the analysis of the inputs and the outputs of the "Disassembler" function is going to be performed.

The inputs are hereby described.

- *xResult*: this sensor detects if the assembly process done so far is compliant to the workplan. The boolean variable is *true* if there is compliance, whilst it is *false* if not.
- *diONo*: it shows the *number* of the processed work order.
- *iOPos*: it shows the *position* of the processed work order.

The only output is the *OperationDone* variable, that says whether the disassembly workplan re-scheduling has taken place.

The "Disassembler" Matlab function script can be found in Appendix C.

8.2.3 Disassembly framework application

The steps to effectively run this peculiar simulation model are hereby described. This procedure is nothing but the application of the *reactive disassembly framework* already shown in Figure 5.3 in Chapter 5. The actions carried out are pointed out with respect to their nature, either physical or digital.

1. A workorder that plans the assembly of a front cover with a PCB and one or more fuses is planned and launched.
2. The Simulink Matlab function checks at every sampled instant if the *xResult* variable changes its value from *true* to *false*.
3. When the order approaches the Camera Inspection workstation and this condition is verified, the *xResult* becomes *false*. This

means that work order that is being assembled is not compliant with the workplan expectations.

The case analyzed is relative to the workplan that assemble, inside the covers of the prototypical phone, a PCB and one or more fuses. The *xResult* changing its value to *false* indicates the lack of fuses on the assembled component, detecting the presence of only front cover and PCB on the pallet and that at least a fuse is missing from the assembled product. Thus, the simulation code aborts the order that just passed by the Camera Inspection (identified by *diONo* and *iOPos*) thanks to the "EndO-Pos" string command (**MClass** = **101**, **MNo** = **21**, cf. Appendix A) sent to the MES.

4. As soon as this task is done, a further order is immediately re-scheduled on the same pallet, where the front cover with the assembled PCB lies. This particular order has been customized on the MES software in order to perform these tasks:
 - unscrewing (simulated) of screws at the *Drilling* workstation;
 - automatic disassembly of the PCB from the front cover at the *Robot* workstation;
 - inspection to check if the disassembly has taken place, at the *Camera Inspection*;
 - manual unmounting of the front cover from the pallet at the *Manual* station, by an operator.

This kind of workplan is automatically planned by the Digital Twin, using the "SetOrder" string (**MClass** = **101**, **MNo** = **2**, cf. Appendix A) in order to avoid scrapping of material due to the found quality inconsistencies. At this point the whole operation can be considered done and the *OperationDone* output changes value. Based on this, thanks to a feedback loop to the Level-2 Matlab S-Function, the *xResult* value is set back to *true*, otherwise it would stay *false*.

This kind of approach further validates the proposal of the optimization frameworks done in Chapter 5. The proposed Digital Twin, thanks to its high integration level with respect to its physical part given by the MES integration, is proven to be able to schedule new work orders as a reaction to peculiar shop-floor situations.

Accordingly, the optimization results, with a set of comments, is given in Chapter 9.

VALIDATION AND RESULTS

In this Chapter, an overview of the results coming from the application of the optimization frameworks performed in Chapter 8 is given.

The analysis concerns both the *error states management* (paragraph 9.1) and the *reactive disassembly* (paragraph 9.2) frameworks.

9.1 RESULTS: ERROR STATES MANAGEMENT

In this section the simulation runs that have been performed to prove the effectiveness of the tool are reported. This is the last step of the optimization procedure, following the detection of error states and the tailoring of a Digital Twin to autonomously cope with them.

The Key Performance Indicator (KPI) chosen to report the magnitude of the optimization performed by the running of the integrated Digital Twin is the **percentage time reduction**. The time decrease is calculated among two separate scenarios:

- *Scenario 1*: un-optimized running of production orders *without* the Digital Twin running. The error states are tackled only thanks to the intervention of an operator. The mean downtimes of this scenario have already been recorded and indicated in Table 8.2 in Chapter 8.
- *Scenario 2*: optimized production of work orders *with* the running of the designed Digital Twin integrated with the MES. The error states are solved with an integrated intervention of human resources and automated smart systems *e.g.* the Digital Twin that communicates with the MES.

The comparative results of the aforementioned scenarios, together with the indication of percentage time reduction thanks to the optimization procedure, are reported in Table 9.1.

Based on the data displayed in Table 9.1, some considerations are worth doing:

- For the case of error states of **type B**, whose percentage reduction is reported in *red* in Table 9.1, the downtime optimization was found to be critically reduced. This has been possible since

Error ID	Type	Mean Downtime <i>Scenario 1</i> [s]	Mean Downtime <i>Scenario 2</i> [s]	Downtime Reduction [%]
1	A	16.28	12.12	25.55
2	B	12.86	1.59	87.54
3	B	13.23	1.39	89.53
4	B	13.86	1.69	87.83
5	A	66.75	27.00	60.64
6	B	14.89	2.40	83.83
7	B	11.84	2.10	82.18
8	B	14.39	2.09	85.55
9	A	19.74	13.96	29.27
10	B	13.05	2.09	85.55
11	B	13.81	2.13	84.59

Table 9.1. Mean downtime reduction of error states thanks to the use of integrated Digital Twin

the functions of the designed Digital Twin make it possible to completely avoid the interaction of the operator with the machines. However, this can be done by imposing a standard way to cope with each kind of error state in the Digital Twin model. For instance, when at the station that drills the front covers, the pallet arrives without front cover, the algorithm *always* aborts the production order. The decision-making ability at the base of this assumption can be given to the Digital Twin with integration of AI models *etc.* in order to give to the algorithm the ability to choose which is the best action to perform, based on certain conditions. Anyhow, this possibility has not been considered in this work of thesis.

However, based on this strong standardization assumption, the downtime was reducible by a percentage range between **80%** and **90%**. Besides, the values of the optimized mean downtime for the error states of type B are almost standard. This is due to the fact that the computational time of the simulation code and of the communication protocols is standard and almost deterministic for every adopted solution to solve the error states. Hence, in a certain amount of time (3 seconds maximum), the issue is solved.

- Taking into consideration the error states of **type A**, it may be noticed that the overall optimization has a smaller impact with respect to the error states of type B. This is due to the nature of the error states of type A themselves. As a matter of fact, as already mentioned, to solve this kind of shop-floor issues, human intervention is mandatory to perform at least one physical activity *e.g.* fill up a magazine of covers. Subsequently, the op-

timization cannot be fully automatized like in errors of type B. Nevertheless, the optimization was proven to be effective, with downtime reduction ranging from **25%** to **60%**. Furthermore, since this kind of errors need human intervention also in the optimized scenario, the gathering of data of the optimized case are subject to a stochastic variability due to the unpredictability of operators work. This leads to a variability of optimized downtime, that differs from the almost deterministic time of error of type B.

- To prove the effectiveness of the *human-machine* interaction optimization, **error ID 5** is worthy of a particular mention. This error type is the one where multiple actions need to be performed: physical and digital filling of the PCB box. With the use of the integrated Digital Twin, the operator is induced to deal with the problem since the code itself creates the condition for the error state to be more noticeable. In fact, *without* the Digital Twin, the only error indicator is on the HMI display. Yet, *with* the running of the Digital Twin, when this peculiar error state occurs, not only the HMI displays an alert message, but the PCB box itself is automatically expelled from the robot cell. This is by far more visible and indicative of an ongoing error state than a simple alert message on a touch-pad, especially if the operator is in charge of several tasks. This is an ultimately effective way to optimize the human-machine interaction when error states occur.

9.2 RESULTS: DISASSEMBLY

For the application of the reactive disassembly framework, the results and benefits have a more qualitative nature.

- The main objective and benefit that has been proven is the ability of the proposed Digital Twin to **reactively schedule disassembly workplan**, based on certain conditions that can be detected by the embedded sensor system. This outcome is a further validation of the possibility to react to shop-floor events that, differently from the error states case, do not block the production flow. In fact, the production keeps running, but the lack of compliance with the assembly standards is automatically identified by the Digital Twin, and the disassembly workplan is automatically scheduled.
- According to a broader perspective, this kind of activity allows the possibility to **reduce scraps** in the facility in which this application is performed. In fact, the pieces that are not compliant with certain standards are usually scrapped, causing potential

money loss and inefficiencies for the company. With the use of a Digital Twin able to perform the activity of reactive disassembly, a sustainable paradigm can be applied to the manufacturing environment. Accordingly, the materials are not scrapped, but they can be recovered and later re-used according to the application of a *circular economy* policy.

9.3 VALIDATION OF MODEL

The application of the frameworks proposed in Chapter 5 in a small-scale manufacturing environment like Industry 4.0 Laboratory of Politecnico di Milano, gives a strong validation of the models themselves.

As a matter of fact, the possibility to optimize shop-floor events both in error states management and in reactive scheduling terms has been proven valid by the numerical results of the application runs of the tailored Digital Twins.

In Chapter 10, an overview of the conclusions of the work of thesis is given, with specific reference to the research objectives and to the way they have been fulfilled with the proposed work.

CONCLUSIONS

The overall thesis work aims at finding solutions to give autonomy and MES integration functionality to a Digital Twin simulation tool. In the big Industry 4.0 paradigm, the research field still lacks the ability to provide to Digital Twins the skill to not only monitor their physical counterparts, but also to actively control them.

The work of thesis, by integrating the *Digital Shadow* — with only monitoring functionalities — of a prototypical assembly line with its Manufacturing Execution System software, finds a way to control the line itself, providing optimization and re-scheduling frameworks to reduce the downtime of shop-floor error events. Besides, the proposed optimization frameworks are applied in the field level by means of the modelling of tailored Digital Twins with *bi-lateral* communication ability between digital and physical parts. The implemented simulation tools on physical assets at Industry 4.0 Laboratory of Politecnico di Milano, provide the necessary validation to the proposed optimization frameworks.

In this Chapter, an analysis of the fulfillment of the research objectives declared in Chapter 4 is performed. This task is carried out by explaining how the implementation of the validated models of the work of thesis open the way to providing the exact validated application cases that are necessary to fill the research gaps.

Furthermore, an overview of the possible future endeavors to enrich the models proposed in the work of thesis, is provided.

10.1 PRACTICAL OBJECTIVES

Based on what anticipated in Chapter 4 about the Research Design, the research field lacks in many application aspects. The literature study led to the definition of practical objectives that needed to be fulfilled in the validation field: Industry 4.0 Laboratory of Politecnico di Milano.

In the laboratory many activities have been performed in the proposed work of thesis, all aimed at the modelling and validation of a Digital Twin, able to autonomously cope with shop-floor issues by communicating with its MES software.

The base of all the modelling lies in the study of technologies that are at the root of the Digital Twin, as modelled in the I4.0 Lab envi-

ronment. As a matter of fact, previous works in the lab implemented a merely monitoring simulation tool, which in this work of thesis is referred to as *Digital Shadow* [1]. With the use of a set of know-hows (e.g. OPC UA Suite), this Digital Shadow was implemented and validated for the monitoring of machine states and energy consumption. Nevertheless, as anticipated in the literature study, the endeavor of this work of thesis lies in finding a way to provide *bi-lateral* communication between physical and digital sides of an asset, enabling the Digital Twin with decision-making control functionalities.

The cornerstone of this whole proposed work lies in the conception of **optimization frameworks** that might be applied in a generic manufacturing environment. The functioning of these theoretical models finds its basis on the parallel use of the *physical* and *digital* side of the same industrial resource. In this way, the digital side, meant as Digital Twin, can enable a *bi-lateral* communication with its physical counterpart, making it possible to optimize the production in terms of error state management and reactive scheduling.

To perform all of these activities in a laboratory environment, and to validate the proposed models, the study of communication protocols to give commands to the MES from external engineering tools (e.g. TCP/IP) turned out to be an important part of the work.

Given this context study, after the formalization of the optimization models, the practical implementations have been carried out:

- Integration of the MES to the Digital Shadow by using a communication protocol, able to give commands to the MES from external sources. The whole model has been implemented on *Matlab & Simulink* engineering tools. With this integration, the *only-monitoring* Digital Shadow, became a *bi-lateral* communication-based Digital Twin.
- Once having designed a model to integrate the MES to the Digital Twin, applications to validate it have been carried out in the field. The first one — that corresponds to the error states management framework — is based on the identification, classification and timing of shop-floor error states concerning the assembly line. Based on this work, the Digital Twin has been tailored in order to autonomously react to the aforementioned error states, providing a substantial optimization via reduction of mean downtime.
- A second application of the model — that coincides with the disassembly framework — concerns the ability to react to assembly lack of conformity with respect to work orders. When this predicament occurs, the Digital Twin is modelled to abort the non-compliant order and re-schedule a disassembly order to avoid scrapping.

The realization of the practical objectives led to the fulfillment of the higher level research objectives, that have been also stated in Chapter 4. This kind of objectives, in turn, come from the definition of *gaps* that have been found in the research environment.

- **Main Objective.** The concept of Digital Twin has been widely studied, but, being at an early stage of development, still many applications are lacking. Specifically, almost no study have been carried out regarding digital simulation models that are able to *control* their physical counterparts. The majority of research endeavors lie in the so-called *Digital Shadows*, able only to gather data from the physical field and to change their state only when the tangible asset does it. This objective proposed to create a **Digital Twin that is able to communicate in both directions between the physical and digital part.**

In this work of thesis, the development of a Digital Twin with the possibility to send commands to the MES when some conditions are met, shows and validates a *bi-lateral* communication between the digital and the physical side of a given asset. Accordingly, when the physical asset changes its state, the Digital Twin changes as well. But most importantly, when the Digital Twin changes its state, also the physical counterpart proceeds consequently. Thanks to the use of communication protocols that are able to send commands to the MES and subsequently to the PLC and the physical resource, the proposed Digital Twin model is validated. As a matter of fact, decision-making control from the digital model to the physical asset is granted.

- **Sub-Objective 1.** Many studies about the possibility to optimize the processes of an assembly line using smart tools have been carried out in literature. This objective puts forward the **application of optimization measures using a Digital Twin in terms of: error states management; human-machine interaction; re-scheduling; disassembly.**

The proposed work of thesis, thanks to the application cases of the integrated Digital Twin on the assembly line at Industry 4.0 Laboratory, introduces and validates *optimization frameworks* over multiple aspects. As a matter of fact, the aforementioned frameworks touch various fields: from *error states management* following a *lean* time-saving policy, to the enhancement of the *human-machine* interaction performance. Furthermore, a framework proposal to avoid the scrapping of work pieces is advanced. Both frameworks are validated with the implementation of tailored Digital Twins able to: *optimize error states down-*

time; *reactively schedule a disassembly workplan* for a piece that is found not to be compliant with the assembly standards.

- **Sub-Objective 2.** According to the MESA International standards, one of the MES functions is to provide a higher level *Process Management* for industrial facilities. What is already done by the MES software tools found in the marketplace is the possibility to plan and manage the flows of work orders in the plants. This objective advances the possibility to **improve the Process Management MES functionality**.

The proposed work of thesis gives a higher sensibility level to the *Process Management* MES capability. In fact, the MES is integrated to a real-time simulation tool, the Digital Twin. The latter exploits the values of the embedded sensors on the production line to deduce and monitor the machine states in real-time. This information is automatically used also by the MES, that is integrated to the Digital twin using communication protocols and commands. This allows the MES to have a higher *sensibility* and *autonomy of control* of the flow of work, based on the actual production activities or error states deduced by the sensors embedded in the line and used by the Digital Twin.

10.3 FUTURE DEVELOPMENTS

This work of thesis, with the ability to send commands in real-time to the MES from a Digital Twin simulation tool, paved the way for many possible future developments.

Some possibilities are offered by **maintenance policies** that might enable the possibility to reactively re-schedule orders according to forecasting models, and not on shop-floor events.

Accordingly, one of the possible future applications may concern the integration of *predictive maintenance* models to the simulation tool. This work would enable the possibility to have a more consistent decision-making tool, based on criticalities shown by the results of the predictive models.

A second future development might be based on *condition based maintenance* models. When the monitoring of a certain tool shows an abnormal behaviour, the orders up to be processed to that particular workstation could be deleted or re-scheduled. For this kind of application, the presence of an historical data-base of relevant physical features of the assets of the assembly line could be useful.

Another possibility can be the monitoring of **energetic anomalies** coming from the physical assets of the production line. Based on specific statistical analysis of these kind of data, real-time decisions about the production orders can be taken using the integrated Digital Twin.

BIBLIOGRAPHY

- [1] Kritzinger, W., Karner, M., Traar, G., Henjes, J., Sihn, W., *Digital Twin in manufacturing: A categorical literature review classification*, (2018) IFAC PapersOnLine 51-11, pp. 1016-1022.
- [2] Liu, Q., Zhang, H., Leng, J., Chen, X., *Digital twin-driven rapid individualised designing of automated flow-shop manufacturing system*, (2018) International Journal of Production Research.
- [3] MESA International - White Paper Number 6, *MES Explained: A High Level Vision*, (1997).
- [4] Zheng, P., Wang, H., Sang, Z., Zhong, R.Y., Liu, Y., Liu, C., Mubarak, K., Yu, S., Xu, X., *Smart manufacturing systems for Industry 4.0: Conceptual framework, scenarios, and future perspectives*, (2018) Frontiers of Mechanical Engineering, 13 (2), pp. 137-150.
- [5] Grieves, M., Vickers, J. *Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems*, (2017) Trans-Disciplinary Perspectives on System Complexity.
- [6] Negri, E., Fumagalli, L., Macchi, M., *A review of the roles of Digital Twin in CPS-based production systems*, (2017), Procedia Manufacturing 11, pp. 939-948.
- [7] MESA International - White Paper Number 1, *The Benefits of MES: A Report from the Field*, (1997).
- [8] Bystricanova, A., Rybovic, A., *Data Communication between Programmable Logic Controllers in the Industrial Distribution Applications*, (2011) Control Engineering, Volume 9, pp. 96-102.
- [9] Kielan, P., Krawczyk, D., *Using The Embedded Ethernet Communication Capability of PLC controller for tailor-made Manufacturing Execution System*, (2015) 16th International Conference on Research and Education in Mechatronics (REM).
- [10] Iwatsu, S., Watanabe, Y., Kodama, K., *Development of communications protocols between manufacturing execution system and production equipment*, (2007) IEEE Conference on Emerging Technologies and Factory Automation, pp. 280-287.
- [11] Nikolic, B., Ignjatic, J., Suzic, N., Stevanov, B., Rikalovic, A., *Predictive manufacturing systems in industry 4.0: Trends, benefits and challenges*, (2017) Annals of DAAAM and Proceedings of the International DAAAM Symposium, pp. 796-802.

- [12] Li, X., Li, D., Wan, J., Vasilakos, A.V., Lai, C., *A review of industrial wireless networks in the context of Industry 4.0*, (2017) *Wireless Netw*, pp. 23:23–41.
- [13] Peres, R.S., Dionisio Rocha, A., Leitao, P., Barata, J., *IDARTS – Towards intelligent data analysis and real-time supervision for industry 4.0*, (2018) *Computers in Industry*, 101, pp. 138-146.
- [14] Wang, S., Liang, Y.C., Li, W.D., Cai, X.T., *Big Data enabled Intelligent Immune System for energy efficient manufacturing management*, (2018) *Journal of Cleaner Production* 195, pp. 507-520.
- [15] Giusti, F., Bevilacqua, M., Tedeschi, S., Emmanouilidis, C., *Data Analytics and Production Efficiency Evaluation on a Flexible Manufacturing Cell*, (2018) *I2MTC 2018 - 2018 IEEE International Instrumentation and Measurement Technology Conference: Discovering New Horizons in Instrumentation and Measurement*, Proceedings, pp. 1-6.
- [16] Lenz, J., Wuest, T., Westkämper, E., *Holistic approach to machine tool data analytics*, (2018) *Journal of Manufacturing Systems*, 48, pp. 180-191.
- [17] Gunasekaran, A., Yusuf, Y.Y., Adeleye, E.O., Papadopoulos, T., *Agile manufacturing practices: the role of big data and business analytics with multiple case studies*, (2018) *International Journal of Production Research*, 56 (1-2), pp. 385-397.
- [18] Suryajaya, B., Chen, C.-C., Hung, M.-H., Liu, Y.-Y., Liu, J.-X., Lin, Y.-C., *A fast large-size production data transformation scheme for supporting smart manufacturing in semiconductor industry*, (2018) *IEEE International Conference on Automation Science and Engineering*, 2017-August, pp. 275-281.
- [19] Cattaneo, L., Fumagalli, L., Macchi, M., Negri, E., *Clarifying Data Analytics Concepts for Industrial Engineering*, (2018) *IFAC-PapersOnLine*, 51 (11), pp. 820-825.
- [20] Woo, J., Shin, S.-J., Seo, W., Meilanitasari, P., *Developing a big data analytics platform for manufacturing systems: architecture, method, and implementation*, (2018) *International Journal of Advanced Manufacturing Technology*, Article in Press.
- [21] O'Donovan, P., Gallagher, C., Bruton, K., O'Sullivan, D.T.J., *A fog computing industrial cyber-physical system for embedded low-latency machine learning Industry 4.0 applications*, (2018) *Manufacturing Letters*, 15, pp. 139-142.
- [22] Verma, S., Kawamoto, Y., Fadlullah, Z.M., Nishiyama, H., Kato, N., *A Survey on Network Methodologies for Real-Time Analytics of*

- Massive IoT Data and Open Research Issues*, (2017) IEEE Communications Surveys and Tutorials, 19 (3), art. no. 7900337, pp. 1457-1477.
- [23] Trunzer, E., Kirchen, I., Folmer, J., Koltun, G., Vogel-Heuser, B., *A flexible architecture for data mining from heterogeneous data sources in automated production systems*, (2017) Proceedings of the IEEE International Conference on Industrial Technology, art. no. 7915517, pp. 1106-1111.
 - [24] Schuh, G., Blum, M., *Design of a data structure for the order processing as a basis for data analytics methods*, (2017) PICMET 2016 - Portland International Conference on Management of Engineering and Technology: Technology Management For Social Innovation, Proceedings, art. no. 7806715, pp. 2164-2169.
 - [25] Zhong, R.Y., Xu, X., Klotz, E., Newman, S.T., *Intelligent Manufacturing in the Context of Industry 4.0: A Review*, (2017) Engineering, 3 (5), pp. 616-630.
 - [26] Liu, G., Yang, J., Hao, Y., Zhang, Y., *Big data-informed energy efficiency assessment of China industry sectors based on K-means clustering*, (2018) Journal of Cleaner Production, 183, pp. 304-314.
 - [27] Accorsi, R., Manzini, R., Pascarella, P., Patella, M., Sassi, S., *Data Mining and Machine Learning for Condition-based Maintenance*, (2017) Procedia Manufacturing, 11, pp. 1153-1161.
 - [28] Kartal, H., Oztekin, A., Gunasekaran, A., Cebi, F., *An integrated decision analytic framework of machine learning with multi-criteria decision making for multi-attribute inventory classification*, (2016) Computers and Industrial Engineering, 101, pp. 599-613.
 - [29] Peña, M., Biscarri, F., Guerrero, J.I., Monedero, I., León, C., *Rule-based system to detect energy efficiency anomalies in smart buildings, a data mining approach*, (2016) Expert Systems with Applications, 56, pp. 242-255.
 - [30] Wang, J., Ma, Y., Zhang, L., Gao, R.X., Wu, D., *Deep learning for smart manufacturing: Methods and applications*, (2018) Journal of Manufacturing Systems, 48, pp. 144-156.
 - [31] Flath, C.M., Stein, N., *Towards a data science toolbox for industrial analytics applications*, (2018) Computers in Industry, 94, pp. 16-25.
 - [32] Caggiano, A., Rimpault, X., Teti, R., Balazinski, M., Chatelain, J.-F., Nele, L., *Machine learning approach based on fractal analysis for optimal tool life exploitation in CFRP composite drilling for aeronautical assembly*, (2018) CIRP Annals, 67 (1), pp. 483-486.

- [33] Ge, Z., Song, Z., Ding, S.X., Huang, B., *Data Mining and Analytics in the Process Industry: The Role of Machine Learning*, (2017) IEEE Access, 5, art. no. 8051033, pp. 20590-20616.
- [34] Miškuf, M., Zolotová, I., *Comparison between multi-class classifiers and deep learning with focus on industry 4.0*, (2016) 2016 Cybernetics and Informatics, K and I 2016 - Proceedings of the 28th International Conference, art. no. 7438633.
- [35] Riedel, M., Memon, A.S., Memon, M.S., *High productivity data processing analytics methods with applications*, (2014) 2014 37th International Convention on Information and Communication Technology, Electronics and Microelectronics, MIPRO 2014 - Proceedings, art. no. 6859579, pp. 289-294.
- [36] Shin, S.-J., Woo, J., Rachuri, S., *Predictive analytics model for power consumption in manufacturing*, (2014) Procedia CIRP, 15, pp. 153-158.
- [37] Dutta, R., Mueller, H., Liang, D., *An interactive architecture for industrial scale prediction: Industry 4.0 adaptation of machine learning*, (2018) 12th Annual IEEE International Systems Conference, SysCon 2018 - Proceedings, pp. 1-5.
- [38] Nedelkoski, S., Stojanovski, G., *Machine learning for large scale manufacturing data with limited information*, (2017) IEEE International Conference on Control and Automation, ICCA, art. no. 8003037, pp. 70-75.
- [39] Choudhary, A.K., Harding, J.A., Tiwari, M.K., *Data mining in manufacturing: a review based on the kind of knowledge*, (2009) Journal of Intelligent Manufacturing, pp. 20:501-521.
- [40] Gröger, C., Niedermann, F., Mitschang, B., *Data Mining-driven Manufacturing Process Optimization*, (2012) Proceedings of the World Congress on Engineering 2012 Vol III.
- [41] Oh, S., Han, J., Cho, H., *Intelligent Process Control System for Quality Improvement by Data Mining in the Process Industry*, (2001) D. Braha (ed.), Data Mining for Design and Manufacturing, pp. 289-309.
- [42] Wang K., *Applying data mining to manufacturing: the nature and implications*, (2007) Journal of Intelligent Manufacturing, pp. 18:487-495.
- [43] Batanov, D., Nagarur, N., Nitikhunkasem, P., *EXPERT-MM: A knowledge-based system for maintenance management*, (1993) Artificial Intelligence in Engineering, pp. 283-291.

- [44] Thoben, K.-D., Wiesner, S., Wuest, T., *"Industrie 4.0" and Smart Manufacturing – A Review of Research Issues and Application Examples*, (2017) IJAT (International Journal of Automation Technology): IJAT 11, pp. 4–16.
- [45] Li, H., Parikh, D., He, Q., Qian, B., Li, Z., Fang, D., Hampapur, A., *Improving rail network velocity: A machine learning approach to predictive maintenance*, (2014) Transportation Research Part C 45, pp. 17–26.
- [46] Harding, J.A., Shahbaz, M., Srinivas, Kusiak, A., *Data Mining in Manufacturing: A Review*, (2006) J. Manuf. Sci. Eng 128(4), pp. 969-976.
- [47] Shahbaz, M., Srinivas, Harding, J.A., Turner, M., *Product design and manufacturing process improvement using association rules*, (2005) Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture, 220 (2), pp. 243-254.
- [48] Lee, J., Bagheri, B., Kao, H.-A., *Recent Advances and Trends of Cyber-Physical Systems and Big Data Analytics in Industrial Informatics ResearchGate*, (2014) Keynote given at the 12th IEEE International Conference on Industrial Informatics (INDIN 2014), Porto Alegre, Brazil.
- [49] Cottyn, J., Van Landeghem, H., Stockman, K., Derammelaere, S., *A method to align a manufacturing execution system with Lean objectives*, (2011) International Journal of Production Research, Vol. 49, pp. 4397-4413.
- [50] D'Antonio, G., Sauza Bedolla, J., Chiabert, P., *A novel methodology to integrate Manufacturing Execution Systems with the lean manufacturing approach*, (2017) Procedia Manufacturing 11, pp. 2243 – 2251.
- [51] You, F.-X., Zhao, X.-Q., You, Y.-L. *Production Management and Optimization in Injection Molding Companies Based on MES*, (2017) 2nd International Conference on Mechatronics, Control and Automation Engineering.
- [52] Pfitzer, F., Mieth, C., Liertz, W., Provost, J., *Event-driven production rescheduling in job shop environments. Application to sheet metal production processes*, (2018) IEEE 14th International Conference on Automation Science and Engineering (CASE).
- [53] Chang, M.M.L., Ong, S.K., Nee, A.Y.C., *Approaches and Challenges in Product Disassembly Planning for Sustainability*, (2017) Procedia CIRP, pp. 506-511.

- [54] Rosen, R., von Wichert, G., Lo, G., Bettenhausen, K.D., *About The Importance of Autonomy and Digital Twins for the Future of Manufacturing*, (2015) IFAC-PapersOnLine 48-3, pp. 567-572.
- [55] Tao, F., Zhang, M., *Digital Twin Shop-Floor: A New Shop-Floor Paradigm Towards Smart Manufacturing*, (2017), IEEE Access (Volume 5), pp. 20418-20427.
- [56] Tao, F., Cheng, J., Qi, Q., Zhang, M., Zhang, H., Sui, F., *Digital twin-driven product design, manufacturing and service with big data*, (2018) Int J Adv MANuf Technol, 94:3563-3576.
- [57] Vathoopan, M., Johny, M., Zoitl, A., Knoll, A., *Modular Fault Ascription and Corrective Maintenance Using a Digital Twin*, (2018) IFAC PapersOnLine 51-11, pp. 1041-1046.
- [58] Qi, Q., Tao, F., *Digital Twin and Big Data Towards Smart Manufacturing and Industry 4.0: 360 Degree Comparison*, (2018) IEEE Access (Volume 6), pp. 3585-3593.
- [59] Zhuang, C., Liu, J., Xiong, H., *Digital twin-based smart production management and control framework for the complex product assembly shop-floor*, (2018) The International Journal of Advanced Manufacturing Technology, 96:1149-1163.
- [60] Zhang, H., Liu, Q., Zhang, D., Leng, J., *A Digital Twin-Based Approach for Designing and Multi-Objective Optimization of Hollow Glass Production Line*, (2017) IEEE Access (Volume 5), pp. 26901-26911.
- [61] Lee, H., Kim, T., *Smart Factory Use Case Model Based on Digital Twin*, (2018) ICIC International ISSN 2185-2766, pp. 931-936.
- [62] Lu, Y., Xu, X., *Cloud-based manufacturing equipment and big data analytics to enable on-demand manufacturing services*, (2019) Robotics and Computer Integrated Manufacturing 57, pp. 92-102.
- [63] Fumagalli, L., Macchi, M., Pozzetti, A., Taisch, M., Tavola, G., Terzi, S., *New methodology for smart manufacturing research and education: The lab approach*, (2016) 21st Summer School Francesco Turco, AIDI-Italian Association of Industrial Operations Professors, pp. 42-47.
- [64] Cavalieri, S., Salafia, M.G., Scroppo, M.S., *Integrating OPC UA with web technologies to enhance interoperability*, (2019) Computer Standards & Interfaces 61, pp. 45-64.
- [65] Amezquita Martinez, D.A., Cataldo, A., Ramírez Herendia, R.E., *TCP/IP Communication for a De-Manufacturing Transport Line*, (2016) 1er Congreso Internacional sobre Soluciones en Inteligencia Ambiental , Ingeniería de Software y Salud Electrónica & Móvil, pp. 55-59.

- [66] Cerf, V.G., Cain, E., *The DoD Internet Architecture Model*, (1983) Computer Networks 7, pp. 307-318.
- [67] Cerf, V.G., Kahn, R.E., *A Protocol for Packet Network Intercommunication*, (1974) IEEE Trans on Comms, Vol Com-22, No 5.
- [68] Gressier-Soudan, E., Lefebvre, M., Natkin, S., *TCP/IP Manufacturing Applications : an experiment with MMS over RPC*, (1995).
- [69] Cheah, R. S.-S., Lee, B.-S., Lim, R.L., *Design and implementation of an MMS environment on ISODE*, (1997) Computer Communications 20, pp- 1354-1364.
- [70] Piazza, E., *Probabilità e Statistica*, (2007), Esculapio.



STRINGS LIBRARY

Here is the complete library of the predefined queries for communication strings to the *MES4* server.

- **MClass = 10:** Set in MES

MNo	Short	Description
10	PartProduced	Booking a produced part at the resource as good or bad

Table A.1. MClass = 10 queries

- **MClass = 100:** Get Operations

MNo	Short	Description
1	GetOpForRsc	Ask for the next task (operation) for a specific resource
2	GetOpForPallet	Ask for the next task (operation) for a specific pallet
3	GetOpForCarrier	Ask for the next task (operation) for a specific carrier
4	GetFirstOpForRsc	Ask for the first task (operation) for a specific resource
5	GetOpForOrderInBuf	Ask for the next task (operation) for a specific buffer
6	GetOpForONoOPos	Ask for the next task (operation) for a specific order Sent information: - ONo - OPos Received information: - StepNo - ONo - OPos - WPNo - ResourceID - OpNo - MainOPos

		- CNo - PNo
7	GetDefOpForRsc	Ask for a specific task (operation) for a specific resource
8	GetOpPNoForBuf	Ask for a task (operation) for a specific part in the buffer
9	GetOpForFiFo	Ask for the next task (operation) for a part in the FIFO
10	GetDetOpPNoForRsc	Ask for a specific task (operation) for a specific part in a specific resource
11	GetOpInBuf	Ask for the next task for a specific part in a specific resource
12	GetFirstOpPos ForRscBuf	Ask for the first task (operation) for a specific buffer at a specific resource Sent information: - ResourceID - BuffNo Received information: -StepNo -ONo -OPos -WPNo -ResourceID -OpNo -MainOPos -CNo -PNo
13	GetFirstOpForOpNo	Ask for a specific first task (operation) for a specific resource
14	GetFirstOp ForRscBufTF	Ask for the next possible order which has the necessary part in its buffer and must be stored somewhere else
30	OrderInfo	Ask for information about a specific order
31	GetCostumerInfo	Ask about the costumer of a product
32	GetTextForManWP	Ask for a text for the manual workplace
41	GetFinOrder	Ask for an order in the finished orders table
43	GetParSubPosFin	Ask for the parameter of a sub-order
101	PL_GetOrder ToDeliver	Special hard coded message for the prolog that gives an order back which could be delivered

Table A.2. MClass = 100 queries

- **MClass = 101: Set Operations**

MNo	Short	<i>Description</i>
2	SetNewOrder	Create a new order
10	OpStart	Start the operation of a specific task
15	OpReset	Send back the order to the previous step
20	OpEnd	Finish the operation of a specific order
21	EndOPos	End order position

Table A.3. MClass = 101 queries

- **MClass = 110: Get Topology**

MNo	Short	<i>Description</i>
1	GetShuntForTarget	Ask for the correct direction of the shunt module

Table A.4. MClass = 110 queries

- **MClass = 150: Get Buffers and Utilities**

MNo	Short	<i>Description</i>
1	GetBufForBufNo	Ask about all the buffer positions of a buffer
2	GetBufPosForRsc	Ask about the buffer position of a resource
5	GetBufPos	Ask about a specific buffer position
6	GetPNoCntFromBox	Ask about the quantity of a specific part inside a box
10	GetBufPosForONoOPosPNo	Ask for a buffer position where the specific part for the specific order could be found
11	GetBufPosIfStore	Ask for a buffer position if the part needs to be stored
12	GetBufPosPNoNe	Find a pallet in the storage without order
13	GetBufPosForPNo	Ask for a free buffer position in which the specific part may be stored
14	GetBufForONoOPos	Ask for a buffer position where the specific order is
50	GetPalletInfo	Ask information about a pallet
51	GetParSubPos	Ask about the parameters of a sub-position
70	GetBoxPosFromBuf	Ask for the position of a specific part inside the box which is booked in a specific buffer
71	GetBoxPosAll	Ask about all positions of the box
72	GetBoxPosAllRsc	Ask about all the positions of the box and the next resource of an order

Table A.5. MClass = 150 queries

- **MClass = 151: Set Buffers and Utilities**

MNo	Short	<i>Description</i>
1	AddToFiFo	Add a new item to the FIFO
2	DelFifoPos	Gets an item from the FIFO and removes it
5	MoveBuf	Moves the content of a buffer into another buffer
10	SetBufPos	Write into a specific buffer position
12	DelBuf	Delete all positions of a specific buffer
50	SetPallet	Write an order into a specific pallet
60	SetCarrier	Write an order into the specific carrier
70	FillBox	Fill the box with specific parts and clear the order information
71	SetBoxPos	Write a specific part into a specific box position
72	SetBoxONo	Write an order into the box

Table A.6. MClass = 151 queries

B

DATA TYPE ID IN LEVEL-2 MATLAB S-FUNCTION

Here is listed the data types that an OPC UA input or output port in a Level-2 Matlab S-Function can have. No further classes are allowed for a port.

DataTypeID	Meaning
-1	<i>'inherited'</i>
0	<i>'double'</i>
1	<i>'single'</i>
2	<i>'int8'</i>
3	<i>'uint8'</i>
4	<i>'int16'</i>
5	<i>'uint16'</i>
6	<i>'int32'</i>
7	<i>'uint32'</i>
8	<i>'boolean'</i>

Table B.1. DataTypeID for an input/output port in a Level-2 Matlab S-Function



MATLAB SCRIPT

Script for "Solver" Simulink Matlab function block of the Drilling workstation:

```
1 function [OperationDone, ErrorID] = Solver(iErrNo, diONo,
      iOPos, xCL_BG4)
2 coder.extrinsic('tcpip')
3 coder.extrinsic('fopen')
4 coder.extrinsic('sprintf')
5 coder.extrinsic('run')
6 coder.extrinsic('fscanf')
7 coder.extrinsic('textscan')
8 coder.extrinsic('cell2mat')
9 coder.extrinsic('str2double')
10
11 if (iErrNo == 4 && xCL_BG4 == 0)
12     ErrorID = 4;
13     t=tcpip('10.48.134.1',2000);
14     fopen(t);
15     fprintf(t,sprintf('444;RequestID=0;MClass=101;MNo=21;
      ErrorState=1;#ONo=%d;#OPos=%d\r',diONo,iOPos));
16     fclose(t);
17     OperationDone = 1;
18 else
19     ErrorID = 0;
20     OperationDone = 0;
21 end
```

Script for the "Disassembler" Simulink Matlab function block:

```
1 function OperationDone = Disassembler(xResult, diONo, iOPos,  
    iCarrierID)  
2 coder.extrinsic('tcpip')  
3 coder.extrinsic('fopen')  
4 coder.extrinsic('sprintf')  
5 coder.extrinsic('run')  
6 coder.extrinsic('fscanf')  
7 coder.extrinsic('textscan')  
8 coder.extrinsic('cell2mat')  
9 coder.extrinsic('str2double')  
10  
11  
12 persistent i  
13 if isempty(i)  
14     i=0;  
15 end  
16  
17 OperationDone = 0;  
18 if (i == 0)  
19     if (xResult == 0)  
20         t=tcpip('10.48.134.1',2000);  
21         fopen(t);  
22         fprintf(t,sprintf('444;RequestID=0;MClass=101;MNo=21;  
            ErrorState=1;#ONo=%d;#OPos=%d\r',diONo,iOPos));  
23         if (diONo ~= 0)  
24             t1=tcpip('10.48.134.1',2000);  
25             fopen(t1);  
26             fprintf(t1,sprintf('444;RequestID=0;MClass=101;MNo  
                =2;ErrorState=0;#PNo=1231;#Aux1Int=1\r'));  
27         end  
28         OperationDone = 1;  
29     else  
30         OperationDone = 0;  
31     end  
32     i = 1;  
33 end  
34  
35 end
```

ACKNOWLEDGEMENTS

Giunto alla fine di questo lungo percorso formativo, l'occasione si rende propizia per ringraziare tutti coloro che mi hanno permesso, in un modo o nell'altro, di raggiungere questo mio traguardo.

Il mio primo pensiero va a Luca Fumagalli, che ringrazio per avermi permesso di portare a termine il mio percorso di studi qui al Politecnico, dandomi la possibilità di mettermi in gioco con questo stimolante lavoro di tesi.

Ringrazio Elisa Negri, che con i suoi preziosissimi consigli e la sua sempre pronta disponibilità, mi ha accompagnato durante tutto il mio percorso di tesi.

Un pensiero lo dedico a Chiara Cimino e a Roberto Rocca, sempre pronti a darmi una mano ed un parere anche e soprattutto quando ne avevo più bisogno.

Un grazie va anche a tutti i componenti del gruppo di ricerca del DIG, con cui ho condiviso piacevoli momenti al di fuori del laboratorio. Un pensiero va a Walter Quadrini e ad Andrea Caielli, per avermi accolto nell'I4.0 Lab, e soprattutto per la loro capacità di sdrammatizzare, rendendo sereni anche i momenti più densi di lavoro.

Ringrazio particolarmente Alberto Franzini, con cui ho condiviso molto negli ultimi mesi, dalle frustrazioni alle soddisfazioni.

Ringrazio tutti i miei amici; anche se non vi nomino uno per uno sapete chi siete, e sapete cosa fate per me ogni giorno.

Un grazie va a Martina, Simona e Lucrezia, che mi hanno insegnato, mi insegnano, e spero che continueranno ancora ad insegnarmi cosa vuol dire la parola Amicizia.

Ringrazio Marta, che mi ha accolto in una città che non conoscevo, e con cui ho un rapporto di cui vado orgoglioso.

Ringrazio la mia famiglia, uno dei miei capisaldi. In particolare ringrazio mia nonna, Lina, che non ha mai smesso di essere la mia fan numero 1.

Ringrazio mia sorella Francesca: due (+ molti altri) anni di convivenza che mi hanno ricordato quanto sia indispensabile per me.

Infine ringrazio i miei genitori: Marina e Sandro. Per non aver mai smesso per un istante di credere in me, e che mi hanno insegnato ogni cosa.

Ringrazio tutto ciò (persone, cose, musica, ...) che mi abbia mai ispirato e che continua ad ispirarmi ogni giorno della mia vita.